arXiv:2603.18110v2 [quant-ph] 3 Sep 2026

Improved quantum circuits for division

Priyanka Mukhopadhyay [*1], Alexandru Gheorghiu [† 1], and Hari Krovi [‡ 1]

[1]IBM Research, Cambridge MA, USA

Abstract

Arithmetic operations are an important component of many quantum algorithms. Optimizing quantum circuits for these operations therefore leads to more efficient implementations of the corresponding algorithms. In this paper, we develop new fault-tolerant quantum circuits for various integer division algorithms (both reversible and non-reversible). These circuits, when implemented in the Clifford+T gate set, achieve improvements in the asymptotic T-count and CNOT-count from $O(n^2)$ to $O(nm)$, and in the asymptotic T-depth from $O(n^2)$ to $O(n \log m)$, where n and m are the bit lengths of the dividend and divisor, respectively. The qubit counts are significantly lower than those in previous works. We achieve this through careful modifications of the division algorithms and by expressing them in terms of a primitive we call COMP-N-SUB that compares two integers and conditionally subtracts them. We show that this primitive can be implemented at a cost, in terms of both Clifford and non-Clifford gates, comparable to that of *one addition*. By contrast, performing comparison and conditional subtraction separately would cost roughly as much as a *controlled addition* plus a regular addition.

1 Introduction

Much of quantum computing research has been devoted to designing algorithms that can outperform their classical counterparts and, in some cases, solve problems of practical significance. Examples include factoring and computing discrete logarithms [1], unstructured database search [2], Hamiltonian simulation [3, 4, 5, 6], solving algebraic problems [7], and machine learning [8], among others. Many of these algorithms require the computation of classical functions using *algebraic* and *arithmetic* operations [1, 9, 10, 11, 12, 13, 14, 15]. As such, designing efficient quantum circuits and procedures for implementing these arithmetic operations is an important and active area of research. Division, in particular, is a fundamental operation that appears in quantum algorithms for computing shifted quadratic characters [9], solving the principal ideal problem [13], solving the hidden shift problem [10], jet clustering [16], differential evolution [17], the finite element method [18], and objective-function maximization [19]. Division circuits can also be used to implement functions such as greatest common divisors, square roots, reciprocals, fractional powers, natural logarithms, and bilinear interpolation [20, 13, 21, 22, 23], which are used in a number of quantum algorithms [24, 25, 13, 20].

[*]Corresponding author : mukhopadhyay.priyanka@gmail.com, Priyanka.Mukhopadhyay@ibm.com
[†]agheorghiu@ibm.com
[‡]Hari.Krovi@ibm.com

When considering *fault-tolerant* implementations of division circuits, the choice of gate set is important because the error-correcting code may make some operations, such as *Clifford* gates, easier to perform than others, such as non-Clifford gates. While Clifford+T is the most popular gate set, other universal fault-tolerant gate sets include Clifford+Toffoli and Clifford+CS. Clifford gates are easier to implement fault-tolerantly in most codes because they can be performed *transversally*. By contrast, non-Clifford gates such as T and Toffoli require large ancilla factories and additional operations such as *gate teleportation* and *magic state distillation* [26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36]. These requirements not only increase the overhead of fault-tolerant implementations but can also make the operations less accurate and more time-consuming. Hence, much of the research on quantum-circuit optimization is devoted to designing fault-tolerant circuits that *minimize the count and depth of non-Clifford gates*; this is also our goal when constructing division circuits.

The arithmetic circuits developed in the literature can be broadly divided into two groups. The first group is inspired by classical reversible circuits. Examples include the quantum ripple-carry adders in [37, 38, 39, 40], the quantum carry-lookahead adder in [41], the multipliers in [42], and the dividers in [43, 44, 45, 46, 47, 48]. The second group performs phase arithmetic using the quantum Fourier transform (QFT). Examples include the adder and multiplier circuits in [49, 50, 51, 52, 53, 54, 55, 56, 57]. Circuits in the first group are dominated by Toffoli and CNOT gates, whereas those in the second group use many controlled-rotation gates. Toffoli gates can be implemented exactly using the fault-tolerant gate sets mentioned above [58, 30, 59], but rotation gates cannot [60, 61, 62, 63, 64]. In most error-correction schemes, rotation gates must therefore be decomposed further into gates from a universal gate set. The lengths and non-Clifford gate counts of these decompositions increase as the synthesis precision increases, and the decompositions may also require additional ancillas [65, 66]. This makes fault-tolerant implementations of rotation gates very costly in most schemes. For NISQ-type implementations, however, QFT-based arithmetic circuits can be more efficient because they have lower gate counts and use fewer ancillas. Their practicality is nevertheless limited to smaller problem sizes by the noise sensitivity of NISQ devices.

The circuits developed in this paper belong to the first group and avoid expensive rotation gates, making them more amenable to fault-tolerant implementation, the focus of this paper. To the best of our knowledge, no QFT-based divider circuit has previously been constructed.

1.1 Our results

We consider three division algorithms that have been studied extensively in prior work: *long division*, *restoring division* (as given in [46]), and *non-restoring division*. These algorithms divide an integer *dividend* by an integer *divisor* and return an integer *quotient* and an integer *remainder*. Long division is a specific type of restoring algorithm. Suppose there is a superposition of divisors. If all divisors have the same length (the distance from the most significant 1 to the least significant bit), then long division can be used. Otherwise, the other two division algorithms are useful.

Integer division can also be used as a subroutine in some floating-point division algorithms [67, 68]. In [47], quantum circuits for integer division are used to divide a fraction by an integer. Hence, quantum circuits developed for the operations above are useful for both integer and floating-point arithmetic.

We achieve asymptotic reductions in gate complexity for our long-division (LONG-DIV), restoring-division (RES-DIV), and non-restoring-division (NON-RES-DIV) algorithms compared with the previous algorithms in [46, 47, 48]. Two major factors play a crucial role in realizing these improvements. The first is a careful modification of existing algorithms, and the second is the introduction

of a new arithmetic primitive that we call COMP-N-SUB. We briefly summarize our first, algorithmic contribution below.

1. **In-place computation of the quotient:** Unlike previous algorithms, we do not allocate extra space for the quotient. Rather, the qubits that initially store the dividend are used to store the quotient and remainder at the end of the computation. This is possible because, as the number of iterations increases, the more significant bits of the dividend become zero during the residue calculation. This reduces qubit usage.

2. **Operations involving fewer qubits:** In RES-DIV and NON-RES-DIV, we do not prepend zeros to the divisor to make its length equal to that of the dividend. Consequently, each intermediate arithmetic operation, such as addition or subtraction, involves fewer qubits. This also leads to an asymptotic improvement in gate complexity.

3. **Reducing the number of iterations:** In RES-DIV and NON-RES-DIV, we introduce flexibility in the extent of overlap between the dividend and divisor in the first iteration. This reduces the number of iterations in many scenarios.

Central to our divider-circuit designs is the development of new quantum circuits for a unitary that performs the following operation. This is our second major contribution.

$$\mathcal{A}: \quad \text{Given two integers } a \text{ and } b, \text{ compute } b-a \text{ if and only if } b \geq a. \tag{1}$$

In bra–ket notation, the associated unitary acts as follows.

$$|a\rangle\,|b\rangle\,|0\rangle \mapsto \begin{cases} |a\rangle\,|b-a\rangle\,|0\rangle\,, & \text{if } b \geq a \\ |a\rangle\,|b\rangle\,|1\rangle\,, & \text{otherwise,} \end{cases}$$

where the last qubit is a "flag" indicating whether $b \geq a$. We refer to this unitary as COMP-N-SUB (compare and subtract). Similar logical operations appear in many quantum algorithms, so more efficient quantum circuits for COMP-N-SUB directly yield optimizations for those algorithms as well. Thus, the COMP-N-SUB unitary can serve as a fundamental arithmetic primitive for implementing many algorithms. In Section 3, we illustrate how COMP-N-SUB can be used to implement a quantum circuit for Stein's GCD algorithm and discuss the resulting advantage.

We show that COMP-N-SUB can be implemented at roughly the same cost (in terms of T-count, Toffoli-count, CNOT-count, or depth) as one addition of two integers. Previous works have achieved this functionality by first performing a comparison and then a conditional subtraction. The cost is dominated by the conditional subtraction, which is effectively a controlled addition. This operation has much greater overhead than a regular addition because it turns every gate in an addition circuit into a controlled version of itself. Compiling the resulting circuit into a standard gate set then produces a large increase in the number of T or Toffoli gates. Thus, performing comparison and conditional subtraction at a cost comparable to that of a regular addition yields division circuits with fewer non-Clifford gates, fewer qubits, and, in some instances, smaller circuit depth than previous approaches.[1]

Keeping in mind the various trade-offs for an efficient implementation, we have designed multiple circuits for COMP-N-SUB, and these can be broadly classified into three types.

[1]The depth reduction can, in principle, be achieved without COMP-N-SUB; however, this comes at the expense of a significantly larger number of non-Clifford gates. We explain this in more detail at the end of Section 2.2.2.

- **Type-I** circuits are fully reversible and optimize the Toffoli or T count, CNOT count, and qubit count.

- **Type-II** circuits are also fully reversible and optimize the Toffoli or T depth and overall circuit depth at the cost of increased qubit, CNOT, and Toffoli or T counts. Type-II circuits are further divided into two subtypes: Type-IIb, with the smallest known Toffoli or T depth, and Type-IIa, whose metrics are intermediate between those of Type-I and Type-IIb. Specifically, Type-IIa circuits have lower T, CNOT, and qubit counts than Type-IIb circuits but greater T-depth than Type-IIb circuits.

- **Type-III** circuits are not fully reversible and optimize the T and CNOT counts using the logical-AND gadget from [30, 40]. These circuits also have a higher qubit count than Types I and II.

Our LONG-DIV and RES-DIV circuits are constructed with mostly COMP-N-SUB unitaries. Hence, depending upon the type of COMP-N-SUB circuit used, we have named our LONG-DIV and RES-DIV circuits as type I, IIa, IIb and III, respectively. Needless to say, that they inherit the trade-offs of the corresponding COMP-N-SUB circuits.

On the contrary, our NON-RES-DIV circuits are constructed with mostly adders and one COMP-N-SUB unitary. So we name them according to the type of adder used. Again, to reflect the various trade-offs we have considered the following three adders.

- The quantum ripple-carry (QRC) adder in [39], that optimize the Toffoli or T-count, CNOT-count and qubit-count. The NON-RES-DIV circuit using this adder is referred to as type-1.

- The quantum carry-lookahead adder (QCLA) adder in [41], that optimizes Toffoli or T-depth, at the cost of an increased Toffoli or T-count, CNOT-count and qubit-count. The NON-RES-DIV circuit using this adder is referred to as type-2.

- The QRC adder in [40], that used temoporary logical AND gadget, hence reducing T-count, but comes at the cost of an increase in the number of qubits, and extra measurement operations. The NON-RES-DIV circuit using this adder is referred to as type-3.

Tables 1 and 2 summarize and compare important cost metrics for the various circuits developed in this paper. Compared with previous constructions in [46], our restoring (RES-DIV) and non-restoring (NON-RES-DIV) division algorithms achieve asymptotic reductions in T-count, CNOT-count, and T-depth. Suppose n and m are the bit lengths of the dividend and divisor, respectively. The T and CNOT counts of previous circuits scale as $O(n^2)$, whereas those of our circuits scale as $O(nm)$. For $m \in O(n^{1-\delta})$, where $\delta \to 0$, this yields an asymptotic improvement. The T-depth scaling is reduced from $O(n^2)$ to $O(n \log m)$ in RES-DIV circuit-IIb and NON-RES-DIV circuit-2.

Our long-division (LONG-DIV) quantum circuits achieve reductions of up to 76.08% and 68.35%[2] in T-count and CNOT-count, respectively. We also achieve an asymptotic reduction in T-depth for LONG-DIV circuit-IIb, from $O(nm)$ to $O(n \log m)$.

The qubit count of our LONG-DIV circuit-I is the smallest among all existing divider circuits. Because $n + m$ qubits are required to store the input dividend and divisor, our qubit count of $n + m + 3$ may be optimal. In all previous algorithms, the qubit count scales as kn for some

[2]Here, reduction means $\frac{\text{cost of old circuit} - \text{cost of new circuit}}{\text{cost of old circuit}}$.

Algorithm	Circuit	T-count	T-depth	Fully reversible
Long division	Yuan et al. [47]	$126m(n-m+1)$	$54m(n-m+1)$	No
	Orts et al. [48]	$46m(n-m+1)$	$20m(n-m+1)$	No
	LD_I **(This paper)**	$18m(n-m+1)$	$9m(n-m+1)$	Yes
	LD_{IIa} **(This paper)**	$76m(n-m+1)$	$3(m+4\log_2 m)(n-m+1)$	Yes
	LD_{IIb} **(This paper)**	$77m(n-m+1)$	$12(n-m+1)\log_2 m$	Yes
	LD_{III} **(This paper)**	$11m(n-m+1)$	$5m(n-m+1)$	No
Restoring division	Thapliyal et al. [46]	$35n^2+42n$	$15n^2+18n$	Yes
	Res_I **(This paper)**	$18m(n-m'+1)$	$9m(n-m'+1)$	Yes
	Res_{IIa} **(This paper)**	$76m(n-m'+1)$	$3(m+4\log_2 m)(n-m'+1)$	Yes
	Res_{IIb} **(This paper)**	$77m(n-m'+1)$	$12(n-m'+1)\log_2 m$	Yes
	Res_{III} **(This paper)**	$11m(n-m'+1)$	$5m(n-m'+1)$	No
Non-restoring division	Thapliyal et al. [46]	$14n^2+49n$	$6n^2+21n$	Yes
	$NRes_1$ **(This paper)**	$14m(n-m'+1)$	$6m(n-m'+1)$	Yes
	$NRes_2$ **(This paper)**	$70m(n-m'+1)$	$12(n-m'+1)\log_2 m$	Yes
	$NRes_3$ **(This paper)**	$4m(n-m'+1)$	$2m(n-m'+1)$	No

Table 1: Comparison of the T-count, T-depth, and full reversibility of different quantum division circuits. LD_x, Res_x, and $NRes_x$ denote LONG-DIV circuit x, RES-DIV circuit x, and NON-RES-DIV circuit x, respectively. Only leading-order terms are shown. Here, n and m are the numbers of bits in the binary representations of the dividend and divisor, respectively. We assume $n \geq m$, and m' is an integer such that $1 \leq m' \leq m$.

Algorithm	Circuit	Qubit-count	CNOT-count	
			Clifford+Toffoli	Clifford+T
Long division	Yuan et al. [47]	$4n-m$	-	-
	Orts et al. [48]	$3n$	-	$60m(n-m+1)$
	LD_I **(This paper)**	$n+m$	$4m(n-m+1)$	$25m(n-m+1)$
	LD_{IIa} **(This paper)**	$n+3m$	$6m(n-m+1)$	$83m(n-m+1)$
	LD_{IIb} **(This paper)**	$n+4m$	$8m(n-m+1)$	$85m(n-m+1)$
	LD_{III} **(This paper)**	$n+2m$	-	$19m(n-m+1)$
Restoring division	Thapliyal et al. [46]	$3n$	$9n^2+10n$	$44n^2+87n$
	Res_I **(This paper)**	$n+2m-m'$	$4m(n-m'+1)$	$25m(n-m'+1)$
	Res_{IIa} **(This paper)**	$n+4m-m'$	$6m(n-m'+1)$	$83m(n-m'+1)$
	Res_{IIb} **(This paper)**	$n+5m-m'$	$8m(n-m'+1)$	$85m(n-m'+1)$
	Res_{III} **(This paper)**	$n+3m-m'$	-	$19m(n-m'+1)$
Non-restoring division	Thapliyal et al. [46]	$3n$	$5n^2+11n$	$19n^2+53n$
	$NRes_1$ **(This paper)**	$n+2m-m'$	$5m(n-m'+1)$	$19m(n-m'+1)$
	$NRes_2$ **(This paper)**	$n+5m-m'$	$4m(n-m'+1)$	$74m(n-m'+1)$
	$NRes_3$ **(This paper)**	$n+3m-m'$	-	$16m(n-m'+1)$

Table 2: Comparison of the total qubit and CNOT counts of different quantum division circuits. LD_x, Res_x, and $NRes_x$ denote LONG-DIV circuit x, RES-DIV circuit x, and NON-RES-DIV circuit x, respectively. Only leading-order terms are shown. Here, n and m are the numbers of bits in the binary representations of the dividend and divisor, respectively. We assume $n \geq m$, and m' is an integer such that $1 \leq m' \leq m$.

constant $k \geq 2$. For each type of algorithm, we reduce the qubit count relative to the previous best estimate.

In Table 6 (Section 2.4), we compare fault-tolerant resource estimates for implementing the various division algorithms with the surface code, following the procedure in [69]. For $n = 512$ and $m = 256$, LONG-DIV circuit-I requires the fewest physical qubits, about 6.36×10^4, closely followed by NON-RES-DIV circuit-1 with $m' = m$ ($\approx 6.57 \times 10^4$). This represents approximately a twofold

reduction relative to previous circuits. NON-RES-DIV circuit-2 has the shortest execution time, about 0.371 s, closely followed by LONG-DIV circuit-IIb (≈ 0.379 s). These times are significantly shorter than those of previous circuits, which range from about 16 s to 47 s.

1.2 Related work

Several works have developed quantum divider circuits. The circuits in [70, 71, 72, 73, 74] require reversible registers or latches with feedback and are therefore challenging to implement on quantum hardware. Other reversible circuit designs have been proposed in [43, 44, 45]. Fault-tolerant implementations of long division have been given in [47, 48], while [46] develops fault-tolerant circuits for restoring and non-restoring division. Other fault-tolerant quantum divider circuits have been proposed in [67, 68, 75, 76]. As mentioned above, we compare our results with the best prior implementations.

2 Results

2.1 Background

Quantum gates and gadgets: The n-qubit Clifford group, denoted $\mathcal{C}_n$, is generated by the one-qubit Hadamard (H) and phase (S) gates and the two-qubit CNOT gate. The n-qubit Clifford+T group is generated by $\mathcal{C}_n$ and the one-qubit T gate. The n-qubit Clifford+Toffoli group is generated by $\mathcal{C}_n$ and the three-qubit Toffoli gate. These groups contain the unitaries that are *exactly* implementable by the Clifford+T and Clifford+Toffoli gate sets, respectively. Equation 2 gives the unitary matrices of the different Clifford+T gates.

$$\mathrm{X} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}; \qquad \mathrm{H} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}; \qquad \mathrm{T} = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{bmatrix}; \qquad \mathrm{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{2}$$

The Toffoli unitary can be decomposed into a Clifford+T circuit using either seven T gates (Figure 1) [58] or four T gates [30]. The reduction in T-count comes at the cost of additional ancillas and measurements. This construction may not be suitable for some applications because the use of measurements makes the resulting circuit non-reversible.

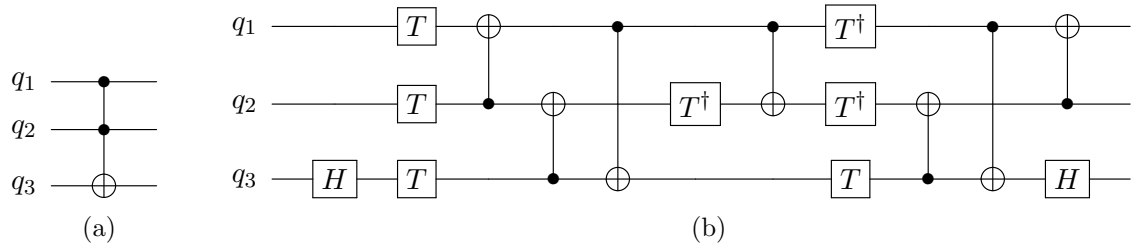


Figure 1: (a) Circuit representation of a Toffoli gate. (b) A Clifford+T circuit for a Toffoli gate.

The ancilla-and-fix-up construction from [30] was recast as the *temporary logical-AND gadget* in [40]. This construction uses four T gates to compute the logical AND of two qubits and place the result in an ancilla register. Figure 2a illustrates this operation symbolically, while Figure 2b

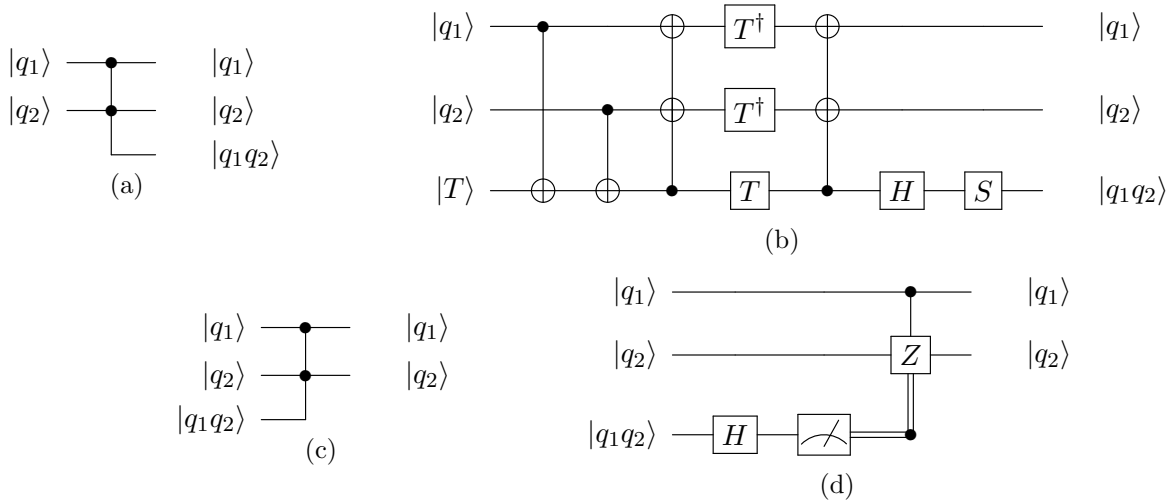


Figure 2: (a) Circuit representation of a temporary logical-AND gate. (b) Clifford+T circuit implementing a temporary logical AND. (c) Circuit representation of logical-AND uncomputation. (d) Circuit implementing logical-AND uncomputation.

shows the explicit circuit. There, $|q_1\rangle$ and $|q_2\rangle$ denote the inputs to the logical AND, while the third qubit is an ancilla initialized to $|0\rangle$ that stores the result $|q_1q_2\rangle$ at the end of the computation. That register can subsequently be "erased" through measurement at *no additional T-gate cost*. This uncomputation is shown symbolically in Figure 2c, and its circuit is given in Figure 2d.

Notation: We use parenthesized subscripts to indicate the qubits on which a gate acts. For example, $\mathrm{X}_{(q)}$ denotes an X gate applied to qubit q. We write $\mathrm{CNOT}_{(q_1;q_2)}$ for a CNOT gate with control qubit q_1 and target qubit q_2. Similarly, $\mathrm{TOF}_{(q_1,q_2;q_3)}$ denotes a Toffoli gate with control qubits q_1 and q_2 and target qubit q_3. This unitary flips the state of q_3 if and only if $|q_1\rangle = |q_2\rangle = |1\rangle$. Our designs also use Toffoli gates with zero-controls. A Toffoli gate with a zero-control on qubit q_1, a one-control on qubit q_2, and target qubit q_3 flips the state of q_3 if and only if $|q_1\rangle = |0\rangle$ and $|q_2\rangle = |1\rangle$. We denote this gate by $\mathrm{TOF}_{(\overline{q_1},q_2;q_3)}$. It can be implemented by conjugating $\mathrm{TOF}_{(q_1,q_2;q_3)}$ with $\mathrm{X}_{(q_1)}$, that is,

$$\mathrm{TOF}_{(\overline{q_1},q_2;q_3)} = \mathrm{X}_{(q_1)} \cdot \mathrm{TOF}_{(q_1,q_2;q_3)} \cdot \mathrm{X}_{(q_1)}. \tag{3}$$

Cost metrics: We list the metrics used to assess and compare the quantum division circuits in our work and previous work and briefly explain why we consider them.

- **T-count:** This is the number of T gates in a quantum circuit. The cost of fault-tolerantly implementing a T gate is higher than that of implementing a Clifford gate in most quantum error-correction schemes. The T-count of a unitary or circuit is denoted by $\mathcal{T}(.)$.
- **Toffoli-count:** This is the number of Toffoli gates in a quantum circuit. A Toffoli gate can be implemented with a Clifford+T circuit (Figure 1b) or by magic-state injection. In either case, a fault-tolerant Toffoli gate is costly to implement. Reducing the Toffoli count also reduces the T-count.

- **CNOT-count:** This is the number of CNOT gates in a quantum circuit. Implementing multiqubit entangling operations is typically error-prone, especially on NISQ or other non-fault-tolerant devices. Recent progress on the surface code [77] suggests that, at least in some error regimes, the cost of a fault-tolerant implementation of the T gate may be comparable to that of the CNOT gate, which is a Clifford gate. It is therefore undesirable to reduce the T-count at the expense of a massive increase in the CNOT count. The CNOT counts of Clifford+Toffoli and Clifford+T circuits are denoted by $\mathcal{N}_{Tof}^{cnot}(.)$ and $\mathcal{N}_{T}^{cnot}(.)$, respectively.

- **Qubit-count:** This is the total number of qubits required to implement a given algorithm. It is an important metric because existing and near-term quantum computers have limited numbers of qubits. We classify qubits as follows.

  - **Input qubits:** These qubits store the input values. We refer to a set of input qubits as an *input register*. For example, in the mapping $U_f \left|x\right\rangle \left|y\right\rangle = \left|x\right\rangle \left|f(x) \oplus y\right\rangle$, where the unitary U_f computes a classical function f, the first register is the input register.
  - **Output qubits:** These are qubits that store information that can be considered as output or useful for further computation. A set of output qubits constitutes an *output register*. In the above example with U_f, the second register is the output register.
  - **Garbage qubits:** After an operation, these qubits are not restored to their initial states, i.e., their states at the beginning of the operation. They also do not store any useful information at the end of the operation.
  - **Ancilla qubits:** These are auxiliary qubits that can help perform certain operations, typically using fewer gates than would be required without ancillas. At the end of the computation, these qubits are restored to their initial states.

- **T-depth and Toffoli-depth:** The T-depth of a circuit is the number of circuit layers (i.e., sets of gates acting in parallel) in which the T/T† gate is the only non-Clifford gate. It indicates the maximum number of T gates that the circuit must execute sequentially on any qubit and hence affects the running time of a quantum circuit. The **Toffoli-depth** is defined analogously. The T-depth of a circuit or unitary is denoted by $\mathcal{T}_d(.)$.

In all our resource estimates, we use the Toffoli construction with a T-count of seven and a T-depth of three (Figure 1(b)). Although other constructions, such as the ancilla-and-fix-up construction in [30], use fewer T gates at the cost of additional ancillas and measurements, we choose to perform the computation coherently and do not consider constructions with intermediate measurements.

2.2 COMP-N-SUB circuits

In this section, we construct efficient quantum circuits for the COMP-N-SUB unitary that implement task $\mathcal{A}$ (Equation 1). We assume that the integers are represented using k bits;[3] if either integer is shorter, we prepend it with zeros. The integers $a = (a_{k-1}, \ldots, a_0)$ and $b = (b_{k-1}, \ldots, b_0)$ are encoded as the quantum states $\left|a\right\rangle = \bigotimes_{i=0}^{k-1} \left|a_i\right\rangle$ and $\left|b\right\rangle = \bigotimes_{i=0}^{k-1} \left|b_i\right\rangle$, respectively. These states are stored in input registers A and B, respectively. We label the qubits, or memory locations, in A

[3]We use k instead of n to avoid confusion when discussing the division algorithms. In those algorithms, which use COMP-N-SUB, m and n denote the bit lengths of the divisor and dividend, respectively.

and B by $A_0, \ldots, A_{k-1}$ and $B_0, \ldots, B_{k-1}$, respectively. The initial states of qubits A_i and B_i are $|a_i\rangle$ and $|b_i\rangle$, respectively.

COMP-N-SUB performs the following operation.

$$|a\rangle\,|b\rangle\,|0\rangle \mapsto \begin{cases} |a\rangle\,|b-a\rangle\,|0\rangle\,, & \text{if } b \geq a \\ |a\rangle\,|b\rangle\,|1\rangle\,, & \text{otherwise} \end{cases}$$

In the difference $b - a$, b and a are called the *minuend* and *subtrahend*, respectively.

Binary subtraction using 1's complement: This is one of the most popular methods for subtracting two binary numbers because it expresses subtraction as addition. The *1's complement* of a binary number is obtained by flipping each of its bits. Denote the 1's complement of a number b by $\overline{b}$. To compute the difference $d = b - a$, we proceed as follows.

1. Compute the 1's complement of the minuend; that is, compute $\overline{b}$.
2. Add a to $\overline{b}$; that is, compute $s = (s_k, \ldots, s_0) = \overline{b} + a = (\overline{b_{k-1}}, \ldots, \overline{b_0}) + (a_{k-1}, \ldots, a_0)$.
3. (a) If there is no carry-over, that is, $s_k = 0$, then the difference is nonnegative and is obtained by taking the 1's complement of s while omitting the carry-over bit. That is, $d = \overline{s}$; equivalently, $(d_{k-1}, \ldots, d_0) = (\overline{s_{k-1}}, \ldots, \overline{s_0})$.

   (b) If there is a carry-over, that is, $s_k = 1$, then the difference is negative. Its magnitude is obtained by adding 1 to s while omitting the carry-over bit. That is, $|d| = (d_{k-1}, \ldots, d_0) = (s_{k-1}, \ldots, s_0) + (0, \ldots, 1)$.

The carry-over bit, s_k, is referred to as the *high bit*. Using the method above, we can perform binary subtraction using an adder. We can also compare two integers using an adder because the high bit indicates whether the difference is positive or negative. We modify existing quantum-adder circuits to combine comparison and conditional subtraction and thereby implement the COMP-N-SUB unitary. As noted above, this gives COMP-N-SUB the same cost, in terms of Clifford and non-Clifford gates, as one addition.

We consider the following three adders for the reasons explained below.

1. *In-place QRC adder of [39]*: This is an optimized version of the QRC adder introduced in [37]. It uses only one extra qubit to store the high bit, the most significant bit of the sum. Its gate count, qubit count, T-count, and depth are $O(k)$, where k is the bit length of each integer being added. It is *in-place* because the output sum is stored in one of the input registers at the end of the computation. The other input register is restored to the state encoding the other input integer. This design saves qubits relative to *out-of-place* designs, in which the sum is stored in a separate register. The circuit implementation is fully reversible.

2. *In-place QCLA adder of [41]*: The main appeal of this adder is that it reduces circuit depth to $O(\log k)$, although the gate, T, and qubit counts each increase by an additive factor of $O(k)$. This quantum circuit is also fully reversible.

3. *Modified QRC adder of [40]*: This variant of the QRC adder is implemented with logical-AND gadgets that reduce the T-count at the cost of $O(k)$ additional ancillas (compared with [39]) and classical measurements, which make the circuit non-reversible.

COMP-N-SUB quantum circuit: In existing designs of both QRC- and QCLA-type adders, the carry bits and partial sums are computed first; the complete sum is then computed using these carry bits. Suppose that the sum of $\overline{b}$ and a is a $(k+1)$-bit integer. That is,

$$s = (s_k, \ldots, s_0) = \overline{b} + a = (\overline{b_{k-1}}, \ldots, \overline{b_0}) + (a_{k-1}, \ldots, a_0).$$

Let c_{i+1} be the carry generated after the ith bitwise addition when computing s_i. Then $c_0 = 0$, and for every $i > 0$, we have

$$\begin{aligned} s_i &= a_i \oplus \overline{b_i} \oplus c_i \\ c_{i+1} &= a_i\overline{b_i} \oplus \overline{b_i}c_i \oplus c_i a_i \end{aligned}$$

Our COMP-N-SUB circuits first compute the high bit, s_k, which encodes the sign of the difference $b - a$ and can therefore be used to compare the two integers. The remainder of the circuit is conditioned on this high bit and either computes the difference or uncomputes the intermediate result. Specifically, $s_k = 0$ implies that $b \geq a$, in which case the remaining part of the circuit computes the difference $b - a = \overline{\overline{b} + a}$. If $s_k = 1$, then $b < a$, and the remaining part of the circuit uncomputes the intermediate result, restoring the input registers to their initial states $|b\rangle$ and $|a\rangle$. Thus, the second part of the circuit performs a subtraction conditioned on the high bit computed in the first part. In this way, we combine comparison and conditional subtraction.

In the next three subsections, we describe the construction of our quantum circuits using the adders in [39] (Section 2.2.1), [41] (Section 2.2.2), and [40] (Section 2.2.3). Our circuits inherit the distinguishing features of these adders, resulting in tradeoffs among the different cost metrics.

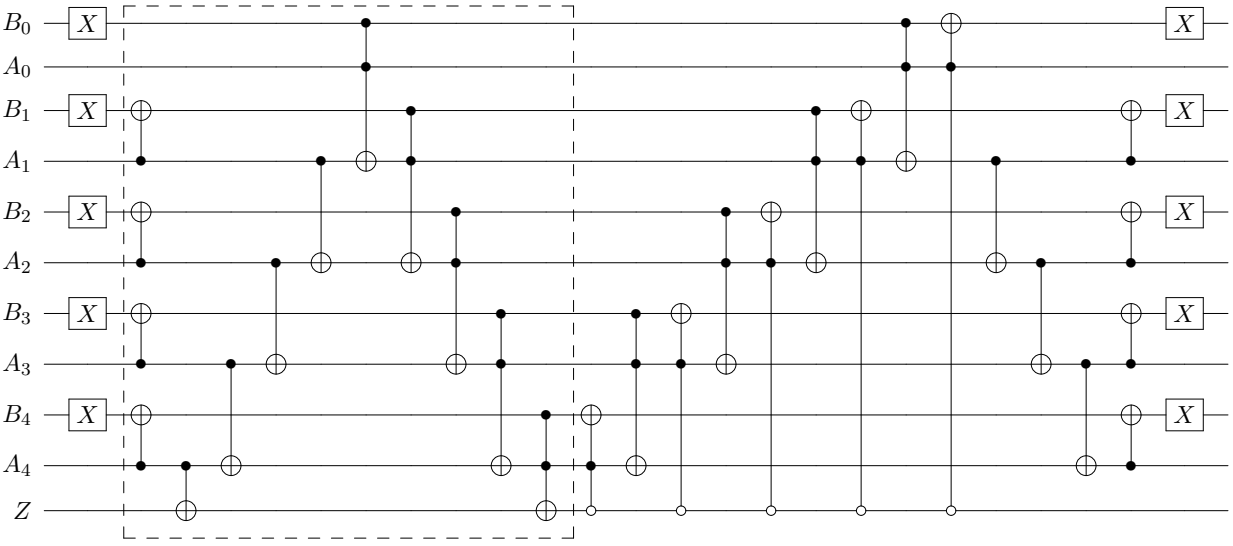


Figure 3: COMP-N-SUB circuit-I for 5-bit integers. Registers B and A are initialized in states $|b\rangle$ and $|a\rangle$, respectively. The first layer of X gates computes $|\overline{b}\rangle$. The boxed part of the circuit computes the high bit in register Z. If $Z = |0\rangle$, the remaining part of the circuit computes $b - a$ in register B; otherwise, both registers are restored to their initial states.

2.2.1 COMP-N-SUB circuit I

In this section, we describe the construction of a COMP-N-SUB circuit using the QRC adder in [39]. We call this COMP-N-SUB circuit-I. Figure 3 shows a circuit for 5-bit integers. In addition to the two input registers, we use a qubit Z initialized to $|0\rangle$. As mentioned earlier, we first compute the high bit in register Z using the circuitry in the dashed box of Figure 3. The high bit contains enough information to compare the input integers a and b. The remaining part of the circuit acts as a conditional subtractor that computes the difference $d = b - a$ in register B if the high bit is $|0\rangle$. Otherwise, input registers A and B are restored to their initial states $|a\rangle$ and $|b\rangle$, respectively. To construct the circuit, we perform the following steps.

1. For $i = 0, \ldots, k-1$: Apply $\mathrm{X}_{(B_i)}$.
2. For $i = 1, \ldots, k-1$: Apply $\mathrm{CNOT}_{(A_i;B_i)}$.
3. (a) Apply $\mathrm{CNOT}_{(A_{k-1};Z)}$.
   (b) For $i = k-2, \ldots, 1$: Apply $\mathrm{CNOT}_{(A_i;A_{i+1})}$.
4. (a) For $i = 0, \ldots, k-2$: Apply $\mathrm{TOF}_{(B_i,A_i;A_{i+1})}$.
   (b) Apply $\mathrm{TOF}_{(B_{k-1},A_{k-1};Z)}$.
5. (a) Apply $\mathrm{TOF}_{(\overline{Z},A_{k-1};B_{k-1})}$.
   (b) For $i = k-2, \ldots, 0$: Apply $\mathrm{TOF}_{(B_i,A_i;A_{i+1})} \cdot \mathrm{TOF}_{(\overline{Z},A_i;B_i)}$.
6. For $i = 1, \ldots, k-2$: Apply $\mathrm{CNOT}_{(A_i;A_{i+1})}$.
7. For $i = 1, \ldots, k-1$: Apply $\mathrm{CNOT}_{(A_i;B_i)}$.
8. For $i = 0, \ldots, k-1$: Apply $\mathrm{X}_{(B_i)}$.

Correctness: To prove that the circuit described above has the intended functionality, we track the changes in the states of the input qubits after each step. The initial state of registers B and A and qubit Z is

$$\left(\bigotimes_{i=0}^{k-1} |b_i\rangle_{B_i} |a_i\rangle_{A_i}\right) |0\rangle_Z .$$

Appendix A.1 shows in detail that the final state after applying the layer of X gates is

$$|\overline{c_k} d_0 \oplus c_k b_0\rangle_{B_0} |a_0\rangle_{A_0} \left(\bigotimes_{i=1}^{k-1} |\overline{c_k} d_i \oplus c_k b_i\rangle_{B_i} |a_i\rangle_{A_i}\right) |c_k\rangle_Z$$

If $c_k = 0$, implying $b \geq a$, we obtain $d = b - a$ in register B; otherwise, we obtain b in register B. Hence, the circuit has the desired COMP-N-SUB functionality.

Resource estimates: We now provide resource estimates for COMP-N-SUB circuit-I.

- **Toffoli-count:** We require $(k-1)+1 = k$ Toffolis in step 4 and $1+2(k-1) = 2k-1$ Toffolis in step 5. Thus, the Toffoli count of our circuit is $3k-1$.
- **CNOT-count:** We require $k-1$ CNOTs in steps 2, 3, and 7 and $k-2$ CNOTs in step 6. Thus, the CNOT count of the Clifford+Toffoli circuit is $3(k-1)+k-2 = 4k-5$. We require at most seven CNOT gates for the Clifford+T implementation of each Toffoli gate. Thus, the CNOT count of the Clifford+T implementation is at most $(4k-5)+7(3k-1) = 25k-12$.
- **T-count:** We first estimate the T-count of the ladder of Toffoli pairs in step 5b. Consider the following four-qubit unitary, whose superscript specifies the qubits on which it acts.

  $$U_1^{q_1,q_2,q_3,q_4} = \mathrm{TOF}_{(q_1,q_2;q_3)} \cdot \mathrm{TOF}_{(q_2,q_3;q_4)}$$

  The T-count of this unitary is 11, as first shown in [78] and later in [79]. Using Equation 3 in step 5b, we see that the ladder of Toffolis is equivalent to

  $$\mathrm{X}_{(Z)} \cdot \left(\prod_{i=k-2}^{0} U_1^{Z,A_i,B_i,A_{i+1}} \right) \cdot \mathrm{X}_{(Z)}.$$

  We have $k+1$ additional Toffoli gates. Thus, the T-count is at most $11(k-1)+7(k+1) = 18k-4$.
- **Qubit-count:** We require $2k$ input qubits and one additional qubit that stores the high bit.
- **Toffoli-depth:** The Toffoli-depth of this circuit is at most $3k-1$.
- **T-depth:** Each Toffoli has T-depth 3; hence, the T-depth of the complete circuit is at most $3(3k-1) = 9k-3$.

2.2.2 COMP-N-SUB circuit II

In this section, we describe the construction of a COMP-N-SUB circuit using the QCLA adder from [41]. We refer to this construction as COMP-N-SUB circuit-II. A k-bit QCLA adder has depth $O(\log k)$, compared with the $O(k)$ depth of a QRC adder. Thus, with this second type of COMP-N-SUB circuit, we aim to reduce the circuit depth.

In addition to the two input registers, we have k auxiliary qubits, indexed by Z_i, that are initialized to $|0\rangle$. During the computation, qubit Z_i stores carry bit c_{i+1}. At the end of the computation, Z_{k-1} stores the high bit, while the other $k-1$ qubits are restored to $|0\rangle$. We also require $k - \omega(k) - \lfloor \log_2 k \rfloor$ additional ancilla qubits initialized to $|0\rangle$. Here, $\omega(k)$ denotes the number of ones in the binary expansion of k. We use the following two unitaries for the reversible computation of the carry bits. The construction of these unitaries is explained in Section 3 of [41]. Here, we briefly describe the main features needed to compute the resource estimates.

- $\mathbf{U_1}$: This unitary computes the carries. Specifically, the input state consists of $|a_i\rangle$ in qubit A_i, $|a_i \oplus \overline{b_i}\rangle$ in qubit B_i, and $|a_i \overline{b_i}\rangle$ in qubit Z_i. After the unitary acts, the state of qubit Z_i is $|c_{i+1}\rangle$. The states of the remaining qubits are unchanged. A quantum circuit for U_1 acting on 5-bit integers is shown in Figure 4b. The circuit has $4k - 3\omega(k) - 3\lfloor \log_2 k \rfloor - 1$ Toffoli gates. The Toffoli depth, which is also the circuit depth, is $\lfloor \log_2 k \rfloor + \lfloor \log_2 \frac{k}{3} \rfloor + 3$.

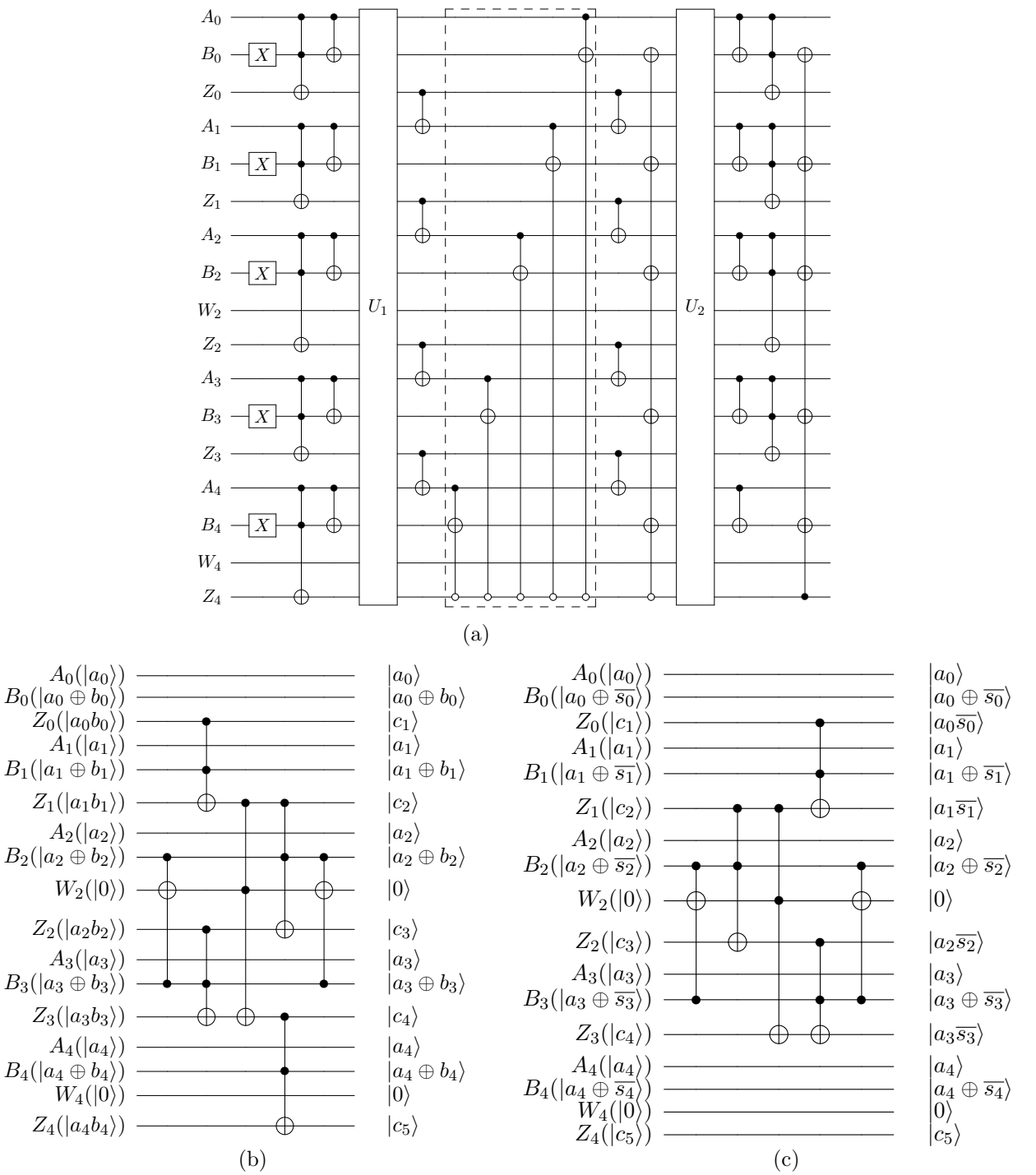


Figure 4: (a) COMP-N-SUB circuit-IIa for 5-bit integers. The first part of the circuit, up to and including unitary U_1, computes the high bit in register Z_4. The remaining circuit computes $b - a$ in register B if $Z_4 = |0\rangle$; otherwise, registers B and A are restored to their initial states $|b\rangle$ and $|a\rangle$, respectively. The Toffoli gates in the boxed part can be parallelized to Toffoli depth 1 by adding extra ancilla qubits, yielding circuit-IIb. (b) Quantum circuit for unitary U_1. (c) Quantum circuit for unitary U_2.

- **$\mathbf{U_2}$**: This unitary uncomputes the carries except for the high bit c_k stored in qubit Z_{k-1}. The circuit is obtained by reversing the circuit of U_1, excluding the Toffoli gates that act on qubit Z_{k-1}. A quantum circuit for U_2 acting on 5-bit integers is shown in Figure 4c. The input state consists of $|a_i\rangle$ in qubit A_i, $|a_i \oplus \overline{s_i}\rangle$ in qubit B_i, and $|c_{i+1}\rangle$ in qubit Z_i. After U_2 acts, the state of qubit Z_i is $|a_i \overline{s_i}\rangle$, except for qubit Z_{k-1}, which remains in state $|c_k\rangle$. The states of the remaining qubits are unchanged. The resource estimates for U_2 are nearly the same as those for U_1.

Figure 4 shows a COMP-N-SUB circuit-II for 5-bit integers. First, the high bit is computed in qubit Z_{k-1}. If $Z_{k-1} = |0\rangle$, the remaining part of the circuit computes the difference in register B; otherwise, the input registers are restored to their initial states. We describe two variants of this circuit: IIa and IIb. The latter is obtained by parallelizing the Toffoli gates that share the same control (the boxed part of the circuit in Figure 4a) using extra CNOT gates and ancilla qubits (Figure 11 in Appendix A.2). We now describe the construction of a quantum circuit for COMP-N-SUB circuit-IIa.

1. For $i = 0, \ldots, k-1$: Apply $\mathrm{X}_{(B_i)}$.
2. For $i = 0, \ldots, k-1$: Apply $\mathrm{TOF}_{(A_i, B_i; Z_i)}$.
3. For $i = 0, \ldots, k-1$: Apply $\mathrm{CNOT}_{(A_i; B_i)}$.
4. Apply U_1.
5. For $i = 1, \ldots, k-1$: Apply $\mathrm{CNOT}_{(Z_{i-1}; A_i)}$.
6. For $i = k-1, \ldots, 0$: Apply $\mathrm{TOF}_{(\overline{Z_{k-1}}, A_i; B_i)}$.
7. For $i = 1, \ldots, k-1$: Apply $\mathrm{CNOT}_{(Z_{i-1}; A_i)}$.
8. For $i = 0, \ldots, k-1$: Apply $\mathrm{CNOT}_{(\overline{Z_{k-1}}; B_i)}$.
9. Apply U_2, excluding the Toffolis that uncompute the high bit.
10. For $i = 0, \ldots, k-1$: Apply $\mathrm{CNOT}_{(A_i; B_i)}$.
11. For $i = 0, \ldots, k-2$: Apply $\mathrm{TOF}_{(A_i, B_i; Z_i)}$.
12. For $i = 0, \ldots, k-1$: Apply $\mathrm{CNOT}_{(Z_{k-1}; B_i)}$.

Correctness: To prove the correctness of our quantum circuit, we track the qubit states at each step described above. We observe that only the states of the input qubits and the ancilla qubits labeled Z_i change before and after the application of unitaries U_1 and U_2. Hence, we write only the states of qubits A_i, B_i, and Z_i. The initial state of these qubits is

$$\bigotimes_{i=0}^{k-1} |a_i\rangle_{A_i} |b_i\rangle_{B_i} |0\rangle_{Z_i} .$$

Appendix A.2 proves in detail that, after the last multi-target CNOT, the state is

$$\left(\bigotimes_{i=0}^{k-2} |a_i\rangle_{A_i} |\overline{c_k} d_i \oplus c_k b_i\rangle_{B_i} |0\rangle_{Z_i}\right) |a_{k-1}\rangle_{A_{k-1}} |\overline{c_k} d_{k-1} \oplus c_k b_{k-1}\rangle_{B_{k-1}} |c_k\rangle_{Z_{k-1}} .$$

Hence, if $c_k = 0$, implying $b \geq a$, we obtain $d = b - a$ in register B; otherwise, we obtain b. Thus, the circuit performs as desired.

Variant: We can parallelize the Toffoli gates in step 6 by using $k-1$ pairs of CNOTs (Figure 11 in Appendix A.2). Using a multi-target CNOT with $k-1$ targets, we make $k-1$ copies of the high bit (the state of qubit Z_{k-1}) in $k-1$ additional ancilla qubits initialized to $|0\rangle$. We can also use these ancilla qubits to parallelize the CNOTs in steps 8 and 12. At the end, we uncompute the ancilla qubits. We refer to this variant as COMP-N-SUB circuit-IIb. In a surface-code implementation, a multi-target CNOT has the same cost and execution time as a single-target CNOT. Thus, a multi-target CNOT can be considered one logical CNOT [69]. A multi-target CNOT is also called a fan-out operator [80]; it can be implemented using only CNOT gates, without ancilla qubits, in depth $O(\log k)$ and size $O(k)$, where k is the number of targets.

Hereafter, COMP-N-SUB circuit-II refers to either variant. We explicitly identify the variant only when necessary.

Resource estimates: We now estimate the cost metrics of interest.

- **Toffoli-count:** We require k Toffoli gates in each of steps 2 and 6 and $k-1$ Toffoli gates in step 11. Additionally, U_1 requires $4k - 3\omega(k) - 3\lfloor \log_2 k \rfloor - 1$ Toffoli gates, and U_2 requires one fewer. Thus, the Toffoli count of our circuit is at most $11k - 6\omega(k) - 6\lfloor \log_2 k \rfloor - 4$.

- **CNOT-count:** We require k CNOT gates in steps 3, 8, 10, and 12 and $k-1$ CNOT gates in steps 5 and 7. Thus, the total number of CNOT gates in a Clifford+Toffoli implementation of circuit-IIa is at most $6k - 2$. Further parallelization increases the CNOT count of COMP-N-SUB circuit-IIb to at most $8k - 4$.

  Since we require at most seven CNOT gates to implement each Toffoli, the CNOT count of a Clifford+T implementation of circuit-IIa is at most $83k - 42\omega(k) - 42\lfloor \log_2 k \rfloor - 30$, while that of circuit-IIb is at most $85k - 42\omega(k) - 42\lfloor \log_2 k \rfloor - 32$.

- **T-count:** Each Toffoli in steps 2 and 11 requires seven T gates. Thus, we require at most $7(2k-1)$ T gates for steps 2 and 11. The number of T gates required to implement U_1 and U_2 is at most $7(8k - 6\omega(k) - 6\lfloor \log_2 k \rfloor - 2)$. In circuit-IIa, the k Toffoli gates in step 6 share a control (taking into account Equation 3). Appendix C shows that the T-count of a unitary comprising a pair of Toffoli gates that share a control is at most 12. Thus, the number of T gates required in step 6 is at most $6k$ if k is even and $6(k-1)+7 = 6k+1$ if k is odd. Hence, the T-count of circuit-IIa is at most $76k - 42\omega(k) - 42\lfloor \log_2 k \rfloor - 21$.

  In circuit-IIb, the Toffoli gates in step 6 do not share a control and therefore require at most $7k$ T gates. Hence, the total T-count of circuit-IIb is at most $77k - 42\omega(k) - 42\lfloor \log_2 k \rfloor - 21$.

  Several Toffoli gates in the implementations of U_1 and U_2 share a control or target. Hence, the T-count estimates above may be improved.

- **Qubit-count:** We require $3k + k - \omega(k) - \lfloor \log_2 k \rfloor = 4k - \omega(k) - \lfloor \log_2 k \rfloor$ qubits for circuit-IIa. Of these, $2k$ are input qubits, and one qubit stores the high bit after computation. The remaining $2k - \omega(k) - \lfloor \log_2 k \rfloor - 1$ ancilla qubits are restored to their initial $|0\rangle$ states after computation. To parallelize the Toffoli gates in step 6 of circuit-IIb, we require $k - 1$ more ancilla qubits, bringing the total qubit count to $5k - \omega(k) - \lfloor \log_2 k \rfloor - 1$. The extra ancilla qubits are restored to their initial states.

- **Toffoli-depth:** The Toffoli depth in steps 2 and 11 is 1. The Toffoli depth of each of U_1 and U_2 is $\lfloor \log_2 k \rfloor + \lfloor \log_2 \frac{k}{3} \rfloor + 3$. The Toffoli depths in step 6 for circuits-IIa and IIb are k and 1, respectively. Thus, the Toffoli depths are at most $k + 2\lfloor \log_2 k \rfloor + 2\lfloor \log_2 \frac{k}{3} \rfloor + 8$ and $2\lfloor \log_2 k \rfloor + 2\lfloor \log_2 \frac{k}{3} \rfloor + 9$ for circuits-IIa and IIb, respectively.

- **T-depth:** Each Toffoli has T-depth 3. Hence, the T-depths of circuits-IIa and IIb are at most $3k + 6\lfloor \log_2 k \rfloor + 6\lfloor \log_2 \frac{k}{3} \rfloor + 24$ and $6\lfloor \log_2 k \rfloor + 6\lfloor \log_2 \frac{k}{3} \rfloor + 27$, respectively.

The QCLA-based COMP-N-SUB circuit presented here will be used to construct division algorithms with depth $O(n \log n)$, where n is the bit length of the dividend. This contrasts with previous work in which division has depth $O(n^2)$. However, it is important to note that this depth reduction does not stem specifically from using COMP-N-SUB. In fact, the standard approach of implementing division as a sequence of $O(n)$ comparisons and conditional subtractions can also achieve depth $O(n \log n)$. This can be accomplished by performing the comparisons in $O(\log n)$ depth and using controlled QCLA adders for the conditional subtractions, which also have $O(\log n)$ depth. The problem with this approach is that a controlled QCLA adder has many more non-Clifford gates than a regular QCLA adder. Performing this controlled adder $O(n)$ times leads to a massive blowup in the number of non-Clifford gates in the division circuit. This may explain why division circuits with depth $O(n \log n)$ have not been discussed in previous work. We avoid this issue by using COMP-N-SUB, thereby keeping the non-Clifford gate count comparable to that of performing $O(n)$ regular QCLA additions.

2.2.3 COMP-N-SUB circuit III

In this section, we describe the construction of a third type of COMP-N-SUB circuit using the modified QRC adder in [40], which reduces the T-count through the use of logical-AND gadgets. In addition to the two input registers, COMP-N-SUB circuit-III uses k auxiliary qubits initialized to $|0\rangle$ and labeled Z_i. The carry bits are computed in these qubits. Figure 5 illustrates COMP-N-SUB circuit-III for 5-bit integers. The boxed part of the circuit acts as a partial comparator, computing the high bit in qubit Z_{k-1}. The unboxed part acts as a conditional subtractor, computing the difference in register B if $Z_{k-1} = |0\rangle$; otherwise, the input registers are restored to their initial states.

We now describe how to construct a quantum circuit for this unitary. We denote the three-qubit unitary that computes the logical AND of the states of qubits q_1 and q_2 by $U_{AND(q_1,q_2;q_3)}$, where q_3 stores the AND result (Figures 2a and 2b). We denote the circuit that uncomputes the logical AND by $\mathcal{C}_{(q_1,q_2;q_3)}$ (Figures 2c and 2d).

1. For $i = 0, \ldots, k-1$: Apply $\mathrm{X}_{(B_i)}$.

2. Apply $U_{AND(B_0,A_0;Z_0)}$.

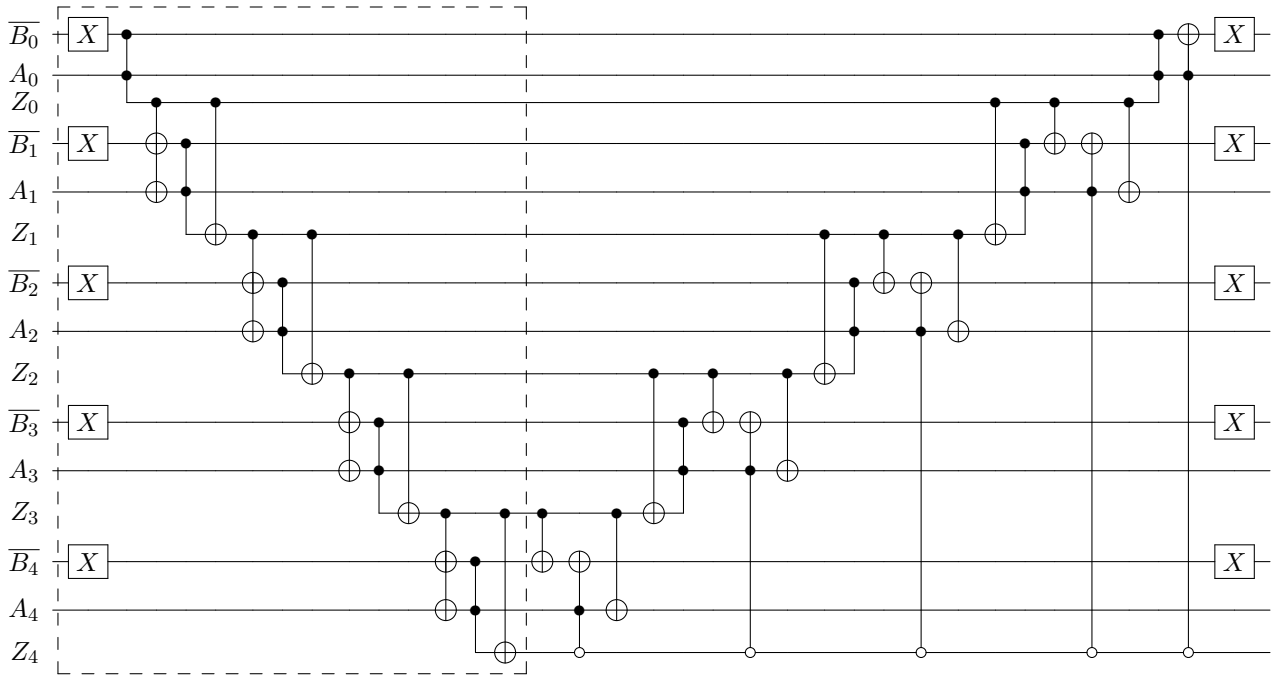


Figure 5: COMP-N-SUB circuit-III for 5-bit integers. The boxed part of the circuit computes the high bit in qubit Z_4. If $Z_4 = |0\rangle$, the remaining part of the circuit computes the difference $b - a$ in register B; otherwise, registers B and A are restored to their initial states $|b\rangle$ and $|a\rangle$, respectively.

3. For $i = 1, \ldots, k-1$:
   (a) Apply $\text{CNOT}_{(Z_{i-1};A_i)} \cdot \text{CNOT}_{(Z_{i-1};B_i)}$.
   (b) Apply $U_{AND(B_i,A_i;Z_i)}$.
   (c) Apply $\text{CNOT}_{(Z_{i-1};Z_i)}$.
4. Apply $\text{CNOT}_{(Z_{k-2};B_{k-1})} \cdot \text{TOF}_{(\overline{Z_{k-1}},A_{k-1};B_{k-1})} \cdot \text{CNOT}_{(Z_{k-2};A_{k-1})}$.
5. For $i = k-2, \ldots, 1$:
   (a) Apply $\text{CNOT}_{(Z_{i-1};Z_i)}$.
   (b) Apply $\mathcal{C}_{(B_i,A_i;Z_i)}$.
   (c) Apply $\text{CNOT}_{(Z_{i-1};B_i)} \cdot \text{TOF}_{(\overline{Z_{k-1}},A_i;B_i)} \cdot \text{CNOT}_{(Z_{i-1};A_i)}$.
6. (a) Apply $\mathcal{C}_{(B_0,A_0;Z_0)}$.
   (b) Apply $\text{TOF}_{(\overline{Z_{k-1}},A_0;B_0)}$.
7. For $i = 0, \ldots, k-1$: Apply $\text{X}_{(B_i)}$.

Correctness: The initial state of the qubits is as follows.

$$\bigotimes_{i=0}^{k-1} |b_i\rangle_{B_i} |a_i\rangle_{A_i} |0\rangle_{Z_i}.$$

Appendix A.3 shows in detail that the final state after applying the X gates is

$$|\overline{c_k s_0} \oplus c_k b_0\rangle_{B_0} |a_0\rangle_{A_0} |0\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-1} |\overline{c_k s_i} \oplus c_k b_i\rangle_{B_i} |a_i\rangle_{A_i} |0\rangle_{Z_i} \right).$$

Hence, if $b \geq a$, that is, $c_k = 0$, we obtain $d = b - a$ in register B; otherwise, we obtain b, as desired.

Resource estimates: The resource requirements for this circuit are as follows.

- **Toffoli-count:** We require k Toffoli gates in steps 4, 5, and 6.
- **CNOT-count:** We require $3(k-1)$, 2, and $3(k-2)$ CNOT gates in steps 3, 4, and 5, respectively. Each U_{AND} and each Toffoli require six and seven CNOT gates, respectively. We require a total of k applications of U_{AND} in steps 2 and 3 and a total of k Toffoli gates in steps 4, 5, and 6. Hence, the total number of CNOT gates required is at most $(3k-3) + 2 + (3k-6) + 6k + 7k = 19k - 7$.
- **T-count:** The T-counts of each U_{AND} and each Toffoli are 4 and 7, respectively. Thus, we require at most $4k + 7k = 11k$ T gates.
- **Qubit-count:** We require $2k$ input qubits, one qubit that stores the high bit, and $k-1$ ancilla qubits that are restored to their initial states. Hence, the total qubit count is $3k$.
- **Toffoli-depth:** The Toffoli depth is at most k.
- **T-depth:** Each U_{AND} has T-depth 2, while each Toffoli has T-depth 3. Hence, the T-depth of our circuit is at most $2k + 3k = 5k$.
- **Measurements:** We require k measurements to uncompute the logical-AND gadgets, unlike COMP-N-SUB circuits I and II, which require no measurements.

		COMP-N-SUB circuit			
		I	**IIa**	**IIb**	**III**
Toffoli-count		$3k-1$	$11k - 6\omega(k) - 6\lfloor \log_2 k \rfloor - 4$	$11k - 6\omega(k) - 6\lfloor \log_2 k \rfloor - 4$	k
Toffoli-depth		$3k-1$	$k + 2\lfloor \log_2 k \rfloor + 2\lfloor \log_2 \frac{k}{3} \rfloor + 8$	$2\lfloor \log_2 k \rfloor + 2\lfloor \log_2 \frac{k}{3} \rfloor + 9$	k
T-count		$18k-4$	$76k - 42\omega(k) - 42\lfloor \log_2 k \rfloor - 21$	$77k - 42\omega(k) - 42\lfloor \log_2 k \rfloor - 21$	$11k$
T-depth		$9k-3$	$3k + 6\lfloor \log_2 k \rfloor + 6\lfloor \log_2 \frac{k}{3} \rfloor + 24$	$6\lfloor \log_2 k \rfloor + 6\lfloor \log_2 \frac{k}{3} \rfloor + 27$	$5k$
Qubit-count	Total	$2k+1$	$4k - \omega(k) - \lfloor \log_2 k \rfloor$	$5k - \omega(k) - \lfloor \log_2 k \rfloor - 1$	$3k$
	Ancilla	0	$2k - \omega(k) - \lfloor \log_2 k \rfloor - 1$	$3k - \omega(k) - \lfloor \log_2 k \rfloor - 2$	$k-1$
	Garbage	1	1	1	1
CNOT-count	Clifford+Toffoli	$4k-5$	$6k-2$	$8k-4$	-
	Clifford+T	$25k-12$	$83k - 42\omega(k) - 42\lfloor \log_2 k \rfloor - 30$	$85k - 42\omega(k) - 42\lfloor \log_2 k \rfloor - 32$	$19k-7$
Fully reversible		Yes	Yes	Yes	No

Table 3: Resource requirements of different quantum circuits for the COMP-N-SUB unitary, which compares and conditionally subtracts k-bit integers.

Summary: Table 3 summarizes the resource requirements of the COMP-N-SUB circuits derived above. Circuit-I uses no extra ancilla qubits and has a T-count about 76% lower than that of circuit-II. The T-count of circuit-III is 38% lower than that of circuit-I. However, circuit-III uses $k-1$ extra ancilla qubits and is not fully reversible, which may limit its applicability. All the other circuits are fully reversible.

The T-depth of circuit-IIb is exponentially lower than those of the other circuits. We observe that all the circuits are dominated by Toffoli gates and that their overall circuit depths are similar to their T-depths. Thus, the overall circuit depth of circuit-IIb is also exponentially lower than those of the other circuits. However, its CNOT count is about 3.4 and 4.5 times those of circuits-I and III, respectively, and its T-count is about 4.3 and 7 times those of circuits-I and III, respectively. It also uses about three times as many ancilla qubits as circuit-III. Nevertheless, for applications in which the circuit depth or T-depth must be limited, circuit-IIb may be the best option. Circuit-IIa generally offers an intermediate trade-off between circuits I and III, which favor lower counts, and circuit-IIb, which favors lower depth.

Remark 2.1. In this paper, we have considered three popular quantum adders to construct quantum circuits for the COMP-N-SUB unitary. Our aim has been to establish a methodology for designing such circuits. Other adder constructions may be considered to develop new and/or more efficient quantum circuits [81, 82, 83, 84, 80].

Remark 2.2. Section 2.1 describes two popular implementations of the Toffoli gate [58, 30] and briefly highlights their advantages and disadvantages. In all our resource estimates, we have exclusively used the circuit in Figure 1(b). However, depending on the design criteria and application, one may instead consider the Toffoli implementation in [30]. This reduces the T-count and T-depth. For example, for COMP-N-SUB circuit-III (Table 3), the T-count decreases from $11k$ to $8k$, and the T-depth decreases from $5k$ to $4k$. However, the ancilla count increases from $k-1$ to $2k-2$. The number of measurements also increases, which may slow the computation. These tradeoffs should be considered when selecting the best circuit implementation.

2.3 Quantum circuit for division

In this section, we construct quantum circuits for integer division. There are two main groups of integer division algorithms: (1) restoring and (2) non-restoring. Algorithms in both groups consist of a number of iterations. In each iteration, the divisor is subtracted from either part of the dividend or a residue to which a bit of the dividend has been appended. This residue is obtained from the subtraction performed in the previous iteration. In restoring algorithms, if the difference is negative, then the minuend (from which the divisor is subtracted) becomes the residue; this is the restoration step. In non-restoring algorithms, this restoration does not occur. Instead, subsequent operations are determined by the sign of the difference, and a final correction may be applied.

In this paper, we describe two restoring division algorithms, (1) LONG-DIV and (2) RES-DIV, and one non-restoring algorithm, NON-RES-DIV. Compared with previous work on long division [47, 48] and on restoring and non-restoring division [46], we introduce a number of algorithmic changes, which we discuss in the following sections. For each algorithm, we show how COMP-N-SUB can be used to design new quantum circuits. Both the algorithmic changes and the use of COMP-N-SUB result in significant reductions in gate counts, depth, and qubit count relative to previous constructions.

Suppose we want to divide an n-bit integer N (the dividend) by an m-bit integer D (the divisor).

We store these integers in two quantum input registers, each containing at most n qubits. Any register that stores the quotient or remainder is an output register. Any other qubit is an ancilla if it is restored to its initial state; otherwise, it is a garbage qubit. This convention differs from that of previous papers such as [48], in which the authors include the qubits storing the quotient among the garbage qubits.

Notation: We use $A_{[k:0]}$ to denote the sequence of qubits or bits $(A_k, \ldots, A_0)$. For two strings $A = (A_a, \ldots, A_0)$ and $B = (B_b, \ldots, B_0)$, $(A : B)$ denotes the concatenated string $(A_a, \ldots, A_0, B_b, \ldots, B_0)$. COMP-N-SUB$^{(k)}(\mathcal{B}, \mathcal{A}, \mathcal{K})$ denotes a COMP-N-SUB unitary that compares and subtracts k-bit integers stored in the k-qubit registers $\mathcal{B}$ and $\mathcal{A}$. The qubit $\mathcal{K}$ stores the high bit. At the end of the computation, either the registers are unchanged and the high bit is $|1\rangle$, or the difference is stored in $\mathcal{B}$, $\mathcal{A}$ is unchanged, and $\mathcal{K}$ is $|0\rangle$.

2.3.1 LONG-DIV: A long division algorithm

Algorithm 1: LONG-DIV

Input: (i) An n-bit dividend $N = (N_{n-1}, \ldots, N_0)$ and (ii) an m-bit divisor $D = (D_{m-1}, \ldots, D_0)$, with $n \geq m$.
Output: (i) Quotient Q and (ii) remainder R.

```
B ← (0 : 0 : N) ;
A ← (0 : D) ;
if B_[n−1:n−m] ≥ A_[m−1:0] then
    B ← (0 : 0 : (B_[n−1:n−m] − A_[m−1:0]) : B_[n−m−1:0]) ;
    B_{n+1} ← 1 ;
end
for i = 1, ..., n − m do
    if B_[n−i:n−m−i] ≥ A then
        B ← (B_[n+1:n−i+1] : (B_[n−i:n−m−i] − A) : B_[n−m−i−1:0]) ;
        B_{n+1−i} ← 1 ;
    end
end
return B_[n+1:m+1] and B_[m−1:0] as the quotient and remainder, respectively. ;
```

Long division is a textbook algorithm with numerous applications. It is a restoring division algorithm consisting of $n - m + 1$ iterations. In each iteration, we subtract the divisor from a minuend. If the difference is negative, the minuend is retained as the residue and the corresponding quotient bit is 0; otherwise, the difference becomes the residue and the quotient bit is 1. In the first iteration, the minuend consists of the m most significant bits of the dividend. Beginning with the next iteration, the minuend is formed by appending the next unprocessed bit of the dividend to the residue from the previous iteration.

The computation always starts with the most significant bit of the dividend. In binary long division, the dividend segment must be less than twice the divisor, and the residue at each iteration must be less than the divisor. Thus, the subtraction in the first iteration is between m-bit integers, whereas beginning with the second iteration it is between $(m+1)$-bit integers. We therefore prepend a 0 to the divisor in order to subtract binary numbers of equal length.

Dividing an n-bit dividend N by an m-bit divisor D results in an $(n - m + 1)$-bit quotient Q and an m-bit remainder R. In previous papers [46, 47, 48], separate qubits are allocated to compute the quotient, while the remainder is computed in place of the dividend. In our LONG-DIV algorithm, the quotient is also computed in place. This substantially reduces the qubit requirement, as explained in more detail below.

In-place computation of the quotient and remainder: We do not allocate any extra space for the quotient. Instead, some of the qubits that store the dividend at the beginning of the computation store part of the quotient at the end; the remaining qubits store the remainder. This is possible because a portion of the dividend changes in each iteration. In the first iteration, we either replace $N_{[n-1:n-m]}$ by $N_{[n-1:n-m]} - D$ or leave it unchanged. Denoting this m-bit residue by $d^{(1)}$, the dividend string becomes $N \leftarrow (d^{(1)}_{[m-1:0]} : N_{[n-m-1:0]})$, and we store Q_{n-m} in an extra bit. In the second iteration, we compute $N_{[n-1:n-m-1]} - (0 : D)$, replace the first $m+1$ bits of N by the residue $d^{(2)}$, and store Q_{n-m-1} in an extra bit. After this operation, bit N_{n-1} must be 0 because the residue is less than $(0 : D)$. In the third iteration, we can therefore use this bit to store Q_{n-m-2}. By similar reasoning, bit N_{n-2} must be 0 after the third iteration and can store the quotient bit in the next iteration. The same argument applies to every iteration, allowing us to store part of the quotient in place of a portion of the dividend.

Pseudocode for LONG-DIV is shown in Algorithm 1. Although we assume that $n \geq m$, the procedure also works when $n < m$. In that case, we prepend zeros to the dividend to make its length equal to m. We now describe a quantum circuit for LONG-DIV using COMP-N-SUB unitaries. It has two registers. The $(m+1)$-qubit register $A = (A_m, \ldots, A_0)$ stores the divisor D prepended with one zero. The $(n+2)$-qubit register $B = (B_{n+1}, \ldots, B_0)$ stores the dividend N prepended with two zeros. At the end of the computation, qubits $B_{[n+1:m+1]}$ store the quotient, qubits $B_{[m-1:0]}$ store the remainder, and $B_m = 0$. The following steps describe the construction of the quantum circuit based on Algorithm 1.

1. Apply COMP-N-SUB$^{(m)}(B_{[n-1:n-m]}, A_{[m-1:0]}, B_{n+1})$.
2. For $i = 1, \ldots, n-m$:
   (a) Apply COMP-N-SUB$^{(m+1)}(B_{[n-i:n-m-i]}, A, B_{n+1-i})$.
3. For $i = n+1, \ldots, m+1$: Apply $\mathrm{X}_{(B_i)}$.

First, we implement steps 3–6 of LONG-DIV (Algorithm 1). After applying the COMP-N-SUB$^{(m)}$ unitary, the residue is stored in $B_{[n-1:n-m]}$, and B_{n+1} stores the high bit. Specifically, $B_{n+1} = |1\rangle$ if the dividend segment is less than the divisor; otherwise, it is $|0\rangle$. Register A remains unchanged.

Next, we implement steps 7–12 by applying a COMP-N-SUB unitary to $(m+1)$-qubit integer segments. Specifically, in iteration i, it acts on qubits $B_{n-i}, \ldots, B_{n-m-i}$ of register B and on $A = |0\rangle \otimes |D\rangle$, which remains unchanged. The qubit B_{n+1-i} is $|1\rangle$ if the dividend segment is less than A; otherwise, it is $|0\rangle$.

After all iterations, applying X gates to qubits $B_{n+1}, \ldots, B_{m+1}$ yields the quotient. The remainder is obtained from qubits $(B_{m-1}, \ldots, B_0)$. Figure 6a shows a LONG-DIV quantum circuit for dividing a 3-bit dividend by a 2-bit divisor.

Resource estimates: A LONG-DIV quantum circuit for dividing an n-bit dividend by an m-bit divisor consists of $(n-m+1)$ COMP-N-SUB unitaries. The first operates on m-bit integers (COMP-N-SUB$^{(m)}$), and the remaining $n-m$ operate on $(m+1)$-bit integers (COMP-N-SUB$^{(m+1)}$). Depending on the type of COMP-N-SUB circuit used, we refer to these as LONG-DIV circuits I, IIa, IIb, and III. The resulting bounds on the different cost metrics are given below. The resource requirements of COMP-N-SUB$^{(k)}$ for arbitrary k can be obtained from Table 3.

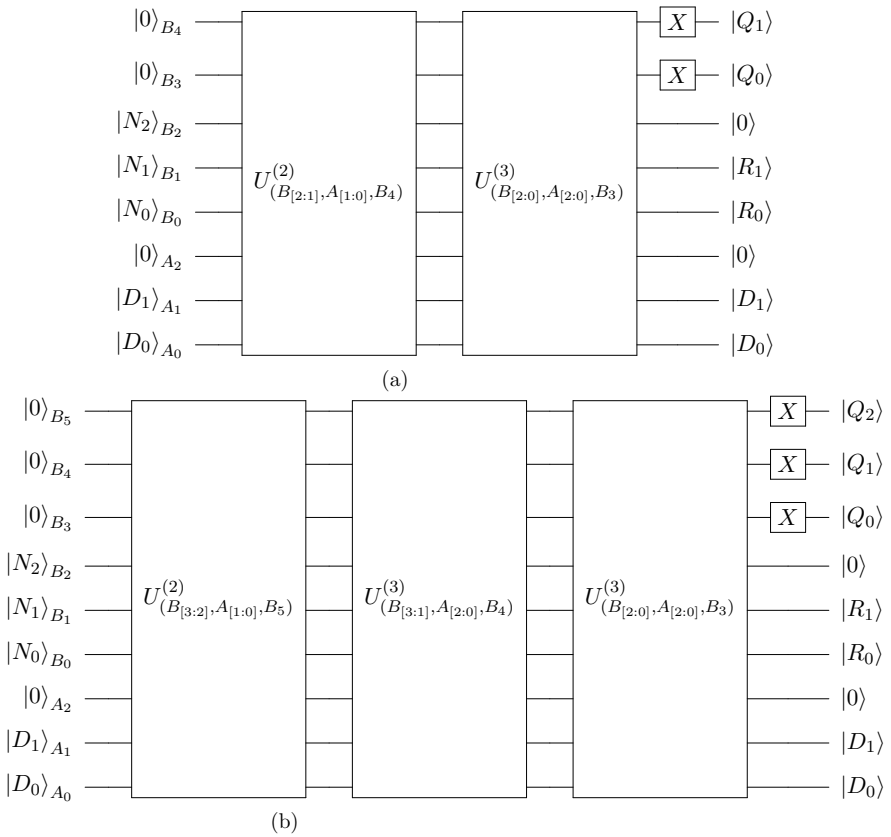


Figure 6: Quantum circuits for (a) LONG-DIV and (b) RES-DIV. The dividend $N = (N_2, N_1, N_0)$ and the divisor $D = (D_1, D_0)$ are 3-bit and 2-bit integers, respectively. The remainder is $R = (R_1, R_0)$. The quotient is $Q = (Q_1, Q_0)$ for (a) and $Q = (Q_2, Q_1, Q_0) = (0, Q_1, Q_0)$ for (b). For (b), $m' = 1$. The unitary $U^{(k)}$ is a COMP-N-SUB unitary that compares and subtracts k-bit integers; its subscript identifies the qubits on which it acts.

- **Toffoli-count:** Using COMP-N-SUB circuit I, the Toffoli count for LONG-DIV circuit I is

$$(3m-1) + (n-m)(3(m+1)-1) = 3nm - 3m^2 + 2n + m - 1.$$

For each of LONG-DIV circuits IIa and IIb, the Toffoli count is at most

$$\begin{aligned} & (11m - 6\omega(m) - 6\lfloor \log_2 m \rfloor - 4) + (n-m)(11(m+1) - 6\omega(m+1) - 6\lfloor \log_2(m+1) \rfloor - 4) \\ = \;& 11nm - 11m^2 + 7n + 4m - 4 - 6\big(\omega(m) + (n-m)\omega(m+1) + \lfloor \log_2 m \rfloor + (n-m)\lfloor \log_2(m+1) \rfloor\big) \\ \leq \;& 11nm - 11m^2 + 7n + 4m - 4. \end{aligned}$$

Using COMP-N-SUB circuit III, the Toffoli count for LONG-DIV circuit III is at most

$$m + (n-m)(m+1) = nm - m^2 + n.$$

- **CNOT-count:** The CNOT counts of Clifford+Toffoli implementations of the LONG-DIV

		RES-DIV circuit			
		I	**IIa**	**IIb**	**III**
Toffoli-count		$3m(n-m'+1)$	$11m(n-m'+1)$	$11m(n-m'+1)$	$m(n-m'+1)$
Toffoli-depth		$3m(n-m'+1)$	$(m+4\log_2 m)(n-m'+1)$	$4(n-m'+1)\log_2 m$	$m(n-m'+1)$
T-count		$18m(n-m'+1)$	$76m(n-m'+1)$	$77m(n-m'+1)$	$11m(n-m'+1)$
T-depth		$9m(n-m'+1)$	$3(m+4\log_2 m)(n-m'+1)$	$12(n-m'+1)\log_2 m$	$5m(n-m'+1)$
Qubit-count	Total	$n+2m-m'$	$n+4m-m'$	$n+5m-m'$	$n+3m-m'$
	Ancilla	2	$2m$	$3m$	m
	Garbage	0	0	0	0
CNOT-count	Clifford+Toffoli	$4m(n-m'+1)$	$6m(n-m'+1)$	$8m(n-m'+1)$	-
	Clifford+T	$25m(n-m'+1)$	$83m(n-m'+1)$	$85m(n-m'+1)$	$19m(n-m'+1)$
Fully reversible		Yes	Yes	Yes	No

Table 4: Resource requirements of quantum circuits for RES-DIV. Here, n and m are the numbers of bits in the binary representations of the dividend and divisor, respectively. The parameter m' is an integer such that $1 \leq m' \leq m$. When $m' = m$, we obtain the resource requirements of LONG-DIV. Only leading-order terms are shown.

circuits are as follows.

$$\begin{aligned}
\text{Circuit-I:} \quad & \mathcal{N}_{Tof}^{cnot}(LD_I) = (4m-5) + (n-m)(4(m+1)-5) \\
& = 4nm - 4m^2 - n + 5m - 5 \qquad (4) \\
\text{Circuit-IIa:} \quad & \mathcal{N}_{Tof}^{cnot}(LD_{IIa}) = (6m-2) + (n-m)(6(m+1)-2) \\
& = 6nm - 6m^2 + 4n + 2m - 2 \qquad (5) \\
\text{Circuit-IIb:} \quad & \mathcal{N}_{Tof}^{cnot}(LD_{IIb}) = (8m-4) + (n-m)(8(m+1)-4) \\
& = 8nm - 8m^2 + 4n + 4m - 4 \qquad (6)
\end{aligned}$$

The CNOT counts for Clifford+T implementations of the various LONG-DIV circuits are as follows.

$$\begin{aligned}
\text{Circuit-I:} \quad & \mathcal{N}_{T}^{cnot}(LD_I) = (25m-12) + (n-m)(25(m+1)-12) \\
& = 25nm - 25m^2 + 13n + 12m - 12 \qquad (7) \\
\text{Circuit-IIa:} \quad & \mathcal{N}_{T}^{cnot}(LD_{IIa}) = (83m-30) + (n-m)(83(m+1)-30) \\
& = 83nm - 83m^2 + 53n + 30m - 30 \qquad (8) \\
\text{Circuit-IIb:} \quad & \mathcal{N}_{T}^{cnot}(LD_{IIb}) = (85m-32) + (n-m)(85(m+1)-32) \\
& = 85nm - 85m^2 + 53n + 32m - 32 \qquad (9) \\
\text{Circuit-III:} \quad & \mathcal{N}_{T}^{cnot}(LD_{III}) = (19m-7) + (n-m)(19(m+1)-7) \\
& = 19nm - 19m^2 + 12n + 7m - 7 \qquad (10)
\end{aligned}$$

- **T-count:** The numbers of T gates required to implement the various LONG-DIV circuits

are as follows.

$$\begin{aligned}
\text{Circuit-I:} \quad & \mathcal{T}(LD_I) = (18m-4) + (n-m)(18(m+1)-4) \\
& = 18nm - 18m^2 + 14n + 4m - 4 \qquad (11) \\
\text{Circuit-IIa:} \quad & \mathcal{T}(LD_{IIa}) = (76m - 21 - 42\omega(m) - 42\lfloor \log_2 m \rfloor) \\
& +(n-m)(76(m+1) - 21 - 42\omega(m+1) - 42\lfloor \log_2(m+1) \rfloor) \\
& \leq 76nm - 76m^2 + 55n + 21m - 21 \qquad (12) \\
\text{Circuit-IIb:} \quad & \mathcal{T}(LD_{IIb}) = (77m - 21 - 42\omega(m) - 42\lfloor \log_2 m \rfloor) \\
& +(n-m)(77(m+1) - 42\omega(m+1) - 42\lfloor \log_2(m+1) \rfloor - 21) \\
& \leq 77nm - 77m^2 + 56n + 21m - 21 \qquad (13) \\
\text{Circuit-III:} \quad & \mathcal{T}(LD_{III}) = 11m + (n-m)11(m+1) = 11nm - 11m^2 + 11n \qquad (14)
\end{aligned}$$

- **Qubit-count:** For LONG-DIV circuit I, registers B and A require $n+2$ and $m+1$ qubits, respectively. Of these, A_m remains in the $|0\rangle$ state, and $B_m = |0\rangle$ at the end of the computation. These two qubits can therefore be regarded as ancillas, and the total qubit count is $n+m+3$.

  For LONG-DIV circuits IIa and IIb, registers A and B require $n+m+3$ qubits in total, as before. In addition, circuits IIa and IIb require $2(m+1) - \omega(m+1) - \lfloor \log_2(m+1) \rfloor - 1$ and $2(m+1) - \omega(m+1) - \lfloor \log_2(m+1) \rfloor - 1 + m$ ancilla qubits, respectively. These ancillas are restored to their initial $|0\rangle$ states. Thus, the total qubit count for circuit IIa is at most

  $$n + 3m + 4 - \omega(m+1) - \lfloor \log_2(m+1) \rfloor \leq n + 3m + 4.$$

  The total qubit count for circuit IIb is at most

  $$n + 4m + 4 - \omega(m+1) - \lfloor \log_2(m+1) \rfloor \leq n + 4m + 4.$$

  For circuit III, in addition to the $n+m+3$ qubits in registers A and B, we require at most m ancilla qubits to uncompute the logical-AND gadgets. These ancillas are restored to their initial states after each iteration and can therefore be reused in the next iteration. Thus, the ancilla count of this circuit is at most $m+2$, and its total qubit count is $n+2m+3$.

- **Toffoli-depth:** The Toffoli depths of the various LONG-DIV circuits are as follows.

$$\begin{aligned}
\text{Circuit-I:} \quad & (3m-1) + (n-m)(3(m+1)-1) = 3nm - 3m^2 + 2n + m - 1. \\
\text{Circuit-IIa:} \quad & (m + 2\lfloor \log_2 m \rfloor + 2\lfloor \log_2 \frac{m}{3} \rfloor + 8) \\
& +(n-m)\left(m + 1 + 2\lfloor \log_2(m+1) \rfloor + 2\lfloor \log_2 \frac{m+1}{3} \rfloor + 8\right) \\
& \leq nm - m^2 + 9n - 8m + 8 + 4(n-m+1)\log_2(m+1) \\
\text{Circuit-IIb:} \quad & 2\lfloor \log_2 m \rfloor + 2\lfloor \log_2 \frac{m}{3} \rfloor + 9 + (n-m)\left(2\lfloor \log_2(m+1) \rfloor + 2\lfloor \log_2 \frac{m+1}{3} \rfloor + 9\right) \\
& \leq (n-m+1)\left(9 + 4\log_2(m+1)\right) \\
\text{Circuit-III:} \quad & m + (n-m)(m+1) = nm - m^2 + n
\end{aligned}$$

- **T-depth:** The T-depths of the various LONG-DIV circuits are as follows.

$$\text{Circuit-I:} \quad \mathcal{T}_d(LD_I) = 9nm - 9m^2 + 6n + 3m - 3 \tag{15}$$

$$\begin{aligned}\text{Circuit-IIa:} \quad \mathcal{T}_d(LD_{IIa}) &= (3m + 6\lfloor \log_2 m \rfloor + 6\lfloor \log_2 \frac{m}{3} \rfloor + 24) \\ &+ (n-m)\left(3(m+1) + 6\lfloor \log_2(m+1) \rfloor + 6\lfloor \log_2 \frac{m+1}{3} \rfloor + 24\right) \\ &\leq 3nm - 3m^2 + 27n - 24m + 24 + 12(n-m+1)\log_2(m+1) \end{aligned} \tag{16}$$

$$\begin{aligned}\text{Circuit-IIb:} \quad \mathcal{T}_d(LD_{IIb}) &= 3(2\lfloor \log_2 m \rfloor + 2\lfloor \log_2 \frac{m}{3} \rfloor + 9) + 3(n-m) \\ &\cdot \left(2\lfloor \log_2(m+1) \rfloor + 2\lfloor \log_2 \frac{m+1}{3} \rfloor + 9\right) \\ &\leq 3(n-m+1)\left(9 + 4\log_2(m+1)\right) \end{aligned} \tag{17}$$

$$\text{Circuit-III:} \quad \mathcal{T}_d(LD_{III}) = 5m + (n-m)5(m+1) = 5nm - 5m^2 + 5n \tag{18}$$

- **Measurements:** LONG-DIV circuits I, IIa, and IIb require no measurements. Because COMP-N-SUB circuit III requires measurements, LONG-DIV circuit III does as well. The number of measurements required is

$$m + (n-m)(m+1) = nm + n - m^2.$$

A summary of the resource requirements of the various LONG-DIV circuits can be obtained from Table 4 by setting $m' = m$.

2.3.2 RES-DIV: A restoring division algorithm

Algorithm 2: RES-DIV

Input: (i) An n-bit dividend $N = (N_{n-1}, \ldots, N_0)$; (ii) an m-bit divisor $D = (D_{m-1}, \ldots, D_0)$, with $n \geq m$; and (iii) an integer m' such that $1 \leq m' \leq m$.
Output: (i) Quotient Q and (ii) remainder R.

$B \leftarrow (0^{m-m'+2} : N)$;
$A \leftarrow (0 : D)$;
if $B_{[n+m-m'-1:n-m']} \geq A_{[m-1:0]}$ **then**
  $B \leftarrow (0 : 0 : (B_{[n+m-m'-1:n-m']} - A_{[m-1:0]}) : B_{[n-m'-1:0]})$;
  $B_{n+m-m'+1} \leftarrow 1$;
end
for $i = 1, \ldots, n-m'$ **do**
  if $B_{[n+m-m'-i:n-m'-i]} \geq A$ **then**
    $B \leftarrow (B_{[n+m-m'+1:n+m-m'+1-i]} : (B_{[n+m-m'-i:n-m'-i]} - A) : B_{[n-m'-1-i:0]})$;
    $B_{n+m-m'+1-i} \leftarrow 1$;
  end
end
return $B_{[n+m-m'+1:m+1]}$ and $B_{[m-1:0]}$ as the quotient and remainder, respectively. ;

In this section, we describe a restoring division algorithm that, unlike long division, does not require the most significant bits (MSBs) of the divisor and dividend to align in the first iteration. Instead, we prepend enough zeros to the dividend that its m' most significant bits overlap the m' least significant bits of the divisor, where $m' \leq m$. Equivalently, the MSB of the dividend aligns with the divisor bit in position m', counted from the least significant end. Beginning with the second iteration, we append the next significant bit of the dividend to the residue. The restoration procedure is the same as in long division (LONG-DIV).

Because the MSBs need not align in the first iteration, this restoring algorithm is useful when divisors of varying lengths occur in superposition. Here, the length is the number of bits from the most significant 1 through the least significant bit, inclusive. Thus, the length of the binary integer 0000111000 is 6, not 10. For RES-DIV, m' is the minimum length of a divisor in the superposition. For a divisor of length $m'' \geq m'$, this procedure ensures that its MSB aligns with the MSB of the dividend in iteration $m'' - m' + 1$. Before this iteration, the minuend remains unchanged as the residue, so no nontrivial computation occurs. Beginning with iteration $m'' - m' + 1$, the computation proceeds exactly as in LONG-DIV. Thus, LONG-DIV is the special case of RES-DIV in which $m' = m$, meaning that all divisors in the superposition have the same length.

In [46], the authors describe a restoring division algorithm in which the dividend is in two's-complement form and is therefore prepended with a sign bit (0 for positive integers). The divisor, also in two's-complement form, is prepended with enough zeros that the dividend and divisor have equal lengths. Consequently, all arithmetic operations in that algorithm involve $(n+1)$-bit integers. Below, we explain several important differences between our RES-DIV algorithm and the restoring division algorithm in [46].

1. **Unequal lengths of the dividend and divisor:** Throughout our algorithm, we prepend at most one zero to the divisor. This ensures that all arithmetic operations involve integers with bit lengths of at most $m + 1$, reducing the implementation cost.

2. **Register allocation:** We change the registers that store the quotient and remainder. At the end of our computation, the quotient is stored in place of the zeros prepended to the dividend, while the remainder is stored in place of the dividend. In [46], these register roles are reversed.

3. **In-place computation of the quotient:** As in LONG-DIV, we reuse dividend qubits that become zero during the residue calculation in one iteration to store quotient bits in the next iteration. We therefore need to prepend at most $m - m' + 2$ zero bits to the dividend. By contrast, the restoring division algorithm in [46] prepends the dividend with n zeros. Our construction therefore requires fewer qubits.

4. **Number of iterations:** In [46], the MSB of the dividend aligns with the LSB of the divisor in the first iteration, so n iterations are required to slide the full length of the divisor across the dividend. Our algorithm instead allows the m' most significant bits of the dividend, where $1 \leq m' \leq m$, to overlap the m' least significant bits of the divisor in the first iteration. This is useful when divisors of length at least m' occur in superposition. We therefore require $n - m' + 1$ iterations. Because $n - m + 1 \leq n - m' + 1 \leq n$, the implementation cost can be lower than that of [46] when $m' > 1$.

The pseudocode for our restoring division algorithm, RES-DIV, is shown in Algorithm 2. Steps 3–6 and 8–11 can be implemented by COMP-N-SUB$^{(m)}$ and COMP-N-SUB$^{(m+1)}$ unitaries, respectively. The qubits $B_{n+m-m'+1}$ and $B_{n+m-m'+1-i}$ in steps 5 and 10, respectively, can store the high bit produced by a COMP-N-SUB unitary. As explained in the previous section, these bits must be flipped because our COMP-N-SUB construction sets the high bit to 1 when the difference is negative and to 0 otherwise. The restoring-division circuit therefore uses an $(m+1)$-qubit register $A = (A_m, \ldots, A_0)$ storing $(0 : D)$ and an $(n+m-m'+2)$-qubit register $B = (B_{n+m-m'+1}, \ldots, B_0)$ whose last n qubits initially store the dividend. At the end of the computation, the first $n-m'+1$ qubits of B store the quotient, while the last m qubits store the remainder.

1. Apply COMP-N-SUB$^{(m)}(B_{[n+m-m'-1:n-m']}, A_{[m-1:0]}, B_{n+m-m'+1})$.
2. For $i = 1, \ldots, n - m'$:
   (a) Apply COMP-N-SUB$^{(m+1)}(B_{[n+m-m'-i:n-m'-i]}, A, B_{n+m-m'+1-i})$.
3. For $i = 0, \ldots, n - m'$, apply $\mathrm{X}_{(B_{n+m-m'+1-i})}$.

Figure 6b illustrates the algorithm with a circuit for dividing a 3-bit dividend by a 2-bit divisor.

Resource estimates: Depending on the COMP-N-SUB circuit used, we refer to these circuits as RES-DIV circuits I, IIa, IIb, and III. The bounds on the various gate counts and depths can be computed in the same way as for LONG-DIV. Appendix B provides detailed calculations of these estimates. Table 4 summarizes the resource requirements of the various quantum circuits for the RES-DIV algorithm.

2.3.3 Non-restoring division

Algorithm 3: NON-RES-DIV

Input: (i) An n-bit dividend $N = (N_{n-1}, \ldots, N_0)$; (ii) an m-bit divisor $D = (D_{m-1}, \ldots, D_0)$, with $n \geq m$; and (iii) m' such that $1 \leq m' \leq m$.
Output: (i) Quotient Q and (ii) remainder R.

$B \leftarrow \left(0^{m-m'+2} : N\right)$;
$A \leftarrow (0 : 0 : D)$;
$B \leftarrow \left((B_{[n+m-m'+1:n-m']} - A) : B_{[n-m'-1:0]}\right)$;
for $i = 0, \ldots, n - m' - 1$ **do**
  $B_{n+m-m'+1-i} \leftarrow \overline{B_{n+m-m'+1-i}}$;
  if $B_{n+m-m'+1-i} == 0$ **then**
    $B \leftarrow \left(B_{[n+m-m'+1:n+m-m'+1-i]} : (B_{[n+m-m'-i:n-m'-i-1]} + A) : B_{[n-m'-i-2:0]}\right)$;
  else
    $B \leftarrow \left(B_{[n+m-m'+1:n+m-m'+1-i]} : (B_{[n+m-m'-i:n-m'-i-1]} - A) : B_{[n-m'-i-2:0]}\right)$;
  end
end
$B_{m+1} \leftarrow \overline{B_{m+1}}$;
if $B_m == 1$ **then**
  $B \leftarrow \left(B_{[n+m-m'+1:m]} : (B_{[m-1:0]} + A_{[m-1:0]})\right)$;
end
return $B_{[n+m-m'+1:m+1]}$ and $B_{[m-1:0]}$ as the quotient and remainder, respectively. ;

Non-restoring division algorithms are iterative algorithms in which the divisor is subtracted from or added to a minuend in each iteration. Unlike restoring division algorithms, they have no restoration phase: if the difference is negative, we do not restore the residue to the minuend. Instead, we adjust the negative difference in later iterations until we obtain a positive difference. A positive difference implies that the corresponding quotient bit is 1; otherwise, it is 0.

Algorithm 3 gives pseudocode for our non-restoring division algorithm, NON-RES-DIV. As in RES-DIV, the parameter m' determines how many bits of the dividend overlap the divisor in the first iteration and hence determines the number of iterations. When divisors occur in superposition, m' is again their minimum length. Register B contains the dividend prepended with $m - m' + 2$ zeros, while register A contains the divisor prepended with two zeros. We begin by subtracting A from the $m + 2$ most significant bits of B. In subsequent iterations, we slide this window by appending the next significant bit of B to the residue. If the MSB of the residue is 1, the difference is negative and is adjusted by adding the divisor in the next iteration; if the MSB is 0, the reverse

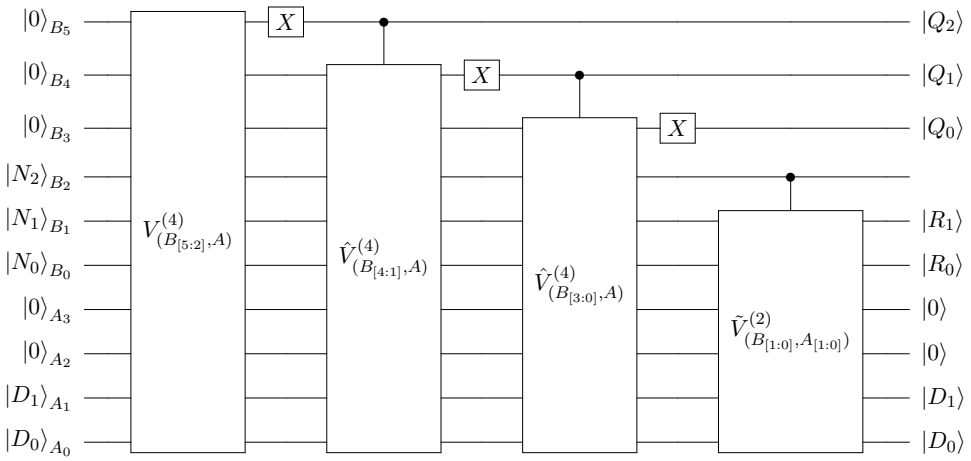


Figure 7: Quantum circuit for NON-RES-DIV. The dividend $N = (N_2, N_1, N_0)$ and the divisor $D = (D_1, D_0)$ are 3-bit and 2-bit integers, respectively. The remainder and quotient are $R = (R_1, R_0)$ and $Q = (Q_2, Q_1, Q_0)$, respectively, and $m' = 1$. The unitaries $V^{(k)}$, $\hat{V}^{(k)}$, and $\tilde{V}^{(k)}$ implement subtraction, Ctrl-AddSub, and c-ADD, respectively, on k-bit integers. Their subscripts identify the qubits on which they act.

operation is performed (steps 6–10). The MSB is also flipped and used to store the corresponding quotient bit (step 5). After all iterations, if bit B_{m+1} is 1, the residue is still negative. Adding the divisor in steps 13–15 corrects it and yields the remainder.

The non-restoring division procedure is similar to the algorithm described in [46], but we introduce the following important changes to enable a more efficient implementation.

1. **Unequal lengths of the dividend and divisor:** As in RES-DIV, we ensure that operations (subtractions or controlled additions) involve integers with bit lengths of at most $m + 2$ by appending at most two zeros to the divisor. By contrast, [46] appends enough zeros to the divisor and dividend to produce operations on $(n + 1)$-bit integers, including the sign bit, which increases the gate complexity per operation.

2. **Number of iterations:** As in RES-DIV, we allow at most m' bits of the dividend to overlap the divisor in the first iteration, resulting in $n - m' + 1$ iterations. In [46], the MSB of the dividend aligns with the LSB of the divisor in the first iteration, so n iterations are required to slide the full length of the divisor across the dividend.

NON-RES-DIV can be implemented with the following two unitaries, which are also discussed in [46].

1. **Ctrl-AddSub:** The inputs to this unitary are two integers a and b and a control bit c. It subtracts the two integers if the control is $|1\rangle$ and adds them if the control is $|0\rangle$. As explained above, $a - b = \overline{\overline{a} + b}$. Thus, this unitary can be implemented with an adder conjugated by CNOT gates whose control is c and whose targets are the qubits of a. Further details about this unitary and its implementation appear in [46].

2. **Ctrl-AddNOP or c-ADD:** Ctrl-AddNOP, as described in [46], is a controlled adder (c-ADD). Any of the COMP-N-SUB circuits can be modified to implement a controlled adder.

		NON-RES-DIV circuit		
		A	**B**	**C**
Toffoli-count		$2m(n-m'+1)$	$10m(n-m'+1)$	-
Toffoli-depth		$2m(n-m'+1)$	$4(n-m'+1)\log_2(m+2)$	-
T-count		$14m(n-m'+1)$	$70m(n-m'+1)$	$4m(n-m'+1)$
T-depth		$6m(n-m'+1)$	$12(n-m'+1)\log_2(m+2)$	$2m(n-m'+1)$
Qubit-count	Total	$n+2m-m'$	$n+5m-m'$	$n+3m-m'$
	Ancilla	2	$2m$	m
	Garbage	0	0	0
CNOT-count	Clifford+Toffoli	$5m(n-m'+1)$	$4m(n-m'+1)$	-
	Clifford+T	$19m(n-m'+1)$	$74m(n-m'+1)$	$16m(n-m'+1)$
Fully reversible		Yes	Yes	No

Table 5: Resource requirements of quantum circuits for the NON-RES-DIV algorithm. Only leading-order terms are shown. Here, n and m are the numbers of bits in the binary representations of the dividend and divisor, respectively. The parameter m' is an integer such that $1 \leq m' \leq m$.

Suppose the addition is conditioned on the state of a qubit c. Because our construction produces no high bit, we need not allocate a qubit for it. From the COMP-N-SUB circuit, we remove all gates that change the high bit. We retain gates controlled on the high bit but instead control them on qubit c. We also remove all X gates at the beginning and end of the circuit.

A quantum circuit implementing NON-RES-DIV (Algorithm 3) can be constructed as follows.

1. Apply $\text{SUB}^{(m+2)}\left(B_{[n+m-m'+1:n-m']}, A\right)$.
2. For $i = 0, \ldots, n-m'-1$:
   (a) Apply $\text{X}_{(B_{n+m-m'+1-i})}$.
   (b) Apply ctrl-$\text{AddSub}^{(m+2)}\left(B_{[n+m-m'-i:n-m'-i-1]}, A\right)$, controlled on $B_{n+m-m'+1-i}$.
3. Apply $\text{X}_{(B_{m+1})}$.
4. Apply $\text{c-ADD}^{(m)}\left(B_{[m-1:0]}, A_{[m-1:0]}\right)$ controlled on B_m.

SUB is a subtractor. As noted above, Ctrl-AddSub can be implemented with an adder and CNOT gates. We consider the following types of adders and give their resource estimates for operations on k-bit integers.

1. The quantum ripple-carry (QRC) adder of [39] has a Toffoli count and Toffoli depth of at most $2k-1$. The CNOT counts for Clifford+Toffoli and Clifford+T implementations are at most $5k-5$ and $19k-12$, respectively. In addition to the $2k$ input qubits, it requires one qubit to store the high bit.
2. The quantum carry-lookahead (QCLA) adder of [41] has a Toffoli count of at most $10k - 3\omega(k) - 3\omega(k-1) - 3\lfloor\log_2 k\rfloor - 3\lfloor\log_2(k-1)\rfloor - 7$ and a Toffoli depth of at most $\lfloor\log_2 k\rfloor + \lfloor\log_2(k-1)\rfloor + \lfloor\log_2 \frac{k}{3}\rfloor + \lfloor\log_2 \frac{k-1}{3}\rfloor + 8$. Its CNOT count is $4k-5$ for a Clifford+Toffoli implementation and at most $(4k-5)+7(10k-7) = 74k-54$ for a Clifford+T implementation. It requires $2k$ input qubits, one additional qubit to store the high bit, and $2k - \omega(k) - \lfloor\log_2 k\rfloor - 1$ ancilla qubits that are restored to their initial $|0\rangle$ states at the end of the computation.

3. The QRC adder of [40], which uses logical-AND gadgets, has T-count $4k-4$, T-depth $2k-2$, and CNOT count $16k-15$. In addition, it uses $k-1$ ancilla qubits that are restored to their initial $|0\rangle$ states at the end of the computation.

We do not use the high bits of any of the controlled adders or subtractors and therefore do not need to allocate those extra qubits.

Resource estimates: The NON-RES-DIV circuits obtained using the adders in (1), (2), and (3) above are called circuits 1, 2, and 3, respectively. The contributions from Ctrl-AddNOP (c-ADD) or COMP-N-SUB are lower-order terms. So, the variant of the NON-RES-DIV circuits depend mostly on the type of adder used, and hence the nomenclature. We use COMP-N-SUB circuits I, IIb, and III with NON-RES-DIV circuits 1, 2, and 3, respectively. The adder in [40] has only a Clifford+T implementation, so we do not report the Toffoli count or depth of circuit 3. Each circuit requires $n-m'+1$ adders acting on $(m+2)$-bit integers, $2(n-m')(m+2)$ CNOT gates for Ctrl-AddSub, and one COMP-N-SUB$^{(m)}$.

- **Toffoli-count:** The Toffoli counts for NON-RES-DIV circuits 1 and 2 are bounded above by

$$(n-m'+1)(2(m+2)-1)+(3m-1),$$

and

$$(n-m'+1)(10(m+2)-7)+11m-4,$$

respectively.

- **CNOT-count:** The CNOT counts for Clifford+Toffoli implementations of the various NON-RES-DIV circuits are as follows.

$$\text{Circuit 1:} \quad \mathcal{N}_{Tof}^{cnot}(NRes_1) = (n-m'+1)(5(m+2)-5)+4m-5 \tag{19}$$
$$\text{Circuit 2:} \quad \mathcal{N}_{Tof}^{cnot}(NRes_2) = (n-m'+1)(4(m+2)-5)+8m-4 \tag{20}$$

For Clifford+T implementations of the various NON-RES-DIV circuits, the CNOT counts are as follows.

$$\text{Circuit 1:} \quad \mathcal{N}_{T}^{cnot}(NRes_1) = (n-m'+1)(19(m+2)-12)+25m-12 \tag{21}$$
$$\text{Circuit 2:} \quad \mathcal{N}_{T}^{cnot}(NRes_2) = (n-m'+1)(74(m+2)-54)+85m-32 \tag{22}$$
$$\text{Circuit 3:} \quad \mathcal{N}_{T}^{cnot}(NRes_3) = (n-m'+1)(16(m+2)-15)+19m-7 \tag{23}$$

- **T-count:** The T-counts for the various NON-RES-DIV circuits are as follows.

$$\text{Circuit 1:} \quad \mathcal{T}(NRes_1) \leq 7(n-m'+1)(2(m+2)-1)+18m-4 \tag{24}$$
$$\text{Circuit 2:} \quad \mathcal{T}(NRes_2) \leq 7(n-m'+1)(10(m+2)-7)+77m-21 \tag{25}$$
$$\text{Circuit 3:} \quad \mathcal{T}(NRes_3) = (n-m'+1)(4(m+2)-4)+11m \tag{26}$$

- **Qubit-count:** Registers A and B consist of $m+2$ and $n+m-m'+2$ qubits, respectively. At the end of the computation, the quotient and remainder are stored in register B. The two qubits prepended to the divisor in A are restored to the $|0\rangle$ state and are therefore ancilla qubits. All three types of circuits require these qubits.

  NON-RES-DIV circuit-1 uses no other qubits, so its total qubit count is $n+2m-m'+4$, of which two are ancillas.

  The QCLA adders in circuit-2 require $2(m+2)-\omega(m+2)-\lfloor\log_2(m+2)\rfloor-1$ ancilla qubits. Because these qubits are restored to their initial states, they can be shared among the adders and COMP-N-SUB unitaries. COMP-N-SUB circuit IIb uses m additional ancilla qubits. Hence, the total number of qubits required for NON-RES-DIV circuit-2 is at most $n+5m-m'+7-\omega(m+2)-\lfloor\log_2(m+2)\rfloor \leq n+5m-m'+7$.

  Each adder in circuit-3 requires $m+1$ ancilla qubits that are restored to their initial states and can therefore be reused by later adders and by COMP-N-SUB circuit III. Thus, the total number of qubits required for NON-RES-DIV circuit-3 is $n+3m-m'+5$.

- **Toffoli-depth:** The Toffoli depth of circuit 1 is at most

$$(n-m'+1)(2(m+2)-1)+(3m-1), \tag{27}$$

  and that of circuit 2 is at most

$$\leq \quad (n-m'+1)\left(4\lfloor\log_2(m+2)\rfloor+8\right)+4\lfloor\log_2 m\rfloor+9. \tag{28}$$

- **T-depth:** The T-depths of the various NON-RES-DIV circuits are as follows.

Circuit 1:	$\mathcal{T}_d(NRes_1) \leq 3(n-m'+1)(2(m+2)-1)+9m-3$	(29)
Circuit 2:	$\mathcal{T}_d(NRes_2) \leq 3(n-m'+1)\left(4\lfloor\log_2(m+2)\rfloor+8\right)+3\left(4\lfloor\log_2 m\rfloor+9\right)$	(30)
Circuit 3:	$\mathcal{T}_d(NRes_3) = (n-m'+1)(2(m+2)-2)+5m$	(31)

- **Measurements:** NON-RES-DIV circuits 1 and 2 require no measurements. For circuit-3, the number of measurements required is at most

$$(n-m'+1)(m+1)+m = nm+n+2m-m'-mm'+1.$$

Table 5 summarizes the resource requirements of the various quantum circuits for the NON-RES-DIV algorithm.

2.4 Comparison

In this section, we compare our results with previous work using the cost metrics for fault-tolerant implementations described earlier. Nearly all papers focused on fault-tolerant implementations use Clifford+T gates and derive bounds on T-count, T-depth, and qubit count. We consider papers that provide the best known bounds, comparing our circuits with the long-division circuits in [48] and the restoring and non-restoring division circuits in [46]. We first summarize the relative applicability of the division algorithms to determine suitable parameters for comparison.

Long division					
$\mathcal{T}$	**OPF24**	**LD$_{\mathbf{I}}$**	**LD$_{\mathbf{IIa}}$**	**LD$_{\mathbf{IIb}}$**	**LD$_{\mathbf{III}}$**
OPF24	1	0.39	1.64	1.67	0.24
LD$_{\mathbf{I}}$	2.56	1	4.17	4.35	0.61
LD$_{\mathbf{IIa}}$	0.61	0.24	1	1.01	0.14
LD$_{\mathbf{IIb}}$	0.60	0.23	0.99	1	0.14
LD$_{\mathbf{III}}$	4.18	1.64	6.91	7	1

(a)

Long division					
$\mathcal{N}_T^{cnot}$	**OPF24**	**LD$_{\mathbf{I}}$**	**LD$_{\mathbf{IIa}}$**	**LD$_{\mathbf{IIb}}$**	**LD$_{\mathbf{III}}$**
OPF24	1	0.35	1.39	1.41	0.32
LD$_{\mathbf{I}}$	2.86	1	4	4	0.90
LD$_{\mathbf{IIa}}$	0.72	0.25	1	1.02	0.23
LD$_{\mathbf{IIb}}$	0.71	0.25	0.98	1	0.22
LD$_{\mathbf{III}}$	3.16	1.11	4.37	4.47	1

(b)

Restoring division					
$\mathcal{T}$	**TMCVH19**	**Res$_{\mathbf{I}}$**	**Res$_{\mathbf{IIa}}$**	**Res$_{\mathbf{IIb}}$**	**Res$_{\mathbf{III}}$**
TMCVH19	1	$\frac{0.52m}{n}$	$\frac{2.17m}{n}$	$\frac{2.22m}{n}$	$\frac{0.31m}{n}$
Res$_{\mathbf{I}}$	$\frac{1.94n}{m}$	1	4.17	4.35	0.61
Res$_{\mathbf{IIa}}$	$\frac{0.46n}{m}$	0.24	1	1.01	0.14
Res$_{\mathbf{IIb}}$	$\frac{0.45n}{m}$	0.23	0.99	1	0.14
Res$_{\mathbf{III}}$	$\frac{3.18n}{m}$	1.64	6.91	7	1

(c)

Restoring division					
$\mathcal{N}_T^{cnot}$	**TMCVH19**	**Res$_{\mathbf{I}}$**	**Res$_{\mathbf{IIa}}$**	**Res$_{\mathbf{IIb}}$**	**Res$_{\mathbf{III}}$**
TMCVH19	1	$\frac{0.57m}{n}$	$\frac{1.89m}{n}$	$\frac{1.92m}{n}$	$\frac{0.43m}{n}$
Res$_{\mathbf{I}}$	$\frac{1.76n}{m}$	1	3.33	3.45	0.76
Res$_{\mathbf{IIa}}$	$\frac{0.53n}{m}$	0.30	1	1.02	0.23
Res$_{\mathbf{IIb}}$	$\frac{0.52n}{m}$	0.29	0.98	1	0.22
Res$_{\mathbf{III}}$	$\frac{2.32n}{m}$	1.32	4.37	4.47	1

(d)

Non-restoring division				
$\mathcal{T}$	**TMCVH19**	**NRes$_{\mathbf{1}}$**	**NRes$_{\mathbf{2}}$**	**NRes$_{\mathbf{3}}$**
TMCVH19	1	$\frac{m}{n}$	$\frac{5m}{n}$	$\frac{0.29m}{n}$
NRes$_{\mathbf{1}}$	$\frac{n}{m}$	1	5	0.29
NRes$_{\mathbf{2}}$	$\frac{0.2n}{m}$	0.2	1	0.06
NRes$_{\mathbf{3}}$	$\frac{3.5n}{m}$	3.5	17.5	1

(e)

Non-restoring division				
$\mathcal{N}_T^{cnot}$	**TMCVH19**	**NRes$_{\mathbf{1}}$**	**NRes$_{\mathbf{2}}$**	**NRes$_{\mathbf{3}}$**
TMCVH19	1	$\frac{m}{n}$	$\frac{3.85m}{n}$	$\frac{0.84m}{n}$
NRes$_{\mathbf{1}}$	$\frac{n}{m}$	1	3.85	0.84
NRes$_{\mathbf{2}}$	$\frac{0.26n}{m}$	0.26	1	0.22
NRes$_{\mathbf{3}}$	$\frac{1.19n}{m}$	1.19	4.63	1

(f)

Figure 8: Ratios of the T-counts (a, c, e) and CNOT-counts (b, d, f) of quantum circuits for different division algorithms. Each row except the first and each column is labeled by a circuit. OPF24 and TMCVH19 denote the circuits in [48] and [46], respectively. LD_x, Res_x, and $NRes_x$ denote LONG-DIV circuit x, RES-DIV circuit x, and NON-RES-DIV circuit x, respectively. The metric, either T-count ($\mathcal{T}$) or CNOT-count ($\mathcal{N}_T^{cnot}$), is specified in the leftmost cell of the second row. The cell at the intersection of column i and row j stores $\mathcal{T}(i)/\mathcal{T}(j)$ for a T-count table or $\mathcal{N}_T^{cnot}(i)/\mathcal{N}_T^{cnot}(j)$ for a CNOT-count table.

Applicability: The long-division algorithm ([47, 48], LONG-DIV), which is a specific type of restoring algorithm, can be applied when there is a single divisor or when all divisors in a superposition have the same length. It requires the MSBs of the dividend and divisor to align in the first iteration. This occurs, for example, in algorithms that perform repeated division by a fixed integer p in a finite field $\mathbb{F}_p$. When divisors of varying lengths occur in superposition, non-restoring algorithms and restoring algorithms other than long division can be used. The algorithms in [46] ensure that only the MSB of the dividend overlaps the LSB of the divisor in the first iteration. Our RES-DIV and NON-RES-DIV algorithms introduce the parameter m', which provides greater flexibility in the amount of overlap.

For a meaningful comparison with any long-division algorithm, we therefore set $m' = m$. Below, we give the T-count, T-depth, and CNOT-count of the circuits in [46] and [48]. The algorithms in [46] operate on integers in two's-complement form. For positive integers, their binary representations are therefore prepended with a 0. Thus, for n-bit integers, the effective input length is $n+1$ bits. The unitaries consequently operate on $(n+1)$-bit integers, and the expressions have been scaled accordingly. The T-counts of those circuits are as follows.

$$\begin{aligned}
\text{Long division [48]:} &\qquad \mathcal{T}(LD_{OPF24}) = 46mn - 46m^2 + 48m - 2n - 2 \\
\text{Restoring division [46]:} &\qquad \mathcal{T}(Res_{TMCVH19}) = 35n^2 + 42n + 7 \\
\text{Non-restoring division [46]:} &\qquad \mathcal{T}(NRes_{TMCVH19}) = 14n^2 + 49n + 7
\end{aligned}$$

The T-depths of those circuits are as follows.

$$\begin{aligned}
\text{Long division [48]:} &\qquad \mathcal{T}_d(LD_{OPF24}) = 20mn - 20m^2 + 20m \\
\text{Restoring division [46]:} &\qquad \mathcal{T}_d(Res_{TMCVH19}) = 15n^2 + 18n + 3 \\
\text{Non-restoring division [46]:} &\qquad \mathcal{T}_d(NRes_{TMCVH19}) = 6n^2 + 21n + 11
\end{aligned}$$

The CNOT-counts are as follows.

$$\begin{aligned}
\text{Long division [48]} &: \mathcal{N}_T^{cnot}(LD_{OPF24}) = 60nm - 60m^2 - 13n + 73m - 13 \\
\text{Restoring division [46]} &: \mathcal{N}_{Tof}^{cnot}(Res_{TMCVH19}) = 9n^2 + 10n + 1 \\
&: \mathcal{N}_T^{cnot}(Res_{TMCVH19}) = 44n^2 + 87n + 43 \\
\text{Non-restoring division [46]} &: \mathcal{N}_{Tof}^{cnot}(NRes_{TMCVH19}) = 5n^2 + 11n \\
&: \mathcal{N}_T^{cnot}(NRes_{TMCVH19}) = 19n^2 + 53n + 42
\end{aligned}$$

Tables 1 and 2 compare the resource requirements of circuits for the different division algorithms. We first discuss the full reversibility of each circuit, because many quantum applications require this feature.

Full reversibility: LONG-DIV circuit III, RES-DIV circuit III, and NON-RES-DIV circuit 3 are not fully reversible (Table 1). The quantum circuit in [48] is not fully reversible, whereas those in [46] are.

Next, we identify the circuits with the best metrics for each algorithm using the tables in Figure 8. Each table corresponds to a particular division algorithm and metric, either T-count ($\mathcal{T}$) or CNOT count ($\mathcal{N}_T^{cnot}$). In every table, each row except the first and each column is labeled by a circuit. OPF24 and TMCVH19 denote the circuits in [48] and [46], respectively. Each cell gives the ratio of the metric for the circuit labeling its column to that for the circuit labeling its row.

For example, in a T-count table, the cell at column i and row j contains $\mathcal{T}(i)/\mathcal{T}(j)$. These ratios are computed from the bounds derived in Section 2.3 and summarized in Tables 1, 2, 4, and 5. We assume that $n - m + 1 > 0$ and $n - m' + 1 > 0$ and take the limit $n \to \infty$.

Long-division algorithm: Figure 8a shows that LONG-DIV circuit III has the lowest T-count, approximately 76.08% lower than that of [48], the previous best long-division circuit. Among the reversible circuits, LONG-DIV circuit I has the lowest T-count, a 60.94% reduction relative to [48]. Both circuits also significantly reduce the CNOT count relative to [48] (Figure 8b), by approximately 68.35% and 65.03% for circuits III and I, respectively. Circuit IIb has the smallest T-depth (Table 1), improving on [48] by a factor of $O(m/\log m)$. Circuit I has the smallest qubit count (Table 2) because it uses no extra ancillas. We are not aware of any work that establishes the optimal qubit count of a divider circuit. Because $n + m$ qubits are required to store the dividend and divisor, the qubit count of LONG-DIV circuit I may be optimal.

Restoring division algorithm: From the tables in Figures 8c and 8d and Table 1, we observe that, for $m \in O(n^{1-\delta})$, where $\delta \to 0$, all our RES-DIV circuits achieve asymptotic reductions in T-count, T-depth, and CNOT count relative to the previous best restoring-division circuit from [46]. Circuit I has the smallest qubit count (Table 2), circuit III has the smallest T-count, and circuit IIb has a significantly smaller T-depth (Table 1) than the other circuits.

Non-restoring division algorithm: From the tables in Figures 8e and 8f and Table 2, we observe that, for $m \in O(n^{1-\delta})$, where $\delta \to 0$, all three NON-RES-DIV circuits achieve asymptotic reductions in the stated cost metrics relative to the previous best circuit in [46]. Circuit-1 has the smallest qubit count, circuit-2 has the smallest T-depth, and circuit-3 has the smallest T-count and CNOT count.

Hence, for all three types of algorithms, the circuits derived in this paper achieve significant reductions in the various cost metrics relative to the previous best circuits in the literature. We now select the best circuit for each cost metric. For this purpose, we use the bounds derived in Section 2.3 and summarized to leading order in Tables 1 and 2.

T-count and CNOT-count: NON-RES-DIV circuit-3 has the smallest T-count and CNOT count among all the circuits, while NON-RES-DIV circuit-1 has the smallest T-count and CNOT count among the fully reversible circuits.

T-depth: The T-depths of LONG-DIV circuit IIb, RES-DIV circuit IIb, and NON-RES-DIV circuit-2 are comparable and are the smallest.

Qubit-count: LONG-DIV circuit I, RES-DIV circuit I, and NON-RES-DIV circuit-1 have comparable qubit counts. More specifically, LONG-DIV circuit I uses one fewer qubit than the others and, as noted above, may achieve the optimal qubit count.

Fault-tolerant implementation

Following the procedures in [69], we estimate the resources required to implement the various division circuits fault-tolerantly with the surface code. We consider dividing a 512-bit dividend by a 256-bit divisor, so $n = 512$ and $m = 256$. We assume a magic-state injection error rate $p_{in} = 10^{-4}$, a per-gate error rate $p_g = 10^{-5}$, a surface-code cycle time $t_{sc} = 200\,\text{ns}$, and a measurement time $t_M = 100\,\text{ns}$.

Algorithm	Circuit		# Physical qubits	Time	d	d_c
Long division	Orts et al. [48]		1.78×10^5	15.79 s	6	6
	LD_I (**This paper**)		6.36×10^4	7 s	6	5
	LD_{IIa} (**This paper**)		1.84×10^5	2.75 s	6	6
	LD_{IIb} (**This paper**)		4.63×10^5	0.379 s	6	6
	LD_{III} (**This paper**)		8.56×10^4	3.97 s	6	5
Restoring division	Thapliyal et al. [46]		1.78×10^5	47.3 s	6	6
	Res_I (**This paper**)	$m' = 1$	1.19×10^5	14.19 s	6	6
	Res_{IIa} (**This paper**)	$m' = 1$	2.13×10^5	5.47 s	6	6
	Res_{IIb} (**This paper**)	$m' = 1$	4.914×10^5	0.756 s	6	6
	Res_{III} (**This paper**)	$m' = 1$	1.495×10^5	7.89 s	6	6
Non-restoring division	Thapliyal et al. [46]		1.782×10^5	19 s	6	6
	$NRes_1$ (**This paper**)	$m' = m$	6.57×10^4	4.79 s	6	5
		$m' = 1$	9.23×10^4	9.548 s	6	6
	$NRes_2$ (**This paper**)	$m' = m$	4.45×10^5	0.371 s	6	6
		$m' = 1$	4.45×10^5	0.739 s	6	6
	$NRes_3$ (**This paper**)	$m' = m$	8.29×10^4	1.334 s	5	5
		$m' = 1$	1.19×10^5	3.173 s	6	6

Table 6: Fault-tolerant resource estimates for various division circuits encoded with the surface code. The quantities d and d_c are the code distances used for non-Clifford and Clifford gates, respectively.

LONG-DIV-I: First, we consider LONG-DIV circuit I. From Equations 11 and 15, the T-count and T-depth are bounded as follows.

$$\mathcal{T}(LD_I) \leq 1.188 \times 10^6; \qquad \mathcal{T}_d(LD_I) \leq 5.937 \times 10^5.$$

Each T gate requires one magic state. The output error rate for state distillation is at most

$$p_{out} = \frac{1}{\mathcal{T}(LD_I)} = 8.419 \times 10^{-7}.$$

To calculate the surface-code distance, we use Algorithm 4 in [85] with parameter $\epsilon = 1$. For the stated values of p_{in} and p_g, the algorithm suggests one layer of distillation at distance $d = 6$. This stage requires 16 logical qubits to generate one magic state. One logical qubit requires $N_p = 2.5 \times 1.25 \times d^2$ physical qubits. Thus, the total is approximately $16 \times N_p = 1800$ physical qubits, and one round of distillation takes $\sigma = 10d = 60$ surface-code cycles. One magic state is therefore produced every $\sigma t_{sc} = 1.2 \times 10^{-5}$ s. We require $\mathcal{T}(LD_I)/\mathcal{T}_d(LD_I) \approx 2$ magic states in parallel per T layer. Consequently, the number of physical qubits required to implement the non-Clifford gates is

$$2 \times 1800 = 3600, \tag{32}$$

and the execution time is

$$\mathcal{T}_d(LD_I) \times 1.2 \times 10^{-5}\,\text{s} \approx 7\,\text{s}. \tag{33}$$

Next, we consider the Clifford-gate implementation. Because the Clifford gates in our circuits are mostly CNOT gates, we estimate the cost of implementing them. From Equation 7, the CNOT count and qubit count are bounded as follows.

$$\mathcal{N}_T^{cnot}(LD_I) \leq 1.648 \times 10^6; \qquad \mathcal{Q}(LD_I) = n + m + 3 = 771.$$

Thus, the overall error rate of the Clifford gates should be less than $\frac{1}{\mathcal{N}_T^{cnot}(LD_I)} = \frac{1}{1.648\times10^6} \approx 6.07 \times 10^{-7}$. The distance is the smallest d_c such that

$$\left(\frac{p_{in}}{0.0125}\right)^{\frac{d_c+1}{2}} = 0.008^{\frac{d_c+1}{2}} < 6.07 \times 10^{-7}, \tag{34}$$

and hence $d_c = 5$. Encoding the Clifford operations requires at most $(n+m+3) \times 2.5 \times 1.25 \times 5^2 =$ 60,000 physical qubits. Thus, implementing LONG-DIV-I requires approximately 60,000 + 3,600 = 63,600 physical qubits in total. We expect to perform approximately $\mathcal{N}_T^{cnot}(LD_I)/((n + m + 3)\mathcal{T}_d(LD_I)) \approx 0.004$ Clifford operations per qubit per T layer. Each logical CNOT takes two surface-code cycles, whereas each T layer requires $\sigma = 60$ surface-code cycles, so the running time is dominated by the non-Clifford T gates.

In summary, implementing LONG-DIV circuit I with $n = 512$ and $m = 256$ requires approximately 6.36×10^4 physical qubits and 7 s.

LONG-DIV-IIb: A similar analysis for LONG-DIV circuit IIb gives the following bounds on its T-count (Equation 13), T-depth (Equation 17), output error rate for distillation, CNOT count (Equation 9), and qubit count.

$$\mathcal{T}(LD_{IIb}) \leq 5.08 \times 10^6; \qquad \mathcal{T}_d(LD_{IIb}) \leq 31611; \qquad p_{out} = \frac{1}{\mathcal{T}(LD_{IIb})} \approx 1.968 \times 10^{-7}$$
$$\mathcal{N}_T^{cnot}(LD_{IIb}) \leq 5.61 \times 10^6; \qquad \mathcal{Q}(LD_{IIb}) \leq 1541$$

Using Algorithm 4 of [85], we apply one layer of distillation at distance $d = 6$. This stage requires $16 \times 2.5 \times 1.25 \times 6^2 = 1800$ physical qubits, and one round takes $\sigma = 10d = 60$ surface-code cycles. Thus, one magic state is produced every $\sigma t_{sc} = 60 \times 200 \times 10^{-9}$ s $= 1.2 \times 10^{-5}$ s. We require $\mathcal{T}(LD_{IIb})/\mathcal{T}_d(LD_{IIb}) \approx 161$ magic states in parallel per T layer. Implementing the non-Clifford gates therefore requires $161 \times 1800 = 289{,}800$ physical qubits and takes $\mathcal{T}_d(LD_{IIb}) \times 1.2 \times 10^{-5}$ s $=$ 0.379 s.

Because the Clifford gates in this circuit are mostly CNOT gates, their overall error rate should be less than $1/\mathcal{N}_T^{cnot}(LD_{IIb}) = 1.784 \times 10^{-7}$. The distance d_c is therefore the smallest integer satisfying the following inequality.

$$0.008^{\frac{d_c+1}{2}} < 1.784 \times 10^{-7}.$$

We obtain $d_c = 6$. Encoding the Clifford operations requires at most $\mathcal{Q}(LD_{IIb}) \times 2.5 \times 1.25 \times d_c^2 \approx$ 173,362 physical qubits, bringing the total to 289,800 + 173,362 $\approx 4.63 \times 10^5$. We expect to perform approximately $\mathcal{N}_T^{cnot}(LD_{IIb})/(\mathcal{Q}(LD_{IIb})\mathcal{T}_d(LD_{IIb})) \approx 0.115$ Clifford operations per qubit per T layer. Each logical CNOT takes two surface-code cycles, whereas each T layer requires $\sigma = 60$ surface-code cycles, so the running time is dominated by the non-Clifford T gates.

In summary, implementing LONG-DIV circuit IIb with $n = 512$ and $m = 256$ requires approximately 4.63×10^5 physical qubits and 0.379 s.

We apply similar procedures to estimate the physical-resource requirements of the other division circuits, as summarized in Table 6. NON-RES-DIV circuit-2, which has the minimum T-depth, requires the least time. LONG-DIV circuit IIb and RES-DIV circuit IIb are also significantly faster than the other circuits. These circuits achieve an asymptotic reduction in T-depth relative to the remaining circuits (Table 1). LONG-DIV circuit I requires the fewest physical qubits, closely followed by NON-RES-DIV circuit-1. Their qubit requirements are nearly optimal (Table 2); in fact, both use only $O(1)$ ancillas.

3 Discussion

In this paper, we have designed novel quantum circuits for several division algorithms. We have analyzed the resource requirements of each circuit and compared them with those of the best circuits in the literature. We have focused on cost metrics such as T-count, T-depth, CNOT-count, qubit count, and full reversibility because these are important from a fault-tolerant perspective. Each of our circuits outperforms previous implementations, achieving asymptotic reductions in these metrics. Two main factors contribute to these improvements. First, we modify the algorithms to reuse qubits, ensure that intermediate operations act on fewer qubits, and reduce the number of iterations, thereby lowering the qubit requirements and gate complexity. Second, we introduce the COMP-N-SUB primitive, which can be implemented at roughly the same cost as an ordinary addition of two integers. Expressing the division algorithms in terms of this primitive further reduces gate complexity. We have estimated the number of physical qubits required to implement the various circuits using the surface code when the dividend and divisor are 512 and 256 bits long, respectively. We have also estimated the time required to perform division using the encoded circuits. Our circuits outperform previous ones by a significant margin with respect to these metrics as well.

To the best of our knowledge, no QFT-based quantum divider circuit has been constructed. Although we expect such circuits to use many rotation gates and therefore to be inefficient for fault-tolerant implementation, it would be interesting to develop one and compare its resource requirements with those of existing circuits. In particular, it would be interesting to develop a QFT-based version of COMP-N-SUB, which plays a crucial role in the implementation of our divider circuits. Comparing the noise sensitivity of the various circuits is another interesting direction.

We also envision using COMP-N-SUB in other algorithms, for instance to compute the greatest common divisor, which is used in quantum algorithms for computing discrete logarithms on elliptic curves [86] and for Jacobi factoring [87]. A detailed resource analysis of this application appears in Appendix E. We conclude by illustrating how COMP-N-SUB can be used to implement Stein's GCD algorithm. We do not optimize the complete algorithm or discuss its efficient implementation in detail. Instead, our aim is to highlight the usefulness of COMP-N-SUB, so we focus on the part of the algorithm in which this primitive can be used and discuss its advantages.

More broadly, COMP-N-SUB illustrates a design philosophy in which a predicate and the data transformation conditioned on that predicate are treated as a single coherent operation, rather than as separate comparison and controlled-arithmetic modules. Treating such decision-and-update operations as first-class primitives can help recast classical algorithms with data-dependent branches into forms that are more natural for reversible computation. This perspective can guide the design of new quantum algorithms by encouraging the algorithm and its arithmetic primitives to be developed together, with recurring decision-and-update patterns determining the appropriate computational building blocks.

3.1 Application of COMP-N-SUB in the binary GCD algorithm

In this section, we discuss how COMP-N-SUB unitaries can be used to implement the binary GCD algorithm whose pseudocode is given in Algorithm 4. This popular GCD algorithm is often referred to as Stein's algorithm. We follow the pseudocode given in [88]. The inputs are two n-bit integers, u and v, and the output is their GCD, another n-bit integer denoted by g.

We emphasize at the outset that several variants of the classical GCD algorithm appear in

Algorithm 4: Binary GCD algorithm [88].

Input: Positive integers a and b.
Output: Positive integer $g = \gcd(a, b)$.

1 $u \leftarrow a$; $v \leftarrow b$; $g \leftarrow 1$;
2 **while** u *and* v *are even* **do**
3 $u \leftarrow u/2$; $v \leftarrow v/2$; $g \leftarrow 2g$;
4 **end**
5 **while** $v \neq 0$ **do**
6 **while** u *is even* **do**
7 $u \leftarrow u/2$;
8 **end**
9 **while** v *is even* **do**
10 $v \leftarrow v/2$;
11 **end**
12 **if** $u > v$ **then**
13 $u \leftarrow (u - v)/2$;
14 **else**
15 $v \leftarrow (v - u)/2$;
16 **end**
17 **end**
18 **return** $g \cdot u$;

the literature [89, 90, 91, 92, 93, 94]. A thorough comparative study of these algorithms and their quantum implementations is beyond the scope of this work. Our goal is to demonstrate the usefulness of the COMP-N-SUB unitary developed here. We do so by considering a widely used algorithm and constructing a quantum circuit for it. We highlight the steps that use COMP-N-SUB and explain its advantage over alternative implementations of those steps. Because the remaining steps are implemented identically, this approach permits a fair comparison of the resource reductions provided by COMP-N-SUB. For completeness, detailed resource estimates for a complete circuit implementation are given in Appendix E.

Figure 9 illustrates a quantum circuit that implements Algorithm 4. We allocate two n-qubit input registers, $(u_{n-1}, \ldots, u_0)$ and $(v_{n-1}, \ldots, v_0)$, which store the input integers u and v, respectively. With a slight abuse of notation, we also denote these registers by u and v. The register $(t_{\lceil \log n \rceil - 1}, \ldots, t_0)$ stores the number t of iterations of the loop in steps 2–4. The unitaries $U_{(2:4)}$ and $U_{(5:17)}$ implement the loops in steps 2–4 and 5–17, respectively. In Algorithm 4, the integer g, initialized to 1, is updated in every iteration of the first loop. Instead of updating g in each iteration, it is sufficient to store t and later apply the unitary c-SHIFT, which cyclically shifts the bits in register u by t places. This operation implements step 18 and yields the GCD $g = (g_{n-1}, \ldots, g_0)$ in register u.

One iteration of the loop in steps 5–17 can be implemented by the circuit shown in Figure 10. The unitary U_{odd} removes the trailing zeros from the binary representation of an integer. It is applied to both registers u and v to implement steps 6–8 and 9–11. CNS is a COMP-N-SUB unitary, and c-ADD is a controlled adder. CNS computes $v - u$ in register v if and only if $v \geq u$; otherwise, it leaves the registers unchanged and flips qubit c to $|1\rangle$. Together, the remaining CNOT gates and c-ADD implement controlled subtraction, computing $u - v$ in register u if and only if $u > v$. Thus, CNS, c-ADD, and the CNOT gates implement steps 12–16. If the loop in steps 5–17 has N_{iter} iterations, this circuit must be repeated N_{iter} times.

We now describe several unitaries used to implement $U_{(2:4)}$ and $U_{(5:17)}$.

- $\mathbf{U_{odd}^{(2)}}$**:** Let u and v be two n-bit integers such that $u = 2^t \cdot u'$ and $v = 2^t \cdot v'$ for integers u'

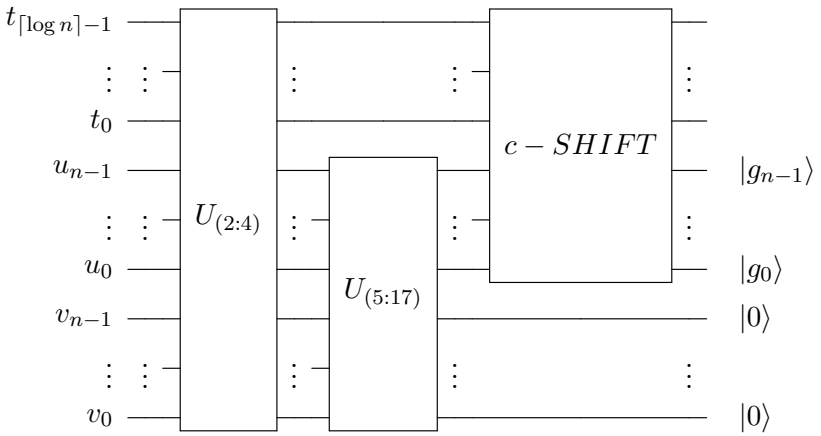


Figure 9: A quantum circuit implementing the binary GCD algorithm specified by the pseudocode in Algorithm 4. The input integers u and v are stored in the n-qubit registers $(u_{n-1}, \ldots, u_0)$ and $(v_{n-1}, \ldots, v_0)$, respectively. The unitaries $U_{(2:4)}$ and $U_{(5:17)}$ implement the loops in steps 2–4 and 5–17, respectively. The register $(t_{\lceil \log n \rceil - 1}, \ldots, t_0)$ stores the number t of iterations of the loop in steps 2–4. The unitary c-SHIFT cyclically shifts the contents of register u by t places. The n-bit GCD g is output in register u. Other qubits are omitted for clarity.

and v', at least one of which is odd. Thus, the trailing t bits of both u and v are 0, and the next bit is 1 in at least one of them. Therefore, t is the number of iterations of the loop in steps 2–4 of Algorithm 4. Our goal is to compute t, u', and v'. Appendix D.2 describes a quantum circuit implementing a unitary that takes u, v, and $|0\rangle^{\lceil \log n \rceil}$ as input and outputs u', v', and t in the corresponding registers. We denote this unitary by $U^{(2)}_{odd}$.

- **$\mathbf{U_{odd}}$:** This unitary takes as input an n-bit integer v such that $v = 2^{t'} \cdot v'$ for some odd integer v' and nonnegative integer t'. It outputs v' in the register storing v and t' in a separate register. A quantum circuit implementing U_{odd} is described in Appendix D.1.
- **c-SHIFT:** We described the operation of c-SHIFT above. A quantum circuit implementing c-SHIFT is described in Appendix D.1. Specifically, it is circuit $\mathcal{C}_{shift}$, the boxed portion of Figure 13.
- **c-ADD:** This controlled adder has two n-qubit input registers that store two n-bit integers. Conditioned on another qubit c, it adds these integers and stores the sum in one of the input registers. An implementation of c-ADD using COMP-N-SUB is described in Section 2.3.3.

Appendix E provides a thorough analysis of the resource requirements of the GCD circuit in Figure 9. We have designed several circuits for the COMP-N-SUB unitary (Section 2.2) that offer different resource trade-offs. A GCD circuit can be implemented using any of these variants. We refer to the resulting GCD implementations as circuits I, IIa, IIb, and III.

Advantage of COMP-N-SUB

The COMP-N-SUB unitary is used to implement steps 12–16 of Algorithm 4. Specifically, it is used twice in every iteration of the loop in steps 5–17 (Figure 10). Alternatively, steps 12–16 can be implemented with a comparator and two controlled subtractors, as follows.

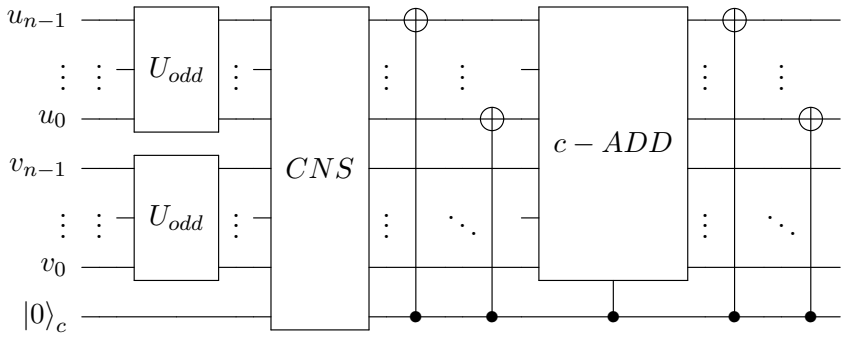


Figure 10: A quantum circuit implementing one iteration of the loop in steps 5–17 of Algorithm 4. The unitary U_{odd} removes the trailing zeros from an input integer. The CNS unitary is a COMP-N-SUB unitary that computes the difference in register v if and only if $v \geq u$. The controlled adder c-ADD conditionally computes a sum in register u. Other qubits are omitted for clarity.

1. Apply a comparator to registers u and v, using an additional qubit c initialized to $|0\rangle$. After the comparison, $c = |0\rangle$ if $v \geq u$; otherwise, $c = |1\rangle$.

2. Apply a controlled subtractor to u and v, controlled on the $|0\rangle$ state of c. The difference $v - u$ is stored in register v if $v \geq u$.

3. Apply a controlled subtractor to u and v, controlled on the $|1\rangle$ state of c. The difference $u - v$ is stored in register u if $u > v$.

As explained earlier, we assume that the cost of a controlled subtractor is approximately equal to that of COMP-N-SUB. The alternative implementation above therefore uses one extra comparator per iteration. Because a comparator costs roughly as much as an adder, our implementation saves approximately the cost of one adder each time steps 12–16 are executed. Specifically, let $\mathcal{T}(.)$ denote the T-count of a circuit. Under these approximate cost assumptions, the ratio of the T-count of our implementation to that of the alternative implementation is

$$\frac{2\mathcal{T}(\text{COMP-N-SUB})}{\mathcal{T}(\text{ADDER}) + 2\mathcal{T}(\text{COMP-N-SUB})} \approx \frac{3}{4}.$$

A similar conclusion holds for other cost metrics, such as Toffoli-count, CNOT-count, Toffoli-depth, and T-depth.[4] The qubit count, however, remains the same.

There can be at most $2n$ executions of steps 12–16 because, in each execution, the bit length of the integer in at least one of the two input registers decreases by at least one. Fewer executions are possible, but this does not affect the ratio above. Further optimizations may be possible when implementing the remaining steps of Algorithm 4. Because our focus is the usefulness of COMP-N-SUB, we restrict the comparison to the steps in which it is used. We do not quantify the percentage savings for the complete circuit because this figure depends heavily on the implementation of the other steps, which we have not optimized. Such an analysis is beyond the scope of this work.

[4]For a QRC adder, we compare against COMP-N-SUB circuit I; for a QCLA adder, we compare against COMP-N-SUB circuit IIa or IIb; and for a logical-AND-based adder, we compare against COMP-N-SUB circuit III.

Acknowledgments

We thank Robbie King and Matt Pocrnic for helpful suggestions. We received no support from external funding agencies. We also acknowledge the use of ChatGPT 5.6 for fixes in typos, grammatical errors and copyediting.

Competing Interests

The authors have no competing interests to declare.

Code and Data Availability

All relevant data are included in the manuscript.

Author Contributions

All authors developed the initial ideas, discussed the results, and prepared the manuscript. P.M. designed the circuits and estimated their resource requirements.

References

[1] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Review*, 41(2):303–332, 1999.

[2] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.

[3] Richard P Feynman. Simulating Physics with Computers. *International Journal of Theoretical Physics*, 21(6/7), 1982.

[4] Seth Lloyd. Universal quantum simulators. *Science*, 273(5278):1073–1078, 1996.

[5] Andrew M. Childs, Dmitri Maslov, Yunseong Nam, Neil J. Ross, and Yuan Su. Toward the first quantum simulation with quantum speedup. *Proceedings of the National Academy of Sciences*, 115(38):9456–9461, 2018.

[6] Earl T Campbell. Early fault-tolerant simulations of the hubbard model. *Quantum Science and Technology*, 7(1):015007, nov 2021.

[7] Andrew M. Childs and Wim van Dam. Quantum algorithms for algebraic problems. *Rev. Mod. Phys.*, 82:1–52, Jan 2010.

[8] Diego Ristè, Marcus P Da Silva, Colm A Ryan, Andrew W Cross, Antonio D Córcoles, John A Smolin, Jay M Gambetta, Jerry M Chow, and Blake R Johnson. Demonstration of quantum advantage in machine learning. *npj Quantum Information*, 3(1):16, 2017.

[9] Wim van Dam and Sean Hallgren. Efficient quantum algorithms for shifted quadratic character problems. *arXiv preprint quant-ph/0011067*, 2000.

[10] Wim Van Dam, Sean Hallgren, and Lawrence Ip. Quantum algorithms for some hidden shift problems. *SIAM Journal on Computing*, 36(3):763–778, 2006.

[11] Wim Van Dam and Igor E Shparlinski. Classical and quantum algorithms for exponential congruences. In *Workshop on Quantum Computation, Communication, and Cryptography*, pages 1–10. Springer, 2008.

[12] Michael Kirkedal Thomsen, Robert Glück, and Holger Bock Axelsen. Reversible arithmetic logic unit for quantum arithmetic. *Journal of Physics A: Mathematical and Theoretical*, 43(38):382002, 2010.

[13] Sean Hallgren. Polynomial-time quantum algorithms for Pell's equation and the principal ideal problem. *Journal of the ACM (JACM)*, 54(1):1–19, 2007.

[14] Jonathan Romero and Alán Aspuru-Guzik. Variational quantum generators: Generative adversarial quantum machine learning for continuous distributions. *Advanced Quantum Technologies*, 4(1):2000003, 2021.

[15] Hai-Sheng Li, Ping Fan, Haiying Xia, Huiling Peng, and Gui-Lu Long. Efficient quantum arithmetic operation circuits for quantum image processing. *Science China Physics, Mechanics & Astronomy*, 63(8):280311, 2020.

[16] Annie Y Wei, Preksha Naik, Aram W Harrow, and Jesse Thaler. Quantum algorithms for jet clustering. *Physical Review D*, 101(9):094015, 2020.

[17] Essam H Houssein, Mohamed A Mahdy, Manal G Eldin, Doaa Shebl, Waleed M Mohamed, and Mahmoud Abdel-Aty. Optimizing quantum cloning circuit parameters based on adaptive guided differential evolution algorithm. *Journal of Advanced Research*, 29:147–157, 2021.

[18] Ahmad M Alkadri, Tyler D Kharazi, K Birgitta Whaley, and Kranthi K Mandadapu. A Quantum Algorithm for the Finite Element Method. *arXiv preprint arXiv:2510.18150*, 2025.

[19] Laszlo Gyongyosi and Sandor Imre. Quantum circuit design for objective function maximization in gate-model quantum computers. *Quantum Information Processing*, 18(7):225, 2019.

[20] Glenn Beach, Chris Lomont, and Charles Cohen. Quantum image processing (quip). In *32nd Applied Imagery Pattern Recognition Workshop, 2003. Proceedings.*, pages 39–44. IEEE, 2003.

[21] Mihir K Bhaskar, Stuart Hadfield, Anargyros Papageorgiou, and Iasonas Petras. Quantum algorithms and circuits for scientific computing. *Quant. Inf. Comput.*, 16(arXiv: 1511.08253):0197–0236, 2016.

[22] Mathias Soeken, Martin Roetteler, Nathan Wiebe, and Giovanni De Micheli. Design automation and design space exploration for quantum computers. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pages 470–475. Ieee, 2017.

[23] Ri-Gui Zhou, Wenwen Hu, Ping Fan, and Hou Ian. Quantum realization of the bilinear interpolation method for NEQR. *Scientific Reports*, 7(1):2511, 2017.

[24] Guodong Sun, Shenghui Su, and Maozhi Xu. Quantum algorithm for polynomial root finding problem. In *2014 Tenth International Conference on Computational Intelligence and Security*, pages 469–473. IEEE, 2014.

[25] Aram W Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15):150502, 2009.

[26] Daniel Gottesman. Stabilizer Codes and Quantum Error Correction. *arXiv preprint quant-ph/9705052*, 1997.

[27] David G Cory, MD Price, W Maas, Emanuel Knill, Raymond Laflamme, Wojciech H Zurek, Timothy F Havel, and Shyamal S Somaroo. Experimental quantum error correction. *Physical Review Letters*, 81(10):2152, 1998.

[28] Sergey Bravyi and Alexei Kitaev. Universal quantum computation with ideal Clifford gates and noisy ancillas. *Physical Review A - Atomic, Molecular, and Optical Physics*, 71(2):022316, 2005.

[29] Austin G Fowler, Ashley M Stephens, and Peter Groszkowski. High-threshold universal quantum computation on the surface code. *Physical Review A - Atomic, Molecular, and Optical Physics*, 80(5):052312, 2009.

[30] Cody Jones. Low-overhead constructions for the fault-tolerant Toffoli gate. *Physical Review A - Atomic, Molecular, and Optical Physics*, 87(2):022328, 2013.

[31] Cody Jones. Composite Toffoli gate with two-round error detection. *Physical Review A - Atomic, Molecular, and Optical Physics*, 87(5):052334, 2013.

[32] Alexandru Paler, Ilia Polian, Kae Nemoto, and Simon J Devitt. Fault-tolerant, high-level quantum circuits: form, compilation and description. *Quantum Science and Technology*, 2(2):025003, 2017.

[33] Daniel Litinski. Magic state distillation: Not as costly as you think. *Quantum*, 3:205, 2019.

[34] Matthew B Hastings and Jeongwan Haah. Distillation with sublogarithmic overhead. *Physical Review Letters*, 120(5):050504, 2018.

[35] Michael Beverland, Earl Campbell, Mark Howard, and Vadym Kliuchnikov. Lower bounds on the non-Clifford resources for quantum computations. *Quantum Science and Technology*, 5(3):035009, 2020.

[36] Sergey Bravyi, Andrew W Cross, Jay M Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J Yoder. High-threshold and low-overhead fault-tolerant quantum memory. *Nature*, 627(8005):778–782, 2024.

[37] Steven A Cuccaro, Thomas G Draper, Samuel A Kutin, and David Petrie Moulton. A new quantum ripple-carry addition circuit. *arXiv preprint quant-ph/0410184*, 2004.

[38] Yasuhiro Takahashi and Noboru Kunihiro. A fast quantum circuit for addition with few qubits. *Quantum Information & Computation*, 8(6):636–649, 2008.

[39] Yasuhiro Takahashi, Seiichiro Tani, and Noboru Kunihiro. Quantum addition circuits and unbounded fan-out. *Quantum Information & Computation*, 10(9):872–890, 2010.

[40] Craig Gidney. Halving the cost of quantum addition. *Quantum*, 2:74, 2018.

[41] Thomas G Draper, Samuel A Kutin, Eric M Rains, and Krysta M Svore. A logarithmic-depth quantum carry-lookahead adder. *Quantum Information & Computation*, 6(arXiv: quant-ph/0406142):351–369, 2006.

[42] Edgard Muñoz-Coreas and Himanshu Thapliyal. Quantum circuit design of a T-count optimized integer multiplier. *IEEE Transactions on Computers*, 68(5):729–739, 2018.

[43] Alireza Khosropour, Hossein Aghababa, and Behjat Forouzandeh. Quantum division circuit based on restoring division algorithm. In *2011 Eighth International Conference on Information Technology: New Generations*, pages 1037–1040. IEEE, 2011.

[44] Lafifa Jamal and Hafiz Md Hasan Babu. Efficient approaches to design a reversible floating point divider. In *2013 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 3004–3007. IEEE, 2013.

[45] Sayanton Vhaduri Dibbo, Hafiz Md Hasan Babu, and Lafifa Jamal. An efficient design technique of a quantum divider circuit. In *2016 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 2102–2105. IEEE, 2016.

[46] Himanshu Thapliyal, Edgard Munoz-Coreas, TSS Varun, and Travis S Humble. Quantum circuit designs of integer division optimizing T-count and T-depth. *IEEE Transactions on Emerging Topics in Computing*, 9(2):1045–1056, 2019.

[47] Suzhen Yuan, Shengwei Gao, Chao Wen, Yuchan Wang, Hong Qu, and Yan Wang. A novel fault-tolerant quantum divider and its simulation: S. Yuan et al. *Quantum Information Processing*, 21(5):182, 2022.

[48] Francisco Orts, Remigijus Paulavičius, and Ernestas Filatovas. Quantum circuit optimization of an integer divider. *Journal of Systems and Software*, 215:112091, 2024.

[49] Thomas G Draper. Addition on a quantum computer. *arXiv preprint quant-ph/0008033*, 2000.

[50] Stéphane Beauregard, Gilles Brassard, and José M Fernandez. Quantum arithmetic on Galois fields. *arXiv preprint quant-ph/0301163*, 2003.

[51] Christopher M Maynard and Einar Pius. A quantum multiply-accumulator. *Quantum information processing*, 13(5):1127–1138, 2014.

[52] Archimedes Pavlidis and Dimitris Gizopoulos. Fast quantum modular exponentiation architecture for Shor's factoring algorithm. *Quantum Info. Comput.*, 14(7 & 8):649–682, May 2014.

[53] Lidia Ruiz-Perez and Juan Carlos Garcia-Escartin. Quantum arithmetic with the quantum fourier transform. *Quantum Information Processing*, 16(6):152, 2017.

[54] Engin Şahin. Quantum arithmetic operations based on quantum fourier transform on signed integers. *International Journal of Quantum Information*, 18(06):2050035, 2020.

[55] Mehdi Ramezani, Morteza Nikaeen, Farnaz Farman, Seyed Mahmoud Ashrafi, and Alireza Bahrampour. Quantum multiplication algorithm based on the convolution theorem. *Physical Review A*, 108(5):052405, 2023.

[56] GAL White, CD Hill, and LCL Hollenberg. Truncated phase-based quantum arithmetic: Error propagation and resource reduction. *Physical Review A*, 108(5):052608, 2023.

[57] Gregory D Kahanamoku-Meyer and Norman Y Yao. Fast quantum integer multiplication with zero ancillas. *arXiv preprint arXiv:2403.18006*, 2024.

[58] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge University Press, 2010.

[59] Priyanka Mukhopadhyay. Cs-count-optimal quantum circuits for arbitrary multi-qubit unitaries. *Scientific Reports*, 14(1):13916, 2024.

[60] Vadym Kliuchnikov, Dmitri Maslov, and Michele Mosca. Practical approximation of single-qubit unitaries by single-qubit quantum Clifford and T circuits. *IEEE Transactions on Computers*, 65(1):161–172, 2015.

[61] Neil J Ross and Peter Selinger. Optimal ancilla-free Clifford+T approximation of Z-rotations. *Quantum Inf. Comput.*, 16(11&12):901–953, 2016.

[62] Vlad Gheorghiu, Michele Mosca, and Priyanka Mukhopadhyay. T-count and T-depth of any multi-qubit unitary. *npj Quantum Information*, 8(1):1–10, 2022.

[63] Christoffer Hindlycke and Jan-Åke Larsson. Single-qubit rotation algorithm with logarithmic Toffoli count and gate depth. *Physical Review Research*, 6(4):L042027, 2024.

[64] Priyanka Mukhopadhyay. Synthesizing Toffoli-optimal quantum circuits for arbitrary multi-qubit unitaries. *arXiv preprint arXiv:2401.08950*, 2024.

[65] Alex Bocharov, Martin Roetteler, and Krysta M Svore. Efficient synthesis of universal repeat-until-success quantum circuits. *Physical review letters*, 114(8):080502, 2015.

[66] Alex Bocharov, Martin Roetteler, and Krysta M Svore. Efficient synthesis of probabilistic quantum circuits with fallback. *Physical Review A*, 91(5):052317, 2015.

[67] SS Gayathri, R Kumar, Samiappan Dhanalakshmi, Gerard Dooly, and Dinesh Babu Duraibabu. T-count optimized quantum circuit designs for single-precision floating-point division. *Electronics*, 10(6):703, 2021.

[68] SS Gayathri, R Kumar, and Samiappan Dhanalakshmi. Efficient floating-point division quantum circuit using Newton-Raphson division. In *Journal of Physics: Conference Series*, volume 2335, page 012058. IOP Publishing, 2022.

[69] Austin G Fowler, Matteo Mariantoni, John M Martinis, and Andrew N Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A - Atomic, Molecular, and Optical Physics*, 86(3):032324, 2012.

[70] Noor Muhammed Nayeem, Adnan Hossain, Mutasimul Haque, Lafifa Jamal, and Hafiz M Hasan Babu. Novel reversible division hardware. In *2009 52nd IEEE International Midwest Symposium on Circuits and Systems*, pages 1134–1138. IEEE, 2009.

[71] Faraz Dastan and Majid Haghparast. A novel nanometric fault tolerant reversible divider. *International Journal of the Physical Sciences*, 6(24):5671–5681, 2011.

[72] Ali Bolhassani and Majid Haghparast. Optimised reversible divider circuit. *International Journal of Innovative Computing and Applications*, 7(1):13–33, 2016.

[73] Hafiz Md Hasan Babu and Md Solaiman Mia. Design of a compact reversible fault tolerant division circuit. *Microelectronics Journal*, 51:15–29, 2016.

[74] Ismail Gassoumi, Lamjed Touil, and Bouraoui Ouni. Division circuit using reversible logic gates. In *2018 International Conference on Advanced Systems and Electric Technologies (IC_ASET)*, pages 60–65. IEEE, 2018.

[75] Hai-Sheng Li, Ping Fan, Haiying Xia, and Gui-Lu Long. The circuit design and optimization of quantum multiplier and divider. *Science China Physics, Mechanics & Astronomy*, 65(6):260311, 2022.

[76] Ping Fan and Hai-Sheng Li. Designs of the divider and special multiplier optimizing T and CNOT gates. *EPJ Quantum Technology*, 11(1):13, 2024.

[77] Craig Gidney, Noah Shutty, and Cody Jones. Magic state cultivation: growing T states as cheap as CNOT gates. *arXiv preprint arXiv:2409.17595*, 2024.

[78] Michele Mosca and Priyanka Mukhopadhyay. A polynomial time and space heuristic algorithm for T-count. *Quantum Science and Technology*, 7(1):015003, 2021.

[79] David Kremer, Ali Javadi-Abhari, and Priyanka Mukhopadhyay. Optimizing the non-Clifford-count in unitary synthesis using Reinforcement Learning. *arXiv preprint arXiv:2509.21709*, 2025.

[80] Maxime Remaud and Vivien Vandaele. Ancilla-free quantum adder with sublinear depth. In *International Conference on Reversible Computation*, pages 137–154. Springer, 2025.

[81] Francisco Orts, Gloria Ortega, Elías F Combarro, and Ester M Garzón. A review on reversible quantum adders. *Journal of Network and Computer Applications*, 170:102810, 2020.

[82] Mohammed Arifuddin Sohel, Naailah Zia, Mohd Akef Ali, and Nida Zia. Quantum computing based implementation of full adder. In *2020 IEEE International Conference for Innovation in Technology (INOCON)*, pages 1–4. IEEE, 2020.

[83] Maxime Remaud. Optimizing T and CNOT gates in quantum ripple-carry adders and comparators. In *Proceedings of Recent Advances in Quantum Computing and Technology*, pages 56–61. 2024.

[84] Maxime Remaud. Quantum adders: on the structural link between the ripple-carry and carry-lookahead techniques. *arXiv preprint arXiv:2510.00840*, 2025.

[85] Matthew Amy, Olivia Di Matteo, Vlad Gheorghiu, Michele Mosca, Alex Parent, and John Schanck. Estimating the cost of generic quantum pre-image attacks on sha-2 and sha-3. In *International Conference on Selected Areas in Cryptography*, pages 317–337. Springer, 2016.

[86] Clémence Chevignard, Pierre-Alain Fouque, and André Schrottenloher. Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves. Cryptology ePrint Archive, Paper 2026/280, 2026.

[87] Gregory D Kahanamoku-Meyer, Seyoon Ragavan, Vinod Vaikuntanathan, and Katherine Van Kirk. The Jacobi Factoring Circuit: Quantum Factoring with Near-Linear Gates and Sublinear Space and Depth. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, pages 1496–1507, 2025.

[88] Jun Li, Xinhua Peng, Jiangfeng Du, and Dieter Suter. An efficient exact quantum algorithm for the integer square-free decomposition problem. *Scientific Reports*, 2(1):260, 2012.

[89] Tudor Jebelean. Comparing several GCD algorithms. In *Proceedings of IEEE 11th Symposium on Computer Arithmetic*, pages 180–185. IEEE, 1993.

[90] Jonathan Sorenson. Two fast GCD algorithms. *Journal of Algorithms*, 16(1):110–144, 1994.

[91] Kenneth Weber. The accelerated integer GCD algorithm. *ACM Transactions on Mathematical Software (TOMS)*, 21(1):111–122, 1995.

[92] Damien Stehlé and Paul Zimmermann. A binary recursive GCD algorithm. In *International Algorithmic Number Theory Symposium*, pages 411–425. Springer, 2004.

[93] Niels Möller. On Schönhage's algorithm and subquadratic integer GCD computation. *Mathematics of Computation*, 77(261):589–607, 2008.

[94] Bernstein, Daniel J and Yang, Bo-Yin. Fast constant-time gcd computation and modular inversion. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 340–398, 2019.

[95] Thomas Haener, Mathias Soeken, Martin Roetteler, and Krysta M Svore. Quantum circuits for floating-point arithmetic. In *International Conference on Reversible Computation*, pages 162–174. Springer, 2018.

[96] Vlad Gheorghiu, Michele Mosca, and Priyanka Mukhopadhyay. A (quasi-) polynomial time heuristic algorithm for synthesizing T-depth optimal circuits. *npj Quantum Information*, 8(1):110, 2022.

Appendix

A Proofs of correctness

In this section, we prove in detail that the COMP-N-SUB circuits described in Section 2.2 are correct; that is, they produce the desired output states from a given initial state and therefore have the intended functionality.

A.1 Correctness of COMP-N-SUB circuit I

Consider the COMP-N-SUB circuit described in Section 2.2.1 and illustrated in Figure 3. We track the changes in the states of the input qubits after each step. The combined initial state of registers B and A and qubit Z is

$$\left(\bigotimes_{i=0}^{k-1} |b_i\rangle_{B_i} |a_i\rangle_{A_i}\right) |0\rangle_Z .$$

1. After applying the X gates, the state is

$$\left(\bigotimes_{i=0}^{k-1} |\overline{b_i}\rangle_{B_i} |a_i\rangle_{A_i}\right) |0\rangle_Z .$$

2. After applying the CNOT layer, the state is

$$|\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} \left(\bigotimes_{i=1}^{k-1} |\overline{b_i} \oplus a_i\rangle_{B_i} |a_i\rangle_{A_i}\right) |0\rangle_Z .$$

3. (a) After applying $\mathrm{CNOT}_{(A_{k-1};Z)}$, the state is

$$|\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} \left(\bigotimes_{i=1}^{k-1} |\overline{b_i} \oplus a_i\rangle_{B_i} |a_i\rangle_{A_i}\right) |a_{k-1}\rangle_Z .$$

   (b) After applying the CNOT ladder, the state is

$$|\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} |\overline{b_1} \oplus a_1\rangle_{B_1} |a_1\rangle_{A_1} \left(\bigotimes_{i=2}^{k-1} |\overline{b_i} \oplus a_i\rangle_{B_i} |a_i \oplus a_{i-1}\rangle_{A_i}\right) |a_{k-1}\rangle_Z .$$

4. (a) After applying the Toffoli ladder, the state is

$$\begin{aligned} & |\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} |\overline{b_1} \oplus a_1\rangle_{B_1} |a_0\overline{b_0} \oplus a_1\rangle_{A_1} \\ & \quad \cdot \left(\bigotimes_{i=2}^{k-1} |\overline{b_i} \oplus a_i\rangle_{B_i} |(\overline{b_{i-1}} \oplus a_{i-1})(c_{i-1} \oplus a_{i-1}) \oplus a_i \oplus a_{i-1}\rangle_{A_i}\right) |a_{k-1}\rangle_Z \\ = \; & |\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} |\overline{b_1} \oplus a_1\rangle_{B_1} |c_1 \oplus a_1\rangle_{A_1} \left(\bigotimes_{i=2}^{k-1} |\overline{b_i} \oplus a_i\rangle_{B_i} |c_i \oplus a_i\rangle_{A_i}\right) |a_{k-1}\rangle_Z . \end{aligned}$$

(b) After applying $\text{TOF}_{(B_{k-1},A_{k-1};Z)}$, the state is

$$\begin{aligned}
&|\overline{b_0}\rangle_{B_0}|a_0\rangle_{A_0}\left(\bigotimes_{i=1}^{k-1}|\overline{b_i}\oplus a_i\rangle_{B_i}|c_i\oplus a_i\rangle_{A_i}\right)|(\overline{b_{k-1}}\oplus a_{k-1})(c_{k-1}\oplus a_{k-1})\oplus a_{k-1}\rangle_Z\\
=\;&|\overline{b_0}\rangle_{B_0}|a_0\rangle_{A_0}\left(\bigotimes_{i=1}^{k-1}|\overline{b_i}\oplus a_i\rangle_{B_i}|c_i\oplus a_i\rangle_{A_i}\right)|c_k\rangle_Z\,.
\end{aligned}$$

5. (a) After applying $\text{TOF}_{(\overline{Z},A_{k-1};B_{k-1})}$, the state is as follows.

$$\begin{aligned}
&|\overline{b_0}\rangle_{B_0}|a_0\rangle_{A_0}\left(\bigotimes_{i=1}^{k-2}|\overline{b_i}\oplus a_i\rangle_{B_i}|c_i\oplus a_i\rangle_{A_i}\right)\\
&\cdot|\overline{c_k}(c_{k-1}\oplus a_{k-1})\oplus\overline{b_{k-1}}\oplus a_{k-1}\rangle_{B_{k-1}}|c_{k-1}\oplus a_{k-1}\rangle_{A_{k-1}}|c_k\rangle_Z
\end{aligned}$$

(b) After applying the ith pair of Toffoli gates, the state changes are

$$\begin{aligned}
|c_{k-i+1}\oplus a_{k-i+1}\rangle &\mapsto |(\overline{b_{k-i}}\oplus a_{k-i})(c_{k-i}\oplus a_{k-i})\oplus c_{k-i+1}\oplus a_{k-i+1}\rangle\\
&= |a_{k-i}\overline{b_{k-i}}\oplus\overline{b_{k-i}}c_{k-i}\oplus a_{k-i}c_{k-i}\oplus a_{k-i}\oplus c_{k-i+1}\oplus a_{k-i+1}\rangle\\
&= |a_{k-i}\oplus a_{k-i+1}\rangle \qquad [i=2,\ldots,k-1]\\
|\overline{b_{k-i}}\oplus a_{k-i}\rangle &\mapsto |\overline{c_k}(c_{k-i}\oplus a_{k-i})\oplus\overline{b_{k-i}}\oplus a_{k-i}\rangle \qquad [i=2,\ldots,k-1].
\end{aligned}$$

After applying the last pair of Toffoli gates, the state changes are

$$\begin{aligned}
|a_1\oplus c_1\rangle &\mapsto |a_0\overline{b_0}\oplus a_1\oplus c_1\rangle = |a_1\rangle\\
|\overline{b_0}\rangle &\mapsto |\overline{c_k}a_0\oplus\overline{b_0}\rangle\,.
\end{aligned}$$

Thus, after applying the ladder of Toffoli pairs, the state is as follows.

$$\begin{aligned}
&|\overline{c_k}a_0\oplus\overline{b_0}\rangle_{B_0}|a_0\rangle_{A_0}|\overline{c_k}(c_1\oplus a_1)\oplus\overline{b_1}\oplus a_1\rangle_{B_1}|a_1\rangle_{A_1}\\
&\cdot\left(\bigotimes_{i=2}^{k-1}|\overline{c_k}(c_i\oplus a_i)\oplus\overline{b_i}\oplus a_i\rangle_{B_i}|a_i\oplus a_{i-1}\rangle_{A_i}\right)|c_k\rangle_Z
\end{aligned}$$

6. After applying the second CNOT ladder, the state is

$$|\overline{c_k}a_0\oplus\overline{b_0}\rangle_{B_0}|a_0\rangle_{A_0}|\overline{c_k}(c_1\oplus a_1)\oplus\overline{b_1}\oplus a_1\rangle_{B_1}|a_1\rangle_{A_1}\left(\bigotimes_{i=2}^{k-1}|\overline{c_k}(c_i\oplus a_i)\oplus\overline{b_i}\oplus a_i\rangle_{B_i}|a_i\rangle_{A_i}\right)|c_k\rangle_Z\,.$$

7. After applying the second CNOT layer, the state is

$$|\overline{c_k}a_0\oplus\overline{b_0}\rangle_{B_0}|a_0\rangle_{A_0}\left(\bigotimes_{i=1}^{k-1}|\overline{c_k}(c_i\oplus a_i)\oplus\overline{b_i}\rangle_{B_i}|a_i\rangle_{A_i}\right)|c_k\rangle_Z\,.$$

Now, $\overline{c_k}(c_i\oplus a_i)\oplus\overline{b_i} = \overline{c_k}(c_i\oplus a_i\oplus\overline{b_i})\oplus\overline{b_i}(1\oplus\overline{c_k}) = \overline{c_k}s_i\oplus c_k\overline{b_i}$. Similarly, $\overline{c_k}a_0\oplus\overline{b_0} = \overline{c_k}s_0\oplus c_k\overline{b_0}$. Hence, we can rewrite the state above as

$$|\overline{c_k}s_0\oplus c_k\overline{b_0}\rangle_{B_0}|a_0\rangle_{A_0}\left(\bigotimes_{i=1}^{k-1}|\overline{c_k}s_i\oplus c_k\overline{b_i}\rangle_{B_i}|a_i\rangle_{A_i}\right)|c_k\rangle_Z\,.$$

8. We have $\overline{c_k}s_i \oplus c_k\overline{b_i} = \overline{c_k}(1 \oplus \overline{s_i}) \oplus c_k(1 \oplus b_i) = 1 \oplus \overline{c_k s_i} \oplus c_k b_i$. Thus, the final state after applying the layer of X gates is as follows.

$$\begin{aligned} & |\overline{c_k s_0} \oplus c_k b_0\rangle_{B_0} |a_0\rangle_{A_0} \left(\bigotimes_{i=1}^{k-1} |\overline{c_k s_i} \oplus c_k b_i\rangle_{B_i} |a_i\rangle_{A_i} \right) |c_k\rangle_Z \\ = \quad & |\overline{c_k} d_0 \oplus c_k b_0\rangle_{B_0} |a_0\rangle_{A_0} \left(\bigotimes_{i=1}^{k-1} |\overline{c_k} d_i \oplus c_k b_i\rangle_{B_i} |a_i\rangle_{A_i} \right) |c_k\rangle_Z \end{aligned}$$

If $c_k = 0$, implying $b \geq a$, we obtain $d = b - a$ in register B; otherwise, we obtain b in register B. Thus, the circuit implements COMP-N-SUB as desired.

A.2 Correctness of COMP-N-SUB circuit II

Consider the COMP-N-SUB circuit described in Section 2.2.2 and shown in Figure 4. We track the changes in the states of the input qubits after each step. Only the states of the input qubits and ancilla qubits labeled Z_i change during the application of unitaries U_1 and U_2. Hence, we write only the states of qubits A_i, B_i, and Z_i. Their combined initial state is

$$\bigotimes_{i=0}^{k-1} |a_i\rangle_{A_i} |b_i\rangle_{B_i} |0\rangle_{Z_i} .$$

1. After applying the X gates, the state is

$$\bigotimes_{i=0}^{k-1} |a_i\rangle_{A_i} |\overline{b_i}\rangle_{B_i} |0\rangle_{Z_i} .$$

2. After applying the Toffoli layer, the state is

$$\bigotimes_{i=0}^{k-1} |a_i\rangle_{A_i} |\overline{b_i}\rangle_{B_i} |a_i\overline{b_i}\rangle_{Z_i} .$$

3. After applying the CNOT layer, the state is

$$\bigotimes_{i=0}^{k-1} |a_i\rangle_{A_i} |a_i \oplus \overline{b_i}\rangle_{B_i} |a_i\overline{b_i}\rangle_{Z_i} .$$

4. After applying U_1, the state is

$$\bigotimes_{i=0}^{k-1} |a_i\rangle_{A_i} |a_i \oplus \overline{b_i}\rangle_{B_i} |c_{i+1}\rangle_{Z_i} .$$

5. After applying the CNOT layer, the state is

$$|a_0\rangle_{A_0} |a_0 \oplus \overline{b_0}\rangle_{B_0} |c_1\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-1} |a_i \oplus c_i\rangle_{A_i} |a_i \oplus \overline{b_i}\rangle_{B_i} |c_{i+1}\rangle_{Z_i} \right) .$$

6. After applying the Toffoli ladder, the state is

$$\begin{aligned}
&|a_0\rangle_{A_0} |\overline{c_k}(a_0 \oplus a_0 \oplus \overline{b_0}) \oplus c_k(a_0 \oplus \overline{b_0})\rangle_{B_0} |c_1\rangle_{Z_0} \\
&\quad \cdot \left(\bigotimes_{i=1}^{k-1} |a_i \oplus c_i\rangle_{A_i} |\overline{c_k}(a_i \oplus c_i \oplus a_i \oplus \overline{b_i}) \oplus c_k(a_i \oplus \overline{b_i})\rangle_{B_i} |c_{i+1}\rangle_{Z_i} \right) \\
= \quad & |a_0\rangle_{A_0} |\overline{c_k}(a_0 \oplus s_0) \oplus c_k(a_0 \oplus \overline{b_0})\rangle_{B_0} |c_1\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-1} |a_i \oplus c_i\rangle_{A_i} |\overline{c_k}(a_i \oplus s_i) \oplus c_k(a_i \oplus \overline{b_i})\rangle_{B_i} |c_{i+1}\rangle_{Z_i} \right)
\end{aligned}$$

7. After applying the CNOT layer, the state is

$$\bigotimes_{i=0}^{k-1} |a_i\rangle_{A_i} |\overline{c_k}(a_i \oplus s_i) \oplus c_k(a_i \oplus \overline{b_i})\rangle_{B_i} |c_{i+1}\rangle_{Z_i} .$$

8. After applying the multi-target CNOT, the state is

$$\bigotimes_{i=0}^{k-1} |a_i\rangle_{A_i} |\overline{c_k}(a_i \oplus \overline{s_i}) \oplus c_k(a_i \oplus \overline{b_i})\rangle_{B_i} |c_{i+1}\rangle_{Z_i} .$$

9. After applying U_2, the state is

$$\begin{aligned}
&\left(\bigotimes_{i=0}^{k-2} |a_i\rangle_{A_i} |\overline{c_k}(a_i \oplus \overline{s_i}) \oplus c_k(a_i \oplus \overline{b_i})\rangle_{B_i} |\overline{c_k} a_i \overline{s_i} \oplus c_k a_i \overline{b_i}\rangle_{Z_i} \right) \\
&\quad \cdot |a_{k-1}\rangle_{A_{k-1}} |\overline{c_k}(a_{k-1} \oplus \overline{s_{k-1}}) \oplus c_k(a_{k-1} \oplus \overline{b_{k-1}})\rangle_{B_{k-1}} |c_k\rangle_{Z_{k-1}} .
\end{aligned}$$

10. After applying the CNOT layer, the state is

$$\left(\bigotimes_{i=0}^{k-2} |a_i\rangle_{A_i} |\overline{c_k s_i} \oplus c_k \overline{b_i}\rangle_{B_i} |\overline{c_k} a_i \overline{s_i} \oplus c_k a_i \overline{b_i}\rangle_{Z_i} \right) |a_{k-1}\rangle_{A_{k-1}} |\overline{c_k s_{k-1}} \oplus c_k \overline{b_{k-1}}\rangle_{B_{k-1}} |c_k\rangle_{Z_{k-1}} .$$

11. After applying the Toffoli layer, the state is

$$\left(\bigotimes_{i=0}^{k-2} |a_i\rangle_{A_i} |\overline{c_k s_i} \oplus c_k \overline{b_i}\rangle_{B_i} |0\rangle_{Z_i} \right) |a_{k-1}\rangle_{A_{k-1}} |\overline{c_k s_{k-1}} \oplus c_k \overline{b_{k-1}}\rangle_{B_{k-1}} |c_k\rangle_{Z_{k-1}} .$$

12. After applying the final multi-target CNOT, the state is

$$\begin{aligned}
&\left(\bigotimes_{i=0}^{k-2} |a_i\rangle_{A_i} |\overline{c_k s_i} \oplus c_k b_i\rangle_{B_i} |0\rangle_{Z_i} \right) |a_{k-1}\rangle_{A_{k-1}} |\overline{c_k s_{k-1}} \oplus c_k b_{k-1}\rangle_{B_{k-1}} |c_k\rangle_{Z_{k-1}} \\
= \quad & \left(\bigotimes_{i=0}^{k-2} |a_i\rangle_{A_i} |\overline{c_k} d_i \oplus c_k b_i\rangle_{B_i} |0\rangle_{Z_i} \right) |a_{k-1}\rangle_{A_{k-1}} |\overline{c_k} d_{k-1} \oplus c_k b_{k-1}\rangle_{B_{k-1}} |c_k\rangle_{Z_{k-1}} .
\end{aligned}$$

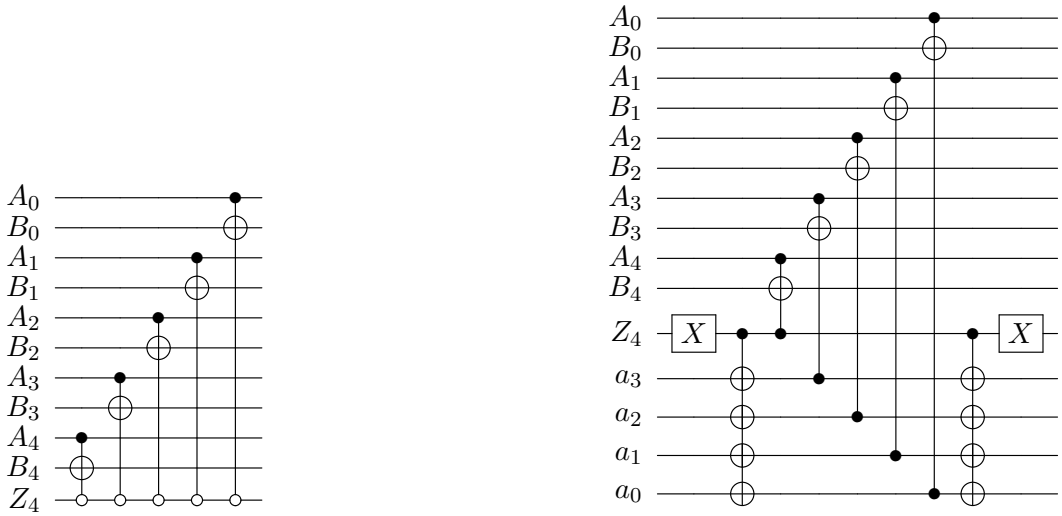


Figure 11: (a) The boxed portion of the circuit in Figure 4a. (b) Parallelization of the Toffoli gates that share the same control using additional ancilla qubits and CNOT gates. The resulting circuit has a Toffoli-depth of 1.

Hence, if $c_k = 0$, implying $b \geq a$, we obtain $d = b - a$ in register B; otherwise, we obtain b. This proves that the circuit performs as desired.

Figure 11 illustrates a technique for parallelizing Toffoli gates that share the same control. Such a pattern occurs in COMP-N-SUB circuit IIa; for example, see the boxed portion of the circuit in Figure 4a. Figure 11a reproduces this boxed portion, and Figure 11b shows an equivalent circuit with a Toffoli-depth of 1. For k Toffoli gates sharing a control, we add $k - 1$ ancilla qubits, all initialized to $|0\rangle$, and $k - 1$ CNOT gates.

A.3 Correctness of COMP-N-SUB circuit III

Consider the COMP-N-SUB circuit described in Section 2.2.3 and shown in Figure 5. We track the changes in the states of the input qubits after each step. Their combined initial state is as follows.

$$\bigotimes_{i=0}^{k-1} |b_i\rangle_{B_i} |a_i\rangle_{A_i} |0\rangle_{Z_i} .$$

The subsequent states of the qubits are as follows.

1. After applying X gates to register B, the state of the qubits is

$$\bigotimes_{i=0}^{k-1} |\overline{b_i}\rangle_{B_i} |a_i\rangle_{A_i} |0\rangle_{Z_i} .$$

2. The state of the qubits after applying the first logical-AND unitary in step 2 is

$$|\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} |c_1\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-1} |\overline{b_i}\rangle_{B_i} |a_i\rangle_{A_i} |0\rangle_{Z_i} \right) . \qquad [c_1 = a_0\overline{b_0}]$$

3. After step 3, the state of the qubits is

$$\begin{aligned} & |\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} |c_1\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-1} |\overline{b_i} \oplus c_i\rangle_{B_i} |a_i \oplus c_i\rangle_{A_i} |(\overline{b_i} \oplus c_i)(a_i \oplus c_i) \oplus c_i\rangle_{Z_i} \right) \\ = \quad & |\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} |c_1\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-1} |\overline{b_i} \oplus c_i\rangle_{B_i} |a_i \oplus c_i\rangle_{A_i} |c_{i+1}\rangle_{Z_i} \right). \end{aligned}$$

4. After step 4, the state of the qubits is as follows.

$$\begin{aligned} & |\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} |c_1\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-2} |\overline{b_i} \oplus c_i\rangle_{B_i} |a_i \oplus c_i\rangle_{A_i} |c_{i+1}\rangle_{Z_i} \right) \\ & \cdot |\overline{b_{k-1}} \oplus \overline{c_k}(a_{k-1} \oplus c_{k-1})\rangle_{B_{k-1}} |a_{k-1}\rangle_{A_{k-1}} |c_k\rangle_{Z_{k-1}} \end{aligned}$$

5. After step 5, the state of the qubits is

$$|\overline{b_0}\rangle_{B_0} |a_0\rangle_{A_0} |c_1\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-1} |\overline{b_i} \oplus \overline{c_k}(a_i \oplus c_i)\rangle_{B_i} |a_i\rangle_{A_i} |0\rangle_{Z_i} \right).$$

6. After step 6, the state of the qubits is

$$|\overline{b_0} \oplus \overline{c_k} a_0\rangle_{B_0} |a_0\rangle_{A_0} |0\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-1} |\overline{b_i} \oplus \overline{c_k}(a_i \oplus c_i)\rangle_{B_i} |a_i\rangle_{A_i} |0\rangle_{Z_i} \right).$$

7. The final state after applying the X gates is as follows.

$$|\overline{c_k s_0} \oplus c_k b_0\rangle_{B_0} |a_0\rangle_{A_0} |0\rangle_{Z_0} \left(\bigotimes_{i=1}^{k-1} |\overline{c_k s_i} \oplus c_k b_i\rangle_{B_i} |a_i\rangle_{A_i} |0\rangle_{Z_i} \right).$$

Hence, if $b \geq a$, that is, if $c_k = 0$, then we obtain $d = b - a$ in register B; otherwise, we obtain b, as desired.

B Resource estimates for RES-DIV circuits

In this section, we compute resource estimates for the RES-DIV circuits described in Section 2.3.2. Each circuit uses one COMP-N-SUB$^{(m)}$ unitary and $n - m'$ copies of the COMP-N-SUB$^{(m+1)}$ unitary, which act on m- and $(m+1)$-bit integers, respectively. Depending on the COMP-N-SUB circuit used, we refer to the resulting circuits as RES-DIV circuits I, IIa, IIb, and III. Table 3 gives the resource estimates for COMP-N-SUB.

- **Toffoli-count:** The Toffoli-count upper bounds for RES-DIV circuit I, circuits IIa and IIb (which have the same bound), and circuit III are, respectively,

$$(3m - 1) + (n - m')(3(m + 1) - 1) = 3m(n - m' + 1) + 2(n - m') - 1;$$

$$(11m-4)+(n-m')(11(m+1)-4) = 11m(n-m'+1)+7(n-m')-4;$$

and

$$m+(n-m')(m+1) = m(n-m'+1)+(n-m').$$

- **CNOT-count:** The CNOT counts of the Clifford+Toffoli implementations of the RES-DIV circuits are as follows.

$$\begin{aligned}
\text{Circuit-I:} \quad & \mathcal{N}_{Tof}^{cnot}(Res_I) = (4m-5)+(n-m')(4(m+1)-5) \\
& = 4m(n-m'+1)-(n-m')-5 && (35)\\
\text{Circuit-IIa:} \quad & \mathcal{N}_{Tof}^{cnot}(Res_{IIa}) = (6m-2)+(n-m')(6(m+1)-2) \\
& = 6m(n-m'+1)+4(n-m')-2 && (36)\\
\text{Circuit-IIb:} \quad & \mathcal{N}_{Tof}^{cnot}(Res_{IIb}) = (8m-4)+(n-m')(8(m+1)-4) \\
& = 8m(n-m'+1)+4(n-m')-4 && (37)
\end{aligned}$$

The CNOT counts of the Clifford+T implementations of the RES-DIV circuits are as follows.

$$\begin{aligned}
\text{Circuit-I:} \quad & \mathcal{N}_{T}^{cnot}(Res_I) = (25m-12)+(n-m')(25(m+1)-12) \\
& = 25m(n-m'+1)+13(n-m')-12 && (38)\\
\text{Circuit-IIa:} \quad & \mathcal{N}_{T}^{cnot}(Res_{IIa}) \leq (83m-30)+(n-m')(83(m+1)-30) \\
& = 83m(n-m'+1)+53(n-m')-30 && (39)\\
\text{Circuit-IIb:} \quad & \mathcal{N}_{T}^{cnot}(Res_{IIb}) \leq (85m-32)+(n-m')(85(m+1)-32) \\
& = 85m(n-m'+1)+53(n-m')-32 && (40)\\
\text{Circuit-III:} \quad & \mathcal{N}_{T}^{cnot}(Res_{III}) = (19m-7)+(n-m')(19(m+1)-7) \\
& = 19m(n-m'+1)+12(n-m')-7 && (41)
\end{aligned}$$

- **T-count:** The T-counts of the RES-DIV circuits are as follows.

$$\begin{aligned}
\text{Circuit-I:} \quad & \mathcal{T}(Res_I) = (18m-4)+(n-m')(18(m+1)-4) \\
& = 18m(n-m'+1)+14(n-m')-4 && (42)\\
\text{Circuit-IIa:} \quad & \mathcal{T}(Res_{IIa}) \leq (76m-21)+(n-m')(76(m+1)-21) \\
& = 76m(n-m'+1)+55(n-m')-21 && (43)\\
\text{Circuit-IIb:} \quad & \mathcal{T}(Res_{IIb}) \leq (77m-21)+(n-m')(77(m+1)-21) \\
& = 77m(n-m'+1)+56(n-m')-21 && (44)\\
\text{Circuit-III:} \quad & \mathcal{T}(Res_{III}) = 11m+(n-m')11(m+1) \\
& = 11m(n-m'+1)+11(n-m') && (45)
\end{aligned}$$

- **Qubit-count:** For all circuits, registers A and B require $m+1$ and $n+m-m'+2$ qubits, respectively. Although we append extra zeros, the quotient and remainder obtained by dividing an n-bit dividend by an m-bit divisor have lengths at most $n-m+1$ and m, respectively. The quotient and remainder are computed in register B. Thus, at the end of the computation, $(n+m-m'+2)-(n-m+1)-m = m-m'+1$ qubits of B return to the $|0\rangle$ state. Register

A remains unchanged, and $A_m = |0\rangle$ at the end of the computation. Hence, all circuits have at least $m - m' + 2$ ancilla qubits. In addition, n and m input qubits store the dividend and divisor, respectively.

RES-DIV circuit I has no garbage qubits. Its total qubit count is $(n+m-m'+2)+(m+1) = n + 2m - m' + 3$, of which $m - m' + 2$ are ancilla qubits, as explained above.

For RES-DIV circuits IIa, IIb, and III, each iteration requires at most $2m-\omega(m)-\lfloor\log_2 m\rfloor-1$, $3m-\omega(m)-\lfloor\log_2 m\rfloor-2$, and $m-1$ additional ancilla qubits, respectively. These qubits are restored to their initial states at the end of each iteration and can therefore be reused in the next iteration. Thus, the total qubit counts of circuits IIa, IIb, and III are $n + 4m - m' - \omega(m) - \lfloor\log_2 m\rfloor + 2$, $n + 5m - m' - \omega(m) - \lfloor\log_2 m\rfloor + 1$, and $n + 3m - m' + 2$, respectively.

- **Toffoli-depth:** The Toffoli depths of the RES-DIV circuits are as follows.

$$\begin{aligned}
\text{Circuit-I:} \quad & (3m-1) + (n-m')(3(m+1)-1) \\
& = 3m(n-m'+1) + 2(n-m') - 1 \\
\text{Circuit-IIa:} \quad & \leq (m + 4\lfloor\log_2 m\rfloor + 8) + (n-m')(m + 1 + 4\lfloor\log_2(m+1)\rfloor + 8) \\
& \leq (n-m'+1)\,(m + 4\lfloor\log_2(m+1)\rfloor) + 9(n-m') + 8 \\
\text{Circuit-IIb:} \quad & \leq (4\lfloor\log_2 m\rfloor + 9) + (n-m')(4\lfloor\log_2(m+1)\rfloor + 9) \\
& \leq (n-m'+1)\,(4\lfloor\log_2(m+1)\rfloor + 9) \\
\text{Circuit-III:} \quad & m + (n-m')(m+1) = m(n-m'+1) + (n-m')
\end{aligned}$$

- **T-depth:** The T-depths of the RES-DIV circuits are as follows.

$$\begin{aligned}
\text{Circuit-I:} \quad & \mathcal{T}_d(Res_I) = (9m-3) + (n-m')(9(m+1)-3) \\
& = 9m(n-m'+1) + 6(n-m') - 3 && (46) \\
\text{Circuit-IIa:} \quad & \mathcal{T}_d(Res_{IIa}) \leq (3m + 12\lfloor\log_2 m\rfloor + 24) + (n-m')(3m + 3 + 12\lfloor\log_2(m+1)\rfloor + 24) \\
& \leq 3(n-m'+1)\,(m + 4\lfloor\log_2(m+1)\rfloor) + 27(n-m') + 24 && (47) \\
\text{Circuit-IIb:} \quad & \mathcal{T}_d(Res_{IIb}) \leq (12\lfloor\log_2 m\rfloor + 27) + (n-m')(12\lfloor\log_2(m+1)\rfloor + 27) \\
& \leq 3(n-m'+1)\,(4\lfloor\log_2(m+1)\rfloor + 9) && (48) \\
\text{Circuit-III:} \quad & \mathcal{T}_d(Res_{III}) = 5m + (n-m')5(m+1) = 5m(n-m'+1) + 5(n-m') && (49)
\end{aligned}$$

- **Measurements:** No measurements are required for RES-DIV circuits I, IIa, and IIb. The number of measurements required for RES-DIV circuit III is at most

$$m + (n-m')(m+1) = nm + n + m - m' - mm'.$$

C Clifford+T quantum circuits for pairs of Toffoli gates and pairs of CSWAP gates

In this section, we show that if two Toffoli gates share a control or target qubit, then the T-count of the resulting unitary is at most 12. Figure 12a shows a quantum circuit for two Toffoli gates that share a control qubit. The resulting unitary is

$$\mathrm{TOF}_{(q_1,q_2;q_3)} \cdot \mathrm{TOF}_{(q_1,q_4;q_5)}.$$

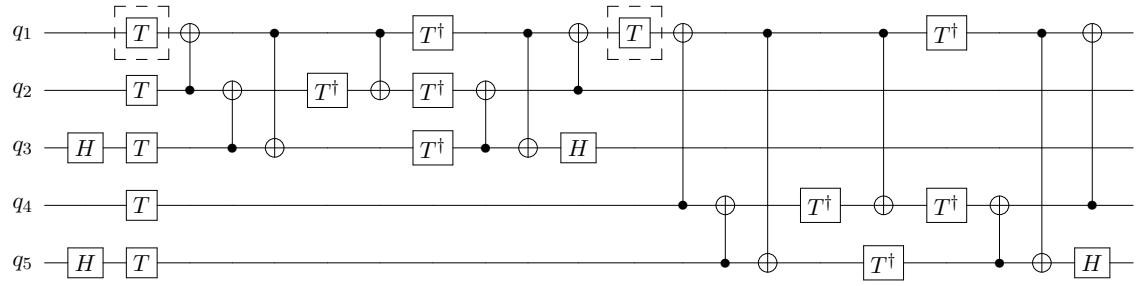
The circuit is obtained from the Clifford+T circuit for the Toffoli gate in Figure 1b. The input to each of the two T gates on qubit q_1 (outlined by dashed boxes in the figure) is the state $|q_1\rangle$. Hence, these T gates can be combined into a single S (phase) gate, which is a Clifford gate. The T-depth and CNOT-count of this circuit are at most 5 and 14, respectively.

Similarly, in the circuit for two Toffoli gates that share a target qubit, shown in Figure 12b, two T gates can be combined into a single S gate. Thus, in both the shared-control and shared-target cases, the T-count of the overall unitary is at most 12. For the shared-target circuit, the T-depth and CNOT-count are at most 5 and 12, respectively.

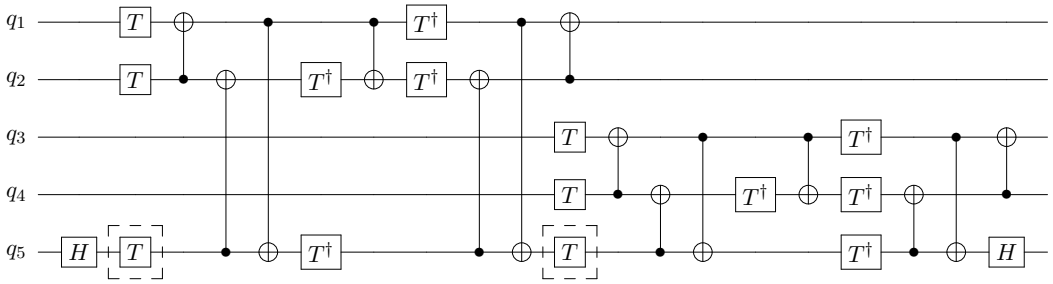
A similar argument applies to two CSWAP (Fredkin) gates that share a control, using the identity

$$\mathrm{CSWAP}_{(q_1;q_2,q_3)} = \mathrm{CNOT}_{(q_3;q_2)} \cdot \mathrm{TOF}_{(q_1,q_2;q_3)} \cdot \mathrm{CNOT}_{(q_3;q_2)}.$$

The unitary $\mathrm{CSWAP}_{(q_1;q_2,q_3)} \cdot \mathrm{CSWAP}_{(q_1;q_4,q_5)}$ has T-count, T-depth, and CNOT-count at most 12, 5, and 18, respectively. If two CSWAP gates share both the control and one target (for example, $\mathrm{CSWAP}_{(q_1;q_2,q_3)} \cdot \mathrm{CSWAP}_{(q_1;q_3,q_4)}$), then the T-count of the combined unitary is 8, as shown in [79].



(a)



(b)

Figure 12: (a) Circuit for $\mathrm{TOF}_{(q_1,q_2;q_3)} \cdot \mathrm{TOF}_{(q_1,q_4;q_5)}$, in which the two Toffoli gates share a control qubit. (b) Circuit for $\mathrm{TOF}_{(q_1,q_2;q_5)} \cdot \mathrm{TOF}_{(q_3,q_4;q_5)}$, in which the two Toffoli gates share a target qubit. In both circuits, the boxed T gates can be combined and replaced by the Clifford phase gate S.

D Removing trailing zeros from binary integers

We first describe a quantum circuit that removes the trailing zeros from a binary integer and outputs the resulting odd integer. We then describe a quantum circuit that performs a similar operation on a pair of integers.

D.1 Quantum circuit for removing trailing zeros from a single integer

Suppose that an integer v has t trailing zero bits, so that $v = 2^t \cdot v'$ for some odd integer v' and nonnegative integer t. We want to compute the binary representation of v' by removing the t trailing zeros from v. A quantum circuit for this task is described in [95] and reproduced in Figure 13. The circuit consists of two parts. The first part (the unboxed portion of Figure 13) counts the trailing zeros, and the second part (the boxed portion of Figure 13) computes v' by cyclically shifting the bits of v. We refer to the unitary implemented by this circuit as U_{odd}.

If $v = (v_{n-1}, \ldots, v_0)$ is an n-bit integer, we allocate n qubits to store its binary representation. We allocate $\lceil \log n \rceil$ qubits, $(t_{\lceil \log n \rceil - 1}, \ldots, t_0)$, all initialized to $|0\rangle$, to compute the binary representation of t, the number of trailing zeros of v. An additional qubit, a, is initialized to $|1\rangle$ by applying X to the $|0\rangle$ state. If n is not a power of two, we allocate $n' - n$ additional ancilla qubits, where $n' = 2^{\lceil \log n \rceil}$. These qubits are initialized to $|0\rangle$ and remain in that state throughout the computation. They are required for the second part of the circuit and are labeled $v_{n'-1}, \ldots, v_n$.

We first describe the part of the circuit that computes t (the unboxed portion of Figure 13). We write $C^{|\mathcal{S}|}X_{(\mathcal{S};x)}$ for a multi-controlled X gate whose controls are the $|\mathcal{S}|$ qubits in the set $\mathcal{S}$ and whose target is qubit x. We refer to this part of the circuit as $\mathcal{C}_t$.

1. Apply $\mathrm{CNOT}_{(v_0;a)}$.
2. For $i = 1, \ldots, n-1$:
   (a) Write the binary decomposition of i as $(i_{\lceil \log n \rceil - 1}, \ldots, i_0)$.
   (b) Set $\mathcal{S}_i = \{j : i_j = 1\}$.
   (c) Apply $\prod_{j \in \mathcal{S}_i} \mathrm{TOF}_{(a, v_i; t_j)}$.
   (d) Apply $C^{|\mathcal{S}_i|}X_{(\mathcal{S}_i;a)}$.

Given t, we can compute $v' = v/2^t$ by cyclically shifting the binary representation of v by t places so that the least significant bit becomes 1 and all trailing zeros are moved to the most significant positions. This can be done by applying groups of CSWAP gates, as shown in the boxed portion of Figure 13. Recall that $n' = 2^{\lceil \log n \rceil}$. We refer to this part of the circuit as $\mathcal{C}_{shift}$.

1. For $j = 0, \ldots, \log n' - 1$:
   (a) For $i = 0, \ldots, n' - 2^j - 1$, apply $\mathrm{CSWAP}_{(t_j; v_i, v_{i+2^j})}$.

Before deriving the resource estimates, we describe a useful circuit optimization.

Optimization of the circuit in Figure 14(a)

Consider a unitary implemented by a set of k Toffoli gates that share both control qubits, as shown in Figure 14(a). The corresponding unitary is

$$\prod_{j=1}^{k} \mathrm{TOF}_{(c_1, c_2; q_j)}.$$

Figure 14(b) shows an equivalent circuit, implemented as follows.

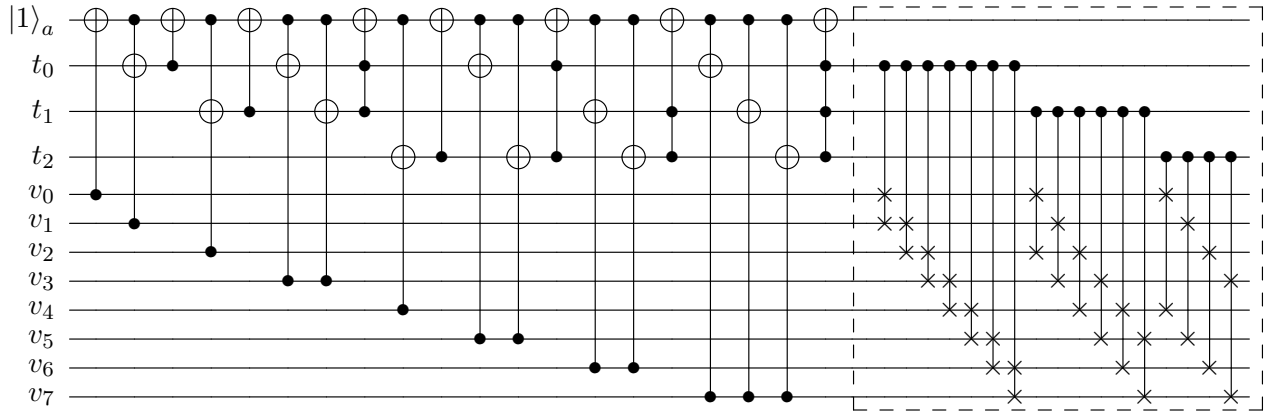


Figure 13: A quantum circuit that counts the trailing zeros in the binary representation of an 8-bit integer v and then computes $v' = v/2^t$. The tuples $(v_7, \ldots, v_0)$ and (t_2, t_1, t_0) encode the integers v and t, respectively. The boxed portion of the circuit shifts the bits of v by t positions so that the least significant bit of the output integer v' is 1. This circuit was described in [95].

1. For $j = 2, \ldots, k$, apply $\text{CNOT}_{(q_1;q_j)}$.
2. Apply $\text{TOF}_{(c_1,c_2;q_1)}$.
3. For $j = 2, \ldots, k$, apply $\text{CNOT}_{(q_1;q_j)}$.

We can verify the equivalence by tracing the qubit states after each step. The output state of the circuit in Figure 14(a) is

$$|c_1\rangle \left(\bigotimes_{j=1}^{k} |c_1 c_2 \oplus q_j\rangle \right) |c_2\rangle \,. \tag{50}$$

Suppose that the input state of the circuit in Figure 14(b) is

$$|c_1\rangle \left(\bigotimes_{j=1}^{k} |q_j\rangle \right) |c_2\rangle \,.$$

After step 1, the state is

$$|c_1\rangle \, |q_1\rangle \left(\bigotimes_{j=2}^{k} |q_1 \oplus q_j\rangle \right) |c_2\rangle \,.$$

After step 2, the state is

$$|c_1\rangle \, |c_1 c_2 \oplus q_1\rangle \left(\bigotimes_{j=2}^{k} |q_1 \oplus q_j\rangle \right) |c_2\rangle \,.$$

After step 3, the state is

$$\begin{aligned} & |c_1\rangle\,|c_1c_2 \oplus q_1\rangle \left(\bigotimes_{j=2}^{k} |c_1c_2 \oplus q_1 \oplus q_1 \oplus q_j\rangle\right) |c_2\rangle \\ = \quad & |c_1\rangle \left(\bigotimes_{j=1}^{k} |c_1c_2 \oplus q_j\rangle\right) |c_2\rangle\,; \end{aligned}$$

This is the same as the state in Equation 50, proving that the circuits in Figures 14(a) and 14(b) are equivalent.

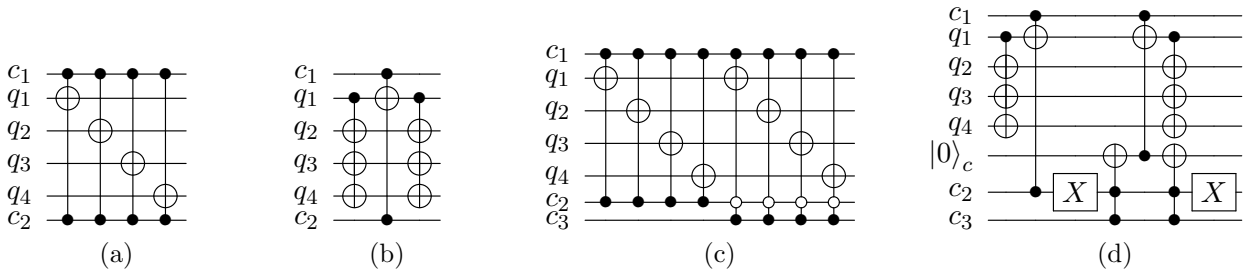


Figure 14: (a) A quantum circuit consisting of a group of Toffoli gates that share both control qubits. (b) A circuit equivalent to (a). (c) A quantum circuit consisting of a group of Toffoli gates that share both control qubits and a group of C^3X gates that share all three control qubits. (d) A circuit equivalent to (c).

Resource estimates

We now count the gates and qubits required to implement U_{odd}. We will use the following identity repeatedly.

$$\begin{aligned} & \sum_{j=2}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j} (j-1) \\ = \quad & \left(-\binom{\lceil \log n \rceil}{1} + \sum_{j=1}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j}\right) - \left(-1 - \lceil \log n \rceil + \sum_{j=0}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j}\right) \\ = \quad & \left(\lceil \log n \rceil 2^{\lceil \log n \rceil - 1} - \lceil \log n \rceil\right) - \left(2^{\lceil \log n \rceil} - \lceil \log n \rceil - 1\right) \\ = \quad & \lceil \log n \rceil 2^{\lceil \log n \rceil - 1} - 2^{\lceil \log n \rceil} + 1 \end{aligned} \tag{51}$$

- **Toffoli-count:** Consider $\mathcal{C}_t$, the unboxed part of the circuit in Figure 13, which computes t. In step 2(c), we apply a product of Toffoli gates similar to that in Figure 14(a). As discussed above, a product of k Toffoli gates can be replaced by an equivalent circuit containing one Toffoli gate and $k-1$ CNOT gates. Thus, the $n-1$ iterations require $n-1$ Toffoli gates.

  In step 2(d), we require multi-controlled X gates whenever the binary representation of i contains at least two 1s. For any $k \geq 2$, a C^kX gate can be implemented with $k-1$ Toffoli

gates and $k-1$ additional ancilla qubits. To reuse these ancillas, we add Toffoli gates to uncompute them.

Thus, the number of Toffoli gates required to implement $\mathcal{C}_t$ is at most

$$\begin{aligned} & (n-1) + 2\sum_{j=2}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j} (j-1) \\ = \;& \lceil \log n \rceil 2^{\lceil \log n \rceil} + n - 2^{\lceil \log n \rceil + 1} + 1 \qquad \text{[Using Equation 51]} \\ \leq \;& \lceil \log n \rceil 2^{\lceil \log n \rceil} - n + 1. \qquad [\because 2^{\lceil \log n \rceil} \geq n] \end{aligned}$$

Next, consider $\mathcal{C}_{shift}$, the boxed part of the circuit in Figure 13, which applies a cyclic shift. In iteration j, we require $2^{\lceil \log n \rceil} - 2^j$ CSWAP gates. Each CSWAP gate can be implemented with one Toffoli gate. Thus, the number of Toffoli gates required is at most

$$\sum_{j=0}^{\lceil \log n \rceil - 1} \left(2^{\lceil \log n \rceil} - 2^j\right) = \lceil \log n \rceil 2^{\lceil \log n \rceil} - 2^{\lceil \log n \rceil} + 1 \leq \lceil \log n \rceil 2^{\lceil \log n \rceil} + 1. \tag{52}$$

Hence, a Clifford+Toffoli implementation of U_{odd} requires at most the following number of Toffoli gates:

$$\lceil \log n \rceil 2^{\lceil \log n \rceil + 1} + n - 3 \cdot 2^{\lceil \log n \rceil} + 2 \leq \lceil \log n \rceil 2^{\lceil \log n \rceil + 1} - 2n + 2. \tag{53}$$

- **T-count:** The compute–uncompute pairs of multi-controlled X gates in step 2(d) of $\mathcal{C}_t$ can be implemented with logical-AND gadgets [40]. The remaining Toffoli gates are implemented using the construction in [58], which has a T-count of 7. Thus, the number of T gates required to implement $\mathcal{C}_t$ is at most

$$\begin{aligned} & 7(n-1) + 4\sum_{j=2}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j} (j-1) \\ = \;& \lceil \log n \rceil 2^{\lceil \log n \rceil + 1} + 7n - 4 \cdot 2^{\lceil \log n \rceil} - 3 \qquad \text{[Using Equation 51]} \\ \leq \;& \lceil \log n \rceil 2^{\lceil \log n \rceil + 1} + 3n - 3. \end{aligned}$$

Next, consider $\mathcal{C}_{shift}$. The CSWAP gates in the first group share a control and a target, so their T-count has the following upper bound [79]:

$$4 \cdot 2^{\lceil \log n \rceil} - 1, \qquad \text{since } 2^{\lceil \log n \rceil} \text{ is even.}$$

The CSWAP gates in the second group share a control, so we can apply the construction discussed in Section C. Because $2^{\lceil \log n \rceil} - 2^j$ is even, the T-count of group j is at most

$$6(2^{\lceil \log n \rceil} - 2^j).$$

Thus, the T-count of $\mathcal{C}_{shift}$ is at most

$$\begin{aligned} & 4 \cdot 2^{\lceil \log n \rceil} - 1 + \sum_{j=1}^{\lceil \log n \rceil - 1} 6(2^{\lceil \log n \rceil} - 2^j) \\ = \; & 4 \cdot 2^{\lceil \log n \rceil} - 1 + 6 \sum_{j=1}^{\lceil \log n \rceil - 1} 2^{\lceil \log n \rceil} - 12 \sum_{j=0}^{\lceil \log n \rceil - 2} 2^j \\ = \; & 6\lceil \log n \rceil 2^{\lceil \log n \rceil} - 8 \cdot 2^{\lceil \log n \rceil} + 11. \end{aligned} \tag{54}$$

Hence, a Clifford+T implementation of U_{odd} requires at most the following number of T gates:

$$8\lceil \log n \rceil 2^{\lceil \log n \rceil} + 7n - 12 \cdot 2^{\lceil \log n \rceil} + 8 \leq 8\lceil \log n \rceil 2^{\lceil \log n \rceil} - 5n + 8. \tag{55}$$

- **CNOT-count:** First, consider $\mathcal{C}_t$. We apply a CNOT gate in step 1. In step 2(c), using the optimization shown in Figures 14(a) and (b), we apply $2(k-1)$ CNOT gates for each group of k Toffoli gates, where $k \geq 2$. Thus, CNOT gates are required whenever the binary representation of i contains at least two 1s. In step 2(d), we apply a CNOT gate when $|\mathcal{S}_i| = 1$. Hence, the number of CNOT gates required is at most

$$\begin{aligned} & 1 + \left(2 \sum_{j=2}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j} (j-1) \right) + (\lceil \log n \rceil - 1) \\ = \; & (\lceil \log n \rceil - 2) 2^{\lceil \log n \rceil} + \lceil \log n \rceil + 2. \qquad \text{[Using Equation 51]} \end{aligned}$$

Next, consider $\mathcal{C}_{shift}$. Each CSWAP gate can be implemented by a Toffoli gate conjugated by two CNOT gates. Thus, the number of CNOT gates required is at most twice the Toffoli count in Equation 52. Hence, a Clifford+Toffoli implementation of U_{odd} requires at most the following number of CNOT gates:

$$\begin{aligned} & (\lceil \log n \rceil - 2) 2^{\lceil \log n \rceil} + \lceil \log n \rceil + 2 + 2 \left(\lceil \log n \rceil 2^{\lceil \log n \rceil} - 2^{\lceil \log n \rceil} + 1 \right) \\ = \; & (3\lceil \log n \rceil - 4) 2^{\lceil \log n \rceil} + \lceil \log n \rceil + 4. \end{aligned} \tag{56}$$

For a Clifford+T implementation, we account for the 7, 6, and 9 CNOT gates required by each Toffoli, logical-AND, and CSWAP implementation, respectively. Hence, the number of CNOT gates required is at most

$$\begin{aligned} & 7(n-1) + 6 \left(\sum_{j=2}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j} (j-1) \right) + 9 \left(\lceil \log n \rceil 2^{\lceil \log n \rceil} - \lceil \log n \rceil + 1 \right) \\ & + \left((3\lceil \log n \rceil - 4) 2^{\lceil \log n \rceil} + \lceil \log n \rceil + 4 \right) \\ = \; & 15\lceil \log n \rceil 2^{\lceil \log n \rceil} + \lceil \log n \rceil + 7n - 19 \cdot 2^{\lceil \log n \rceil} + 12 \qquad \text{[Using Equation 51]} \\ \leq \; & 15\lceil \log n \rceil 2^{\lceil \log n \rceil} + \lceil \log n \rceil - 12n + 12. \end{aligned} \tag{57}$$

- **Qubit-count:** The circuit uses n input qubits to store the integer v. We prepend $2^{\lceil \log n \rceil} - n$ additional ancilla qubits, which return to their initial $|0\rangle$ states and can therefore be reused in later computations. We also require $\lceil \log n \rceil - 1$ ancilla qubits to apply the MCX gates. These ancillas are restored to their initial states after each compute–uncompute MCX pair. The circuit leaves $\lceil \log n \rceil + 1$ qubits in non-restored states: qubit a and the $\lceil \log n \rceil$ qubits that store t. At the component level, the latter are output qubits, while a is a garbage qubit. When U_{odd} is used within the GCD circuit, t is not part of the final GCD output, so all $\lceil \log n \rceil + 1$ non-restored qubits count as garbage. In summary, in addition to the n input qubits, we require $2^{\lceil \log n \rceil} + \lceil \log n \rceil - n - 1$ ancilla qubits, $\lceil \log n \rceil$ output qubits, and one garbage qubit; within the GCD circuit, the output and garbage qubits together are counted as $\lceil \log n \rceil + 1$ garbage qubits.

- **Toffoli-depth:** For a Clifford+Toffoli implementation, the Toffoli depth does not exceed the Toffoli count:

$$\lceil \log n \rceil 2^{\lceil \log n \rceil + 1} + n - 3 \cdot 2^{\lceil \log n \rceil} + 2 \leq \lceil \log n \rceil 2^{\lceil \log n \rceil + 1} - 2n + 2. \tag{58}$$

- **T-depth:** The optimal T-depth of the Toffoli construction in [58] is 3 [96], while that of a logical-AND gadget is 2. Hence, the T-depth of $\mathcal{C}_t$ is at most

$$\begin{aligned} & 3(n-1) + 2 \sum_{j=2}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j} (j-1) \\ = \;& \lceil \log n \rceil 2^{\lceil \log n \rceil} + 3n - 2 \cdot 2^{\lceil \log n \rceil} - 1 \qquad \text{[Using Equation 51]} \\ \leq \;& \lceil \log n \rceil 2^{\lceil \log n \rceil} + n - 1. \end{aligned}$$

  The optimal T-depth of a CSWAP gate is 3 [96]. Thus, the T-depth of $\mathcal{C}_{shift}$ is at most

$$3 \sum_{j=0}^{\lceil \log n \rceil - 1} \left(2^{\lceil \log n \rceil} - 2^j \right) = 3 \lceil \log n \rceil 2^{\lceil \log n \rceil} - 3 \cdot 2^{\lceil \log n \rceil} + 3. \tag{59}$$

  Hence, the T-depth of a Clifford+T implementation of U_{odd} is at most

$$4 \lceil \log n \rceil 2^{\lceil \log n \rceil} + 3n - 5 \cdot 2^{\lceil \log n \rceil} + 2 \leq 4 \lceil \log n \rceil 2^{\lceil \log n \rceil} - 2n + 2. \tag{60}$$

D.2 Removing trailing zeros from a pair of integers

In this section, we consider a pair of n-bit integers, u and v, such that $u = 2^t \cdot u'$ and $v = 2^t \cdot v'$ for integers u' and v', at least one of which is odd. Thus, the t least significant bits in the binary representations of both u and v are 0, and the next bit is 1 in at least one representation. Our goal is to compute u' and v'. We follow a procedure similar to that in Appendix D.1: first, we compute t, and then we cyclically shift the bits of u and v by t places. We denote this unitary by $U^{(2)}_{odd}$.

As in the circuit for U_{odd} (Figure 13), we allocate $\lceil \log n \rceil$ qubits to store the binary representation of t. We also allocate n qubits each to store the binary representations $u = (u_{n-1}, \ldots, u_0)$ and $v = (v_{n-1}, \ldots, v_0)$. We call these the u and v registers, respectively. Qubit a is initialized to the $|1\rangle$ state. If n is not a power of two, we prepend $2^{\lceil \log n \rceil} - n$ ancilla qubits to each of the u and v registers. These qubits remain in the $|0\rangle$ state throughout the computation.

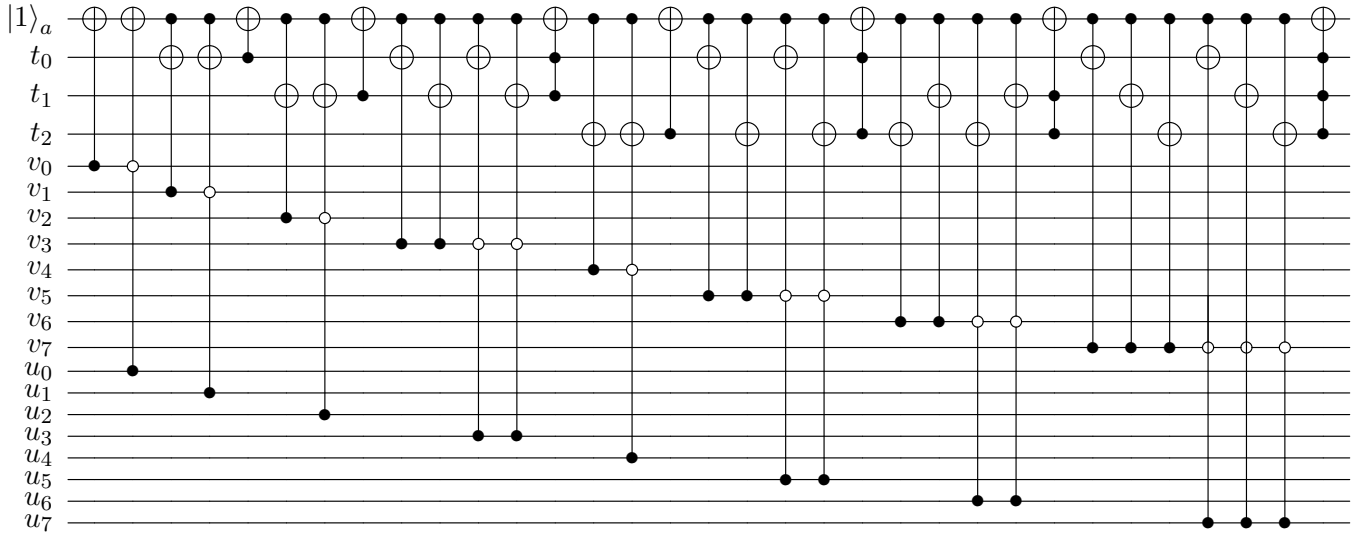


Figure 15: A quantum circuit that takes two 8-bit integers, u and v, with binary representations $(u_7, \ldots, u_0)$ and $(v_7, \ldots, v_0)$, respectively. It computes the largest integer t such that the binary representations of both u and v end in at least t consecutive zeros. The tuple (t_2, t_1, t_0) is the binary representation of t.

First, we describe the circuit that computes t. It is obtained by modifying $\mathcal{C}_t$, the unboxed part of the circuit in Figure 13, to operate on a pair of integers. We refer to the resulting circuit as $\mathcal{C}_t^{(2)}$; Figure 15 illustrates it.

1. Apply $\mathrm{CNOT}_{(v_0;a)} \cdot \mathrm{TOF}_{(u_0,\overline{v_0});a}$.
2. For $i = 1, \ldots, n-1$:
   (a) Set $(i_{\lceil \log n \rceil - 1}, \ldots, i_0) \leftarrow$ the binary representation of i.
   (b) Set $\mathcal{S}_i = \{j : i_j = 1\}$.
   (c) Apply $\left(\prod_{j \in \mathcal{S}_i} \mathrm{TOF}_{(a,v_i;t_j)}\right) \cdot \left(\prod_{j \in \mathcal{S}_i} C^3X_{(a,\overline{v_i},u_i;t_j)}\right)$.
   (d) Apply $C^{|\mathcal{S}_i|}X_{(\mathcal{S}_i;a)}$.

The circuit that cyclically shifts the bits of u and v by t places is the same as $\mathcal{C}_{shift}$, the boxed part of the circuit in Figure 13. Here, however, we apply it twice: once to register u and once to register v. Both applications are controlled by qubits $t_{\lceil \log n \rceil - 1}, \ldots, t_0$.

Resource estimates

We now count the gates and qubits required to implement $U_{odd}^{(2)}$.

- **Toffoli-count:** We require one Toffoli gate in step 1. Another significant difference between $\mathcal{C}_t$ and $\mathcal{C}_t^{(2)}$ occurs in step 2(c), where we apply an additional group of C^3X gates. As shown in Figures 14(a) and (b), the first group of Toffoli gates can be replaced by one Toffoli gate and a few CNOT gates. We can apply the same optimization to the second group of C^3X

gates, as illustrated in Figures 14(c) and (d). We first decompose each C^3X gate into Toffoli gates using an additional ancilla qubit c, initialized to $|0\rangle$. This qubit can be restored to $|0\rangle$ later. Hence, we obtain the following identity.

$$\prod_{j\in\mathcal{S}_i} C^3X_{(a,\overline{v_i},u_i;t_j)} = \mathrm{TOF}_{(\overline{v_i},u_i;c)} \cdot \left(\prod_{j\in\mathcal{S}_i} \mathrm{TOF}_{(a,c;t_j)}\right) \cdot \mathrm{TOF}_{(\overline{v_i},u_i;c)}$$

We then apply the circuit equivalence shown in Figures 14(a) and (b) and cancel the intermediate CNOT gates.

The remaining steps are identical to those of $\mathcal{C}_t$. Hence, the number of Toffoli gates required for $\mathcal{C}_t^{(2)}$ is at most

$$\begin{aligned} & 1 + 4(n-1) + 2\sum_{j=2}^{\lceil \log n\rceil} \binom{\lceil \log n\rceil}{j}(j-1) \\ = \;& \lceil \log n\rceil 2^{\lceil \log n\rceil} + 4n - 2\cdot 2^{\lceil \log n\rceil} - 1 \qquad \text{[Using Equation 51]} \\ \leq \;& \lceil \log n\rceil 2^{\lceil \log n\rceil} + 2n - 1. \end{aligned}$$

We then apply two copies of $\mathcal{C}_{shift}$. Using Equation 52, the number of Toffoli gates required for a Clifford+Toffoli implementation of $U_{odd}^{(2)}$ is therefore at most

$$\begin{aligned} & \lceil \log n\rceil 2^{\lceil \log n\rceil} + 4n - 2\cdot 2^{\lceil \log n\rceil} - 1 + 2\left(\lceil \log n\rceil 2^{\lceil \log n\rceil} - 2^{\lceil \log n\rceil} + 1\right) \\ = \;& 3\lceil \log n\rceil 2^{\lceil \log n\rceil} + 4n - 4\cdot 2^{\lceil \log n\rceil} + 1 \leq 3\lceil \log n\rceil 2^{\lceil \log n\rceil} + 1. \end{aligned} \tag{61}$$

- **T-count:** The multi-controlled X gates in step 2(d) can be implemented with logical-AND gadgets, as described for $\mathcal{C}_t$. Step 2(c) requires four Toffoli gates, two of which form a compute–uncompute pair and can therefore be implemented using a logical-AND gadget. Thus, the number of T gates required to implement $\mathcal{C}_t^{(2)}$ is at most

$$\begin{aligned} & 7 + 7\cdot 2(n-1) + 4(n-1) + 4\sum_{j=2}^{\lceil \log n\rceil} \binom{\lceil \log n\rceil}{j}(j-1) \\ = \;& 2\lceil \log n\rceil 2^{\lceil \log n\rceil} + 18n - 4\cdot 2^{\lceil \log n\rceil} - 7 \qquad \text{[Using Equation 51]} \\ \leq \;& 2\lceil \log n\rceil 2^{\lceil \log n\rceil} + 14n - 7. \end{aligned}$$

Because we require two copies of $\mathcal{C}_{shift}$, Equation 54 implies that the number of T gates required for a Clifford+T implementation of $U_{odd}^{(2)}$ is at most

$$\begin{aligned} & 2\lceil \log n\rceil 2^{\lceil \log n\rceil} + 18n - 4\cdot 2^{\lceil \log n\rceil} - 7 + 2\left(6\lceil \log n\rceil 2^{\lceil \log n\rceil} - 8\cdot 2^{\lceil \log n\rceil} + 11\right) \\ = \;& 14\lceil \log n\rceil 2^{\lceil \log n\rceil} + 18n - 20\cdot 2^{\lceil \log n\rceil} + 15 \leq 14\lceil \log n\rceil 2^{\lceil \log n\rceil} - 2n + 15. \end{aligned} \tag{62}$$

- **CNOT-count:** The number of CNOT gates required for a Clifford+Toffoli implementation of $U_{odd}^{(2)}$ is the same as that for U_{odd} and is at most

$$(3\lceil \log n\rceil - 4)2^{\lceil \log n\rceil} + \lceil \log n\rceil + 4. \tag{63}$$

For a Clifford+T implementation of $U^{(2)}_{odd}$, we account for the CNOT counts of the Toffoli, CSWAP, and logical-AND implementations. The number of CNOT gates required is therefore at most

$$\begin{aligned} & 7 + 7 \cdot 2(n-1) + 6(n-1) + 6 \sum_{j=2}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j} (j-1) \\ & +9 \cdot 2 \left(\lceil \log n \rceil 2^{\lceil \log n \rceil} - \lceil \log n \rceil + 1 \right) \\ = \; & 21 \lceil \log n \rceil 2^{\lceil \log n \rceil} + 20n - 6 \cdot 2^{\lceil \log n \rceil} - 18 \lceil \log n \rceil + 11 \qquad \text{[Using Equation 51]} \\ \leq \; & 21 \lceil \log n \rceil 2^{\lceil \log n \rceil} + 14n - 18 \lceil \log n \rceil + 11 \end{aligned} \tag{64}$$

- **Qubit-count:** We require $2n$ input qubits to store u and v. We prepend $2\left(2^{\lceil \log n \rceil} - n\right)$ additional ancilla qubits, which return to their initial $|0\rangle$ states. We also require $(\lceil \log n \rceil - 1) + 1$ ancilla qubits to apply the MCX gates in step 2(d) and the C^3X gates in step 2(c). These ancillas are also restored to their initial states. The circuit leaves $\lceil \log n \rceil + 1$ qubits in non-restored states: qubit a and the $\lceil \log n \rceil$ qubits that store t. At the component level, the latter are output qubits, while a is a garbage qubit. When $U^{(2)}_{odd}$ is used within the GCD circuit, t is not part of the final GCD output, so all $\lceil \log n \rceil + 1$ non-restored qubits count as garbage. In summary, in addition to the $2n$ input qubits, we require $2^{\lceil \log n \rceil + 1} + \lceil \log n \rceil - 2n$ ancilla qubits, $\lceil \log n \rceil$ output qubits, and one garbage qubit; within the GCD circuit, the output and garbage qubits together are counted as $\lceil \log n \rceil + 1$ garbage qubits.

- **Toffoli-depth:** For a Clifford+Toffoli implementation, the Toffoli depth does not exceed the Toffoli count:

$$3 \lceil \log n \rceil 2^{\lceil \log n \rceil} + 4n - 4 \cdot 2^{\lceil \log n \rceil} + 1 \leq 3 \lceil \log n \rceil 2^{\lceil \log n \rceil} + 1. \tag{65}$$

- **T-depth:** Using the optimal T-depths of the Toffoli and logical-AND implementations, the T-depth of $\mathcal{C}^{(2)}_t$ is at most

$$\begin{aligned} & 3 + 3 \cdot 2(n-1) + 2(n-1) + 2 \sum_{j=2}^{\lceil \log n \rceil} \binom{\lceil \log n \rceil}{j} (j-1) \\ = \; & \lceil \log n \rceil 2^{\lceil \log n \rceil} + 8n - 2 \cdot 2^{\lceil \log n \rceil} - 3 \qquad \text{[Using Equation 51]} \\ \leq \; & \lceil \log n \rceil 2^{\lceil \log n \rceil} + 6n - 3. \end{aligned}$$

  Because two copies of $\mathcal{C}_{shift}$ are required, Equation 59 gives the following upper bound on the T-depth of $U^{(2)}_{odd}$:

$$\begin{aligned} & \lceil \log n \rceil 2^{\lceil \log n \rceil} + 8n - 2 \cdot 2^{\lceil \log n \rceil} - 3 + 2 \left(3 \lceil \log n \rceil 2^{\lceil \log n \rceil} - 3 \cdot 2^{\lceil \log n \rceil} + 3 \right) \\ = \; & 7 \lceil \log n \rceil 2^{\lceil \log n \rceil} + 8n - 8 \cdot 2^{\lceil \log n \rceil} + 3 \leq 7 \lceil \log n \rceil 2^{\lceil \log n \rceil} + 3 \end{aligned} \tag{66}$$

E Resource estimates for a quantum circuit computing the GCD

In this section, we compute the numbers of gates and qubits required to implement the quantum circuit in Figure 9 (Section 3.1), which computes the GCD of two n-bit integers. The pseudocode for the binary GCD algorithm used here is given in Algorithm 4.

A more detailed description of the quantum circuit and its constituent unitaries is given in Section 3.1. The variants of the GCD circuit obtained by using COMP-N-SUB circuits I, IIa, IIb, and III are called GCD-I, GCD-IIa, GCD-IIb, and GCD-III, respectively. Here, N_{iter} denotes the number of iterations of the loop in steps 5–17 and is at most $2n$.

- **Toffoli-count:** Let $\mathcal{T}^{of}(.)$ denote the Toffoli count of a unitary. The number of Toffoli gates required is at most

$$\begin{aligned}
& \mathcal{T}^{of}\left(U_{odd}^{(2)}\right) + \mathcal{T}^{of}\left(\mathcal{C}_{shift}\right) + N_{iter}\left(2\mathcal{T}^{of}(U_{odd}) + 2\mathcal{T}^{of}(\text{COMP-N-SUB})\right) \\
\leq\ & \left(3\lceil\log n\rceil 2^{\lceil\log n\rceil}+1\right) + \left(\lceil\log n\rceil 2^{\lceil\log n\rceil}+1\right) + 2N_{iter}\left(2\lceil\log n\rceil 2^{\lceil\log n\rceil} - 2n + 2\right) \\
& +2N_{iter}\mathcal{T}^{of}(\text{COMP-N-SUB}) \qquad \text{[Using the estimates in Appendix D]} \\
=\ & 4(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 4N_{iter}(n-1) + 2 + 2N_{iter}\mathcal{T}^{of}(\text{COMP-N-SUB}).
\end{aligned}$$

  Substituting the costs of the various COMP-N-SUB circuits from Section 2.2 gives the following upper bounds on the Toffoli counts of Clifford+Toffoli implementations of the GCD circuits.

$$\begin{aligned}
\mathcal{T}^{of}(GCD_I) &\leq 4(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 4N_{iter}(n-1) + 2 + 2N_{iter}(3n-1) \\
&= 4(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 2N_{iter}(n+1) + 2 \qquad (67) \\
\mathcal{T}^{of}(GCD_{IIa}) &= \mathcal{T}^{of}(GCD_{IIb}) \\
&\leq 4(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 4N_{iter}(n-1) + 2 + 2N_{iter}(11n-4) \\
&= 4(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 2N_{iter}(9n-2) + 2 \qquad (68) \\
\mathcal{T}^{of}(GCD_{III}) &\leq 4(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 4N_{iter}(n-1) + 2 + 2N_{iter}n \\
&= 4(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 2N_{iter}(n-2) + 2 \qquad (69)
\end{aligned}$$

- **T-count:** Let $\mathcal{T}(.)$ denote the T-count of a unitary. The number of T gates required is at most

$$\begin{aligned}
& \mathcal{T}\left(U_{odd}^{(2)}\right) + \mathcal{T}\left(\mathcal{C}_{shift}\right) + N_{iter}\left(2\mathcal{T}(U_{odd}) + 2\mathcal{T}(\text{COMP-N-SUB})\right) \\
\leq\ & \left(14\lceil\log n\rceil 2^{\lceil\log n\rceil}+15\right) + \left(6\lceil\log n\rceil 2^{\lceil\log n\rceil}+11\right) + 2N_{iter}\left(8\lceil\log n\rceil 2^{\lceil\log n\rceil} - 5n + 8\right) \\
& +2N_{iter}\mathcal{T}(\text{COMP-N-SUB}) \qquad \text{[Using the estimates in Appendix D]} \\
=\ & 4(4N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 2N_{iter}(5n-8) + 26 + 2N_{iter}\mathcal{T}(\text{COMP-N-SUB}).
\end{aligned}$$

  Substituting the costs of the various COMP-N-SUB circuits from Section 2.2 gives the following upper bounds on the numbers of T gates required by Clifford+T implementations of

the GCD circuits.

$$\begin{aligned}
\mathcal{T}(GCD_I) &\leq 4(4N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 2N_{iter}(5n-8) + 26 + 2N_{iter}(18n-4) \\
&= 4(4N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 2N_{iter}(13n+4) + 26 \quad (70) \\
\mathcal{T}(GCD_{IIa}) &\leq 4(4N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 2N_{iter}(5n-8) + 26 + 2N_{iter}(76n-21) \\
&\leq 4(4N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 142N_{iter}n \quad (71) \\
\mathcal{T}(GCD_{IIb}) &\leq 4(4N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 2N_{iter}(5n-8) + 26 + 2N_{iter}(77n-21) \\
&\leq 4(4N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 144N_{iter}n \quad (72) \\
\mathcal{T}(GCD_{III}) &\leq 4(4N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} - 2N_{iter}(5n-8) + 26 + 2N_{iter}\cdot 11n \\
&= 4(4N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 2N_{iter}(6n+8) + 26 \quad (73)
\end{aligned}$$

- **CNOT-count:** Let $\mathcal{N}_{Tof}^{cnot}(.)$ and $\mathcal{N}_{T}^{cnot}(.)$ denote the CNOT counts of a unitary implemented using the Clifford+Toffoli and Clifford+T gate sets, respectively. For a Clifford+Toffoli implementation, the number of CNOT gates required is at most

$$\begin{aligned}
& \mathcal{N}_{Tof}^{cnot}\left(U_{odd}^{(2)}\right) + \mathcal{N}_{Tof}^{cnot}\left(\mathcal{C}_{shift}\right) + N_{iter}\left(2\mathcal{N}_{Tof}^{cnot}(U_{odd}) + 2\mathcal{N}_{Tof}^{cnot}(\text{COMP-N-SUB})\right) \\
\leq\ & \left(3\lceil\log n\rceil 2^{\lceil\log n\rceil} + \lceil\log n\rceil + 4\right) + \left(2\lceil\log n\rceil 2^{\lceil\log n\rceil} + 2\right) \\
& + 2N_{iter}\left(3\lceil\log n\rceil 2^{\lceil\log n\rceil} + \lceil\log n\rceil + 4\right) + 2N_{iter}\mathcal{N}_{Tof}^{cnot}(\text{COMP-N-SUB}) \\
=\ & (6N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + (2N_{iter}+1)\lceil\log n\rceil + 6 + 8N_{iter} + 2N_{iter}\mathcal{N}_{Tof}^{cnot}(\text{COMP-N-SUB}).
\end{aligned}$$

Similarly, the number of CNOT gates required for a Clifford+T implementation is at most

$$\begin{aligned}
& \mathcal{N}_{T}^{cnot}\left(U_{odd}^{(2)}\right) + \mathcal{N}_{T}^{cnot}\left(\mathcal{C}_{shift}\right) + N_{iter}\left(2\mathcal{N}_{T}^{cnot}(U_{odd}) + 2\mathcal{N}_{T}^{cnot}(\text{COMP-N-SUB})\right) \\
\leq\ & \left(21\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 11\right) + \left(9\lceil\log n\rceil 2^{\lceil\log n\rceil} + 9\right) + 2N_{iter}\left(15\lceil\log n\rceil 2^{\lceil\log n\rceil} + 12\right) \\
& + 2N_{iter}\mathcal{N}_{T}^{cnot}(\text{COMP-N-SUB}) \\
=\ & 30(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 24N_{iter} + 20 + 2N_{iter}\mathcal{N}_{T}^{cnot}(\text{COMP-N-SUB}).
\end{aligned}$$

Substituting the costs of the different COMP-N-SUB variants gives the following upper bounds on the CNOT counts of Clifford+Toffoli implementations of the GCD circuits.

$$\begin{aligned}
\mathcal{N}_{Tof}^{cnot}(GCD_I) &\leq (6N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + (2N_{iter}+1)\lceil\log n\rceil + 6 + 8N_{iter} + 2N_{iter}\,(4n-5) \\
&\leq (6N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + (2N_{iter}+1)\lceil\log n\rceil + 8N_{iter}n + 6 \quad (74) \\
\mathcal{N}_{Tof}^{cnot}(GCD_{IIa}) &\leq (6N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + (2N_{iter}+1)\lceil\log n\rceil + 6 + 8N_{iter} + 2N_{iter}\,(6n-2) \\
&= (6N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + (2N_{iter}+1)\lceil\log n\rceil + 4N_{iter}(3n+1) + 6 \quad (75) \\
\mathcal{N}_{Tof}^{cnot}(GCD_{IIb}) &\leq (6N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + (2N_{iter}+1)\lceil\log n\rceil + 6 + 8N_{iter} + 2N_{iter}\,(8n-4) \\
&= (6N_{iter}+5)\lceil\log n\rceil 2^{\lceil\log n\rceil} + (2N_{iter}+1)\lceil\log n\rceil + 16N_{iter}n + 6 \quad (76)
\end{aligned}$$

Similarly, the following are upper bounds on the CNOT counts of Clifford+T implementations

of the GCD circuits.

$$\begin{aligned}
\mathcal{N}_T^{cnot}(GCD_I) &\leq 30(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 24N_{iter} + 20 + 2N_{iter}\,(25n-12) \\
&= 30(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 50N_{iter}n + 20 \qquad (77) \\
\mathcal{N}_T^{cnot}(GCD_{IIa}) &\leq 30(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 24N_{iter} + 20 + 2N_{iter}\,(83n-30) \\
&\leq 30(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 166N_{iter}n \qquad (78) \\
\mathcal{N}_T^{cnot}(GCD_{IIb}) &\leq 30(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 24N_{iter} + 20 + 2N_{iter}\,(85n-32) \\
&\leq 30(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 170N_{iter}n \qquad (79) \\
\mathcal{N}_T^{cnot}(GCD_{III}) &\leq 30(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 24N_{iter} + 20 + 2N_{iter}\,(19n-7) \\
&= 30(N_{iter}+1)\lceil\log n\rceil 2^{\lceil\log n\rceil} + 14n + 38N_{iter}n + 20 + 10N_{iter} \qquad (80)
\end{aligned}$$

- **Qubit-count:** We require $2n$ input qubits to store the two input integers.

  Let $\mathcal{Q}_g(.)$ denote the number of garbage qubits required to implement a unitary. The number of garbage qubits required to implement the GCD circuits is

$$\begin{aligned}
& \mathcal{Q}_g\left(U_{odd}^{(2)}\right) + \mathcal{Q}_g\left(\mathcal{C}_{shift}\right) + N_{iter}\left(2\mathcal{Q}_g(U_{odd}) + 2\mathcal{Q}_g(\text{COMP-N-SUB})\right) \\
= \;& (1+\lceil\log n\rceil) + 0 + N_{iter}\left(2(1+\lceil\log n\rceil)+1\right) \\
= \;& 2\lceil\log n\rceil N_{iter} + 3N_{iter} + \lceil\log n\rceil + 1.
\end{aligned}$$

  We require one garbage qubit to implement any variant of COMP-N-SUB. Next, let $\mathcal{Q}_a(.)$ denote the number of ancilla qubits required to implement a unitary. These qubits are returned to their initial states and can therefore be reused in later computations. Thus, the number of ancilla qubits required to implement the GCD circuits is

$$\begin{aligned}
& \max\left\{\mathcal{Q}_a\left(U_{odd}^{(2)}\right), \mathcal{Q}_a\left(\mathcal{C}_{shift}\right), 2\mathcal{Q}_a\left(U_{odd}\right), \mathcal{Q}_a\left(\text{COMP-N-SUB}\right)\right\} \\
= \;& \max\left\{\left(2\cdot 2^{\lceil\log n\rceil} + \lceil\log n\rceil - 2n\right), 0, \left(2\cdot 2^{\lceil\log n\rceil} + 2\lceil\log n\rceil - 2n - 2\right), \mathcal{Q}_a\left(\text{COMP-N-SUB}\right)\right\} \\
= \;& \max\left\{\left(2\cdot 2^{\lceil\log n\rceil} + 2\lceil\log n\rceil - 2n - 2\right), \mathcal{Q}_a\left(\text{COMP-N-SUB}\right)\right\}.
\end{aligned}$$

  Substituting the ancilla-qubit requirements of the various COMP-N-SUB unitaries gives the following requirements for the GCD circuits.

$$\begin{aligned}
\mathcal{Q}_a(GCD_I) &= 2\cdot 2^{\lceil\log n\rceil} + 2\lceil\log n\rceil - 2n - 2 \\
\mathcal{Q}_a(GCD_{IIa}) &= \max\left\{2\cdot 2^{\lceil\log n\rceil} + 2\lceil\log n\rceil - 2n - 2,\; 2n - \omega(n) - \lfloor\log n\rfloor - 1\right\} \\
\mathcal{Q}_a(GCD_{IIb}) &= \max\left\{2\cdot 2^{\lceil\log n\rceil} + 2\lceil\log n\rceil - 2n - 2,\; 3n - \omega(n) - \lfloor\log n\rfloor - 2\right\} \\
\mathcal{Q}_a(GCD_{III}) &= \max\left\{2\cdot 2^{\lceil\log n\rceil} + 2\lceil\log n\rceil - 2n - 2,\; n - 1\right\}.
\end{aligned}$$

  Let $\mathcal{Q}_T(.)$ denote the total qubit count. The total qubit counts of the GCD circuits are then

as follows.

$$\begin{aligned}
\mathcal{Q}_T(GCD_I) &= 2\cdot 2^{\lceil \log n\rceil} + 2\lceil \log n\rceil N_{iter} + 3N_{iter} + 3\lceil \log n\rceil - 1 && (81)\\
\mathcal{Q}_T(GCD_{IIa}) &= (2N_{iter}+1)\lceil \log n\rceil + 2n + 3N_{iter} + 1 \\
&\quad + \max\left\{2\cdot 2^{\lceil \log n\rceil} + 2\lceil \log n\rceil - 2n - 2,\ 2n - \omega(n) - \lfloor \log n\rfloor - 1\right\} && (82)\\
\mathcal{Q}_T(GCD_{IIb}) &= (2N_{iter}+1)\lceil \log n\rceil + 2n + 3N_{iter} + 1 \\
&\quad + \max\left\{2\cdot 2^{\lceil \log n\rceil} + 2\lceil \log n\rceil - 2n - 2,\ 3n - \omega(n) - \lfloor \log n\rfloor - 2\right\} && (83)\\
\mathcal{Q}_T(GCD_{III}) &= (2N_{iter}+1)\lceil \log n\rceil + 2n + 3N_{iter} + 1 \\
&\quad + \max\left\{2\cdot 2^{\lceil \log n\rceil} + 2\lceil \log n\rceil - 2n - 2,\ n - 1\right\}. && (84)
\end{aligned}$$

- **Toffoli-depth:** Let $\mathcal{T}_d^{of}(.)$ denote the Toffoli-depth of a unitary. The Toffoli-depth of the GCD circuits is at most

$$\begin{aligned}
& \mathcal{T}_d^{of}\left(U_{odd}^{(2)}\right) + \mathcal{T}_d^{of}\left(\mathcal{C}_{shift}\right) + N_{iter}\left(\mathcal{T}_d^{of}(U_{odd}) + 2\mathcal{T}_d^{of}(\text{COMP-N-SUB})\right)\\
\leq\ & \left(3\lceil \log n\rceil 2^{\lceil \log n\rceil} + 1\right) + \left(\lceil \log n\rceil 2^{\lceil \log n\rceil} + 1\right) + N_{iter}\left(2\lceil \log n\rceil 2^{\lceil \log n\rceil} - 2n + 2\right)\\
& + 2N_{iter}\mathcal{T}_d^{of}(\text{COMP-N-SUB})\\
\leq\ & 2(N_{iter}+2)\lceil \log n\rceil 2^{\lceil \log n\rceil} - 2N_{iter}(n-1) + 2 + 2N_{iter}\mathcal{T}_d^{of}(\text{COMP-N-SUB}).
\end{aligned}$$

For a Clifford+Toffoli implementation using the COMP-N-SUB circuits in Section 2.2, the Toffoli-depths of the GCD circuits are bounded as follows.

$$\begin{aligned}
\mathcal{T}_d^{of}(GCD_I) &\leq 2(N_{iter}+2)\lceil \log n\rceil 2^{\lceil \log n\rceil} - 2N_{iter}(n-1) + 2 + 2N_{iter}\left(3n-1\right)\\
&= 2(N_{iter}+2)\lceil \log n\rceil 2^{\lceil \log n\rceil} + 4N_{iter}n + 2 && (85)\\
\mathcal{T}_d^{of}(GCD_{IIa}) &\leq 2(N_{iter}+2)\lceil \log n\rceil 2^{\lceil \log n\rceil} - 2N_{iter}(n-1) + 2 + 2N_{iter}\left(n + 4\lfloor \log n\rfloor + 8\right)\\
&= 2(N_{iter}+2)\lceil \log n\rceil 2^{\lceil \log n\rceil} + 2N_{iter}\left(4\lfloor \log n\rfloor + 9\right) + 2 && (86)\\
\mathcal{T}_d^{of}(GCD_{IIb}) &\leq 2(N_{iter}+2)\lceil \log n\rceil 2^{\lceil \log n\rceil} - 2N_{iter}(n-1) + 2 + 2N_{iter}\left(4\lfloor \log n\rfloor + 9\right)\\
&= 2(N_{iter}+2)\lceil \log n\rceil 2^{\lceil \log n\rceil} + 2N_{iter}\left(4\lfloor \log n\rfloor - n + 10\right) + 2 && (87)\\
\mathcal{T}_d^{of}(GCD_{III}) &\leq 2(N_{iter}+2)\lceil \log n\rceil 2^{\lceil \log n\rceil} - 2N_{iter}(n-1) + 2 + 2N_{iter}n\\
&= 2(N_{iter}+2)\lceil \log n\rceil 2^{\lceil \log n\rceil} + 2(N_{iter}+1) && (88)
\end{aligned}$$

- **T-depth:** Let $\mathcal{T}_d(.)$ denote the T-depth of a unitary. The T-depth of the GCD circuits is at most

$$\begin{aligned}
& \mathcal{T}_d\left(U_{odd}^{(2)}\right) + \mathcal{T}_d\left(\mathcal{C}_{shift}\right) + N_{iter}\left(\mathcal{T}_d(U_{odd}) + 2\mathcal{T}_d(\text{COMP-N-SUB})\right)\\
\leq\ & \left(7\lceil \log n\rceil 2^{\lceil \log n\rceil} + 3\right) + \left(3\lceil \log n\rceil 2^{\lceil \log n\rceil} + 3\right) + 4N_{iter}\lceil \log n\rceil 2^{\lceil \log n\rceil} + 2N_{iter}\mathcal{T}_d(\text{COMP-N-SUB})\\
=\ & 2\left(5 + 2N_{iter}\right)\lceil \log n\rceil 2^{\lceil \log n\rceil} + 6 + 2N_{iter}\mathcal{T}_d(\text{COMP-N-SUB}).
\end{aligned}$$

For a Clifford+T implementation using the various COMP-N-SUB circuits from Section 2.2,

the T-depths of the GCD circuits are bounded as follows.

$$\begin{aligned}
\mathcal{T}_d(GCD_I) &\leq 2\left(5+2N_{iter}\right)\lceil \log n \rceil 2^{\lceil \log n \rceil} + 6 + 2N_{iter}\left(9n-3\right) \\
&\leq 2\left(5+2N_{iter}\right)\lceil \log n \rceil 2^{\lceil \log n \rceil} + 18N_{iter}n \qquad (89) \\
\mathcal{T}_d(GCD_{IIa}) &\leq 2\left(5+2N_{iter}\right)\lceil \log n \rceil 2^{\lceil \log n \rceil} + 6 + 2N_{iter}\left(3n+12\lfloor \log n \rfloor + 24\right) \\
&= 2\left(5+2N_{iter}\right)\lceil \log n \rceil 2^{\lceil \log n \rceil} + 6N_{iter}\left(n+4\lfloor \log n \rfloor + 8\right) + 6 \qquad (90) \\
\mathcal{T}_d(GCD_{IIb}) &\leq 2\left(5+2N_{iter}\right)\lceil \log n \rceil 2^{\lceil \log n \rceil} + 6 + 2N_{iter}\left(12\lfloor \log n \rfloor + 27\right) \\
&= 2\left(5+2N_{iter}\right)\lceil \log n \rceil 2^{\lceil \log n \rceil} + 6N_{iter}\left(4\lfloor \log n \rfloor + 9\right) + 6 \qquad (91) \\
\mathcal{T}_d(GCD_{III}) &\leq 2\left(5+2N_{iter}\right)\lceil \log n \rceil 2^{\lceil \log n \rceil} + 6 + 2N_{iter}\left(5n\right) \\
&= 2\left(5+2N_{iter}\right)\lceil \log n \rceil 2^{\lceil \log n \rceil} + 10N_{iter}n + 6 \qquad (92)
\end{aligned}$$